

Amirkabir University of Technology
(Tehran Polytechnic)

Department of Electrical Engineering

Undergraduate Project - Instrumentation Course

Project Title:

Design and Implementation of a Data Acquisition (DAQ) System for a Xwing Quadcopter

Prepared By:

Amirali Oliaeimehr, Amirsaeed Pashaei, Seyed Mohammadamin Mirkazemi Moghadam

Supervisor:

Dr. Ahmad Afshar

July 2026

In the Name of God

Statement of Originality

We, **Amirali Oliaeimehr, Amirsaeed Pashaei, and Seyed Mohammadamin Mirkazemi Moghadam**, hereby declare that the contents of this project report are the result of our own original research conducted under the supervision of faculty members at Amirkabir University of Technology. All references to the work of others have been properly cited according to standard academic practices. This project has not been previously submitted for any comparable or higher academic degree.

Authors' Signatures

.....

Contents

1 Introduction to X-wing Quadqupter **10**

1.1 Equations Of Motion . . . 10
1.1.1 Rotational Kinematics and Euler Angles . . . 10
1.1.2 Translational Equations (Forces) . . . 10
1.1.3 Rotational Equations (Moments) . . . 10
1.1.4 Control Allocation (Mixer) . . . 11
1.2 How Motor Speed Makes Quad Change Direction . . . 11
1.3 Project Scope . . . 13
1.4 Explanation of Operational Modes and Operator Interaction in Automatic and Remote Modes. . . . 14
1.4.1 System Operational Modes . . . 14
1.4.2 Operator Interaction . . . 15
1.5 Safety and Security Requirements . . . 15
1.5.1 Safety Considerations . . . 15
1.5.2 Security Considerations . . . 16
1.6 Expected Errors and Disturbances . . . 16
1.6.1 Anticipated Disturbances . . . 17
1.6.2 Prioritization & System Response . . . 17
1.7 Solution to Error/Disturbances and Noise . . . 17
1.7.1 Sensor Noise Management . . . 17
1.7.2 Disturbance Rejection . . . 18
1.8 Analyzing and Proposing Solutions for Foreseeable Operational Defects(Fault in the System) . . . 18
1.8.1 Actuator Faults . . . 18
1.8.2 Sensor and Navigation Failures . . . 18
1.8.3 Communication and Power Failures . . . 19

2 Design of the Flight Controllers **20**

2.1 Chapter Introduction . . . 20
Nomenclature . . . 21
2.2 The Cascaded Control Architecture . . . 21
2.2.1 Motor-Mixing Algorithm . . . 21
2.2.2 Inner Loop — Attitude and Altitude . . . 22
2.2.3 Why an Outer Loop is Needed . . . 22
2.2.4 Frame Rotation of the Acceleration Command . . . 22
2.2.5 System Inputs and Outputs . . . 23
2.3 Design of the Inner-Loop Controllers . . . 23
2.3.1 Controller Design . . . 23

2.3.2 Implementation in Simulink . . . 24
2.3.3 Simulation . . . 25
2.4 Design of the Outer-Loop Controllers . . . 27
2.4.1 Controller Design . . . 27
2.4.2 Implementation in Simulink . . . 27
2.4.3 Simulation . . . 28
2.5 Disturbance Rejection: Wind and PID Tuning . . . 29
2.5.1 Final Benchmark — Square-Trajectory Tuning by RMSE . . . 30
2.6 Chapter Summary . . . 32

3 Sensing and State Estimation 33
3.1 A Look at the Necessary Sensors . . . 33
3.1.1 The GPS Sensor . . . 33
3.1.2 The IMU Sensor . . . 34
3.1.3 The Barometer . . . 34
3.1.4 The Ultrasonic Rangefinder (MB1240) . . . 34
3.1.5 The Magnetometer . . . 35
3.2 Redundancy . . . 36
3.3 The Simulink Sensor Subsystem . . . 37
3.3.1 Inputs and Signal Distribution . . . 37
3.3.2 The GPS Path and the LLA-to-Flat-Earth Conversion . . . 37
3.3.3 The Barometer Path . . . 39
3.3.4 The IMU Path . . . 39
3.3.5 The Magnetometer (Yaw) Path . . . 39
3.3.6 The Two Output Buses . . . 40
3.4 The Problems with the Sensors . . . 40
3.4.1 No Direct Sensor for Roll and Pitch . . . 40
3.4.2 Noise on the Sensors . . . 40
3.5 The Solution: The Extended Kalman Filter . . . 41
3.5.1 The Theory . . . 41
3.5.2 Updated Project Scope . . . 42
3.5.3 The Implementation . . . 42
3.6 Simulation Results of the Kalman Filter . . . 44
3.7 Chapter Summary . . . 48

4 The Conditioning Circuits 49
4.1 The Processing of the Sensors . . . 49
4.1.1 The I^2C Protocol . . . 50
4.1.2 The USART Protocol . . . 50
4.2 Circuit 1 — Power Supply Conditioning . . . 50
4.3 Circuit 2 — I^2C Pull-up Rise-Time Characterisation . . . 52
4.4 Circuit 3 — Multi-Device I^2C Bus Validation . . . 53
4.5 Circuit 4 — Logic-Level Translation . . . 55
4.6 Circuit 5 — GPS Reception and LCD Display (UART/NMEA) . . . 57
4.7 Circuit 6 — Analog Conditioning of the Rangefinder . . . 60
4.7.1 The Sensor and its Signal . . . 60
4.7.2 Design of the Anti-Aliasing Butterworth Filter . . . 60
4.7.3 Design of the Zero-Suppression and Span Circuit . . . 61

4.8 Chapter Summary . . . 63

5 Selecting the Required Actuators 64
5.1 Chapter Introduction . . . 64
5.2 Brushless DC Motors (BLDC) . . . 64
5.3 Propellers and Aerodynamic Linkage . . . 65
5.4 Electronic Speed Controllers (ESC) . . . 65
5.5 The Motor Mixer: Virtual to Physical Translation . . . 65
5.6 Chapter Summary . . . 66

6 Graphical User Interface (GUI) 67
6.1 Listener . . . 67
6.2 Designing the App . . . 67
6.2.1 The Design View . . . 67
6.2.2 The Code View . . . 67
6.3 The Run_CoSimulation File . . . 70

7 Hardware Implementation 74
7.1 The Circuit . . . 74
7.2 The Microcontroller Code . . . 76
7.3 The GUI and the Proteus Simulation . . . 76
7.3.1 GUI . . . 76
7.3.2 Proteus . . . 76
7.4 Bluetooth . . . 77

8 Innovation and Commercialisation 78
8.1 Introduction and Strategic Framing . . . 78
8.2 Fault-Aware Adaptive Estimation . . . 78
8.2.1 The Gap in a Conventional EKF . . . 79
8.2.2 The Four Documented Failure Modes . . . 79
8.2.3 Mathematical Development of the Detection Mechanism . . . 79
8.2.4 The Four Adaptive Responses . . . 83
8.2.5 Implementation, Line by Line . . . 83
8.2.6 Novelty and Honest Limitations . . . 86
8.3 Precision-Agriculture Monitoring Service . . . 87
8.3.1 The Venture in One Paragraph . . . 87
8.3.2 Market Sizing: the Iranian Pistachio Sector . . . 87
8.3.3 The Problem, Quantified: Water . . . 88
8.3.4 Technical Validation: the Science is Already Proven on this Crop, in Iran . . . 89
8.3.5 The Product: What the Customer Receives . . . 90
8.3.6 Service Delivery Workflow . . . 90
8.3.7 Unit Economics . . . 91
8.3.8 Phased Rollout . . . 93
8.3.9 Competitive Position and Defensibility . . . 94
8.3.10 Risk Register . . . 95
8.3.11 Gap Analysis: Current Platform versus Commercial Requirement . . . 96
8.4 Drone-in-a-Box Autonomous Site Monitoring . . . 97
8.4.1 The Concept . . . 97

8.4.2 Why This Fits the Delivered Platform Unusually Well 97
8.4.3 Target Sites 98
8.4.4 Why A1 is a Prerequisite, Not a Feature 98
8.4.5 Business Model and Remaining Gaps 99
8.5 The Integrated Strategy 100
8.6 Summary 100
8.7 Sources Consulted 101

9 Conclusion **102**

List of Figures

1.1 Mechanism of generating Thrust and Pitch torque in a standard cross configuration . . . 12
1.2 Mechanism of generating Roll and Yaw torques in a standard cross configuration . . . 13
1.3 Sequential Block Diagram of the Project Scope and Workflow . . . 13

2.1 The Control Architecture Diagram . . . 23
2.2 Single-channel PD implementation: the roll error is scaled by $K_p = 2.16$ and the measured rate by K_d, then combined to form u_2. . . . 25
2.3 Complete inner-loop controller: four PD channels (roll, pitch, yaw, altitude) fed by the reference inputs and the State_X bus, multiplexed into U_{vector}. . . . 25
2.4 Altitude step response to $z_{ref} = 1$ m: measured P.O. ≈ 4.1 %, $T_p \approx 0.918$ s (design targets 5 %, 1 s). . . . 26
2.5 Roll step response to $\phi_{ref} = 0.1$ rad at $t = 2$ s: measured P.O. $= 5$ %, $T_p = 0.29$ s. . . . 26
2.6 Full cascaded model: two sources drive the position controller, whose tilt commands feed the Attitude_Controller and the Newton–Euler plant; outputs are visualised on the XY graph. . . . 27
2.7 Interior of the position controller and the $R(\psi)$ Rotation block, which converts the global acceleration command into body-referenced tilt setpoints. . . . 28
2.8 Circular tracking (radius 5 m): after the acquisition transient, the vehicle follows the commanded circle in the XY plane. . . . 28
2.9 Corresponding $x(t)$ and $y(t)$ time histories — quadrature sinusoids of amplitude 5 m. . . . 29
2.10 Wind test: PID outer loop. The x channel is disturbed at $t \approx 10$ s; the integral term returns the position to its reference, whereas a PD loop would hold a steady offset. . . . 30
2.11 Square-trajectory tracking for the three PID variants, annotated with gains and RMSE ($3.891 \rightarrow 3.515 \rightarrow 3.244$ m). . . . 31
2.12 Wind robustness re-verified with the final coefficients ($K_p = 9$, $K_d = 5.4$, $K_I = 2$): the disturbance is again rejected to zero steady-state error. . . . 32

3.1 The Simulink sensor subsystem: the true state (input 1) is distributed to the GPS, barometer, IMU, and magnetometer models, producing the IMU_output and GPS&Magno_output measurement buses. . . . 37
3.2 Updated project scope block diagram. . . . 42

3.3 Roll: true ϕ (left) versus EKF-estimated ϕ (right). The unmeasured roll angle is reconstructed almost exactly. 45
3.4 Pitch: true θ (left) versus EKF-estimated θ (right). Like roll, pitch is recovered despite never being measured directly. 45
3.5 x position: true (left), raw GPS sensor spanning ±10 m of noise (centre), and EKF estimate tracking truth to a few tenths of a metre (right). 46
3.6 y position: true (left), raw sensor (centre), EKF estimate (right). Same one-to-two order-of-magnitude noise reduction as x. 46
3.7 z altitude: true, barometer sensor, and EKF estimate. The barometric measurement is already clean; all three agree closely on the 3 m set-point. 47
3.8 Yaw: true (left), magnetometer sensor (centre, ±0.1 rad noise), EKF estimate (right). The filter recovers the small true heading from the noisier magnetometer signal. 48

4.1 Circuit 1 — power-supply conditioning: ferrite bead, LC filter, LM1117 regulator, and decoupling. 51
4.2 Circuit 1 simulation: oscilloscope showing the noisy input rail (Channel A) versus the clean regulated output (Channel B). 51
4.3 Circuit 2 — I^2C pull-up test: open-drain driver, 1.8 kΩ pull-up, and 150 pF bus-capacitance model. 53
4.4 Circuit 3 — three 24C02 EEPROMs sharing the I^2C bus at addresses 0x50–0x52, with the microcontroller, pull-ups, and I^2C debugger. 54
4.5 Circuit 3 simulation: the I^2C scanner terminal output listing the three devices, and the I^2C-debugger transaction trace. 55
4.6 Circuit 4 — BSS138 bidirectional level shifter between the 5 V and 3.3 V I^2C rails. 56
4.7 Circuit 4 simulation: the 3.3 V side and 5 V side tracking each other in both translation directions. 56
4.8 Circuit 5 — GPS/UART interface: microcontroller, 16×2 LCD in 4-bit mode, and virtual terminal injecting NMEA sentences. 58
4.9 Circuit 5 simulation: the LCD displaying parsed latitude and longitude from an injected $GPGGA sentence (and the no-fix warning case). 58
4.10 Circuit 6 — MB1240 conditioning chain: input buffer, zero-suppression-and-span difference amplifier, and second-order Sallen-Key Butterworth low-pass filter. 62
4.11 Circuit 6 simulation: the raw sensor voltage (≈98–976 mV) and the conditioned output spanning the full 0–5 V ADC range; and the filter's frequency response showing −3 dB at the design cutoff. 63

7.1 The analog conditioning circuit and voltage divider. 74
7.2 The Proteus circuit using the COMPIM block. 77

List of Tables

2.1 Principal symbols used in this Chapter. 21
2.2 Input/output interface of the controllers and the overall system. 23
2.3 Consolidated inner-loop PD gains. 24
2.4 Square-trajectory tuning: three PID gain sets and their RMSE. 30

3.1 GP2Y0A21YK distance vs. output voltage 35
3.2 Table 3.1. Sensor suite: measured variables, chosen models, and full-scale ranges (Added). 36
3.3 Table 3.0a. The two measurement buses produced by the sensor subsystem. 40
3.4 Table 3.2. EKF covariance settings for the seven-measurement filter (Added). 44

4.1 Table 4.1. Measured/predicted I^2C rise time: the designed value passes; the common 4.7 kΩ default fails at 400 kHz on a three-device bus. 52
4.2 Arduino UNO to ATMEGA328P pin mapping 52
4.3 Table 4.2. Summary of the six conditioning circuits and their principal results. 63

8.1 The three proposals and their relationship to the project's existing capabilities. 78
8.2 The four estimator failure modes diagnosed during development, each of which A1 converts into a runtime fault detection. 79
8.3 Chi-squared thresholds by measurement dimension. The threshold is read from the distribution, not tuned. 81
8.4 The two complementary indicators. Together they cover both classes of estimator failure. 82
8.5 Mapping from detected fault to filter reconfiguration and vehicle-level fail-safe. 83
8.6 Iranian pistachio sector: scale and structure of the initial target market. . 87
8.7 The water crisis in the target region — the problem the service exists to address. 88
8.8 Sensing modalities and their function within the service product. 89
8.9 Service delivery workflow. Stages 2 and 4 are automated; stages 5 and 7 are where agronomic credibility is built. 91
8.10 Illustrative capital requirement for one survey unit. 91
8.11 Illustrative operating capacity of a single survey unit. 92
8.12 Illustrative customer return on the service fee. 93
8.13 Phased rollout, structured so that each phase de-risks a distinct assumption before capital is committed to the next. 93
8.14 Competitive landscape and the basis of defensibility. 94
8.15 Principal risks and mitigations. 95

8.16 Gap between the delivered project and the commercial requirement. 96
8.17 Constraint-to-advantage mapping: the mission is chosen so that the platform's limitations are irrelevant and its strengths are decisive. 97
8.18 Candidate deployment sites, ordered by fit to the platform's current capability. 98
8.19 Gaps between the current capability and a deployable drone-in-a-box product. 99
8.20 The three proposals as a sequenced strategy. 100

Chapter 1

Introduction to X-wing Quadqupter

1.1 Equations Of Motion

To accurately model the flight dynamics, the system is analyzed using two primary reference frames: the earth-fixed Inertial Reference Frame (R) and the Body-Fixed Frame (R_3) originating at the center of gravity (CG) of the drone.

1.1.1 Rotational Kinematics and Euler Angles

The rotation from the inertial frame to the body frame is defined by three successive Euler angle rotations: Roll (ϕ), Pitch (θ), and Yaw (ψ). The absolute angular velocity of the drone in the body frame ($\vec{\omega}^{R_3} = p\hat{i}_3 + q\hat{j}_3 + r\hat{k}_3$) is related to the Euler angle rates as follows:

$$p = \dot{\phi} - \dot{\psi}\sin\theta$$
$$q = \dot{\theta}\cos\phi + \dot{\psi}\cos\theta\sin\phi$$
$$r = -\dot{\theta}\sin\phi + \dot{\psi}\cos\theta\cos\phi$$

1.1.2 Translational Equations (Forces)

Based on Newton's second law, and accounting for the gravitational and thrust forces, the translational equations of motion in the body-fixed frame are derived as:

- **X-axis:**
$$mg\sin\theta = m[\dot{u} + wq - vr]$$
- **Y-axis:**
$$-mg\cos\theta\sin\phi = m[\dot{v} - wp + ur]$$
- **Z-axis:**
$$T - mg\cos\theta\cos\phi = m[\dot{w} + vp - uq]$$

1.1.3 Rotational Equations (Moments)

Assuming a symmetric structure for the drone where the products of inertia are zero, the inertia matrix is diagonal, defined as $I = \text{diag}(I_{xx}, I_{yy}, I_{zz})$. The Euler equations governing the rotational dynamics are:

- **Roll Moment:**
$$\tau_\phi = \bar{I}_{xx}\dot{p} + qr(\bar{I}_{zz} - \bar{I}_{yy})$$
- **Pitch Moment:**
$$\tau_\theta = \bar{I}_{yy}\dot{q} + pr(\bar{I}_{xx} - \bar{I}_{zz})$$
- **Yaw Moment:**
$$\tau_\psi = \bar{I}_{zz}\dot{r} + pq(\bar{I}_{yy} - \bar{I}_{xx})$$

1.1.4 Control Allocation (Mixer)

The mapping between the control outputs (u_1, u_2, u_3, u_4) and the squared motor speeds (ω_i^2) is determined by the mixer matrix. For the baseline four-rotor configuration provided, the relationships are (Note: This must be updated for the specific Y4 tail configuration):

- **Total Thrust (u_1):**
$$b(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2)$$
- **Roll Command (u_2):**
$$bl(\omega_4^2 - \omega_2^2)$$
- **Pitch Command (u_3):**
$$bl(\omega_3^2 - \omega_1^2)$$
- **Yaw Command (u_4):**
$$d(\omega_2^2 + \omega_4^2 - \omega_1^2 - \omega_3^2)$$

1.2 How Motor Speed Makes Quad Change Direction

The flight dynamics and directional control of a quadcopter are achieved by imposing precise variations in the rotational speeds of its motors (Ω) relative to the nominal hover speed (Ω_H). In flight control systems, the mapping between the virtual control inputs (U_1 to U_4) and the actual motor speeds (ω_i) is defined by the Control Allocation Matrix, commonly referred to as the Mixer.

The standard cross (or x-wing) configuration quadcopter features a symmetric architecture. For this mathematical derivation based on the provided matrix, we assume motor 1 is located at the front, motor 2 on the right, motor 3 at the rear, and motor 4 on the left. Motors 1 and 3 rotate in one direction, while motors 2 and 4 rotate in the opposite direction to cancel out reactive torques in hover. The maneuvering mechanics in this architecture are as follows:

- **Altitude Control (Total Thrust - U_1):** To ascend along the $+z_B$ axis, the speed of all motors is proportionally increased so that the total generated thrust overcomes the gravitational force (mg). The total thrust is the scalar sum of the lift generated by all four rotors:
$$U_1 = k_f(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2)$$
where k_f is the aerodynamic lift coefficient of the propellers.

- **Roll Control (U_2):** The roll maneuver (rotation around the x_B axis) is executed exclusively by creating a thrust differential between the lateral motors (left and right). Since the front and rear motors lie directly on the longitudinal axis, they generate zero roll moment. By increasing the speed of the left motor (motor 4) and decreasing the right motor's speed (motor 2), a roll torque is produced:

$$U_2 = -l_y k_f(\omega_2^2 - \omega_4^2)$$

where l_y is the lateral distance from the right/left motors to the center axis.

- **Pitch Control (U_3):** Pitch rotation (around the y_B axis) requires a thrust imbalance between the longitudinal motors (front and rear). Since the left and right motors lie on the lateral axis, they generate zero pitch moment. This moment is generated by modulating the speed of the front motor against the rear motor:

$$U_3 = -l_{fx} k_f(\omega_1^2) + l_{rx} k_f(\omega_3^2)$$

where l_{fx} and l_{rx} are the longitudinal distances from the Center of Gravity (CG) to the front and rear motors, respectively.

- **Yaw Control (U_4):** Yaw rotation exploits the reactive drag torque of the propellers. A pure yawing moment around the z_B axis is created by imposing a speed differential between the motors rotating clockwise and those rotating counter-clockwise. This provides crisp yaw authority without altering the net vertical thrust:

$$U_4 = -k_m(\omega_1^2 - \omega_2^2 + \omega_3^2 - \omega_4^2)$$

where k_m is the aerodynamic drag torque coefficient.

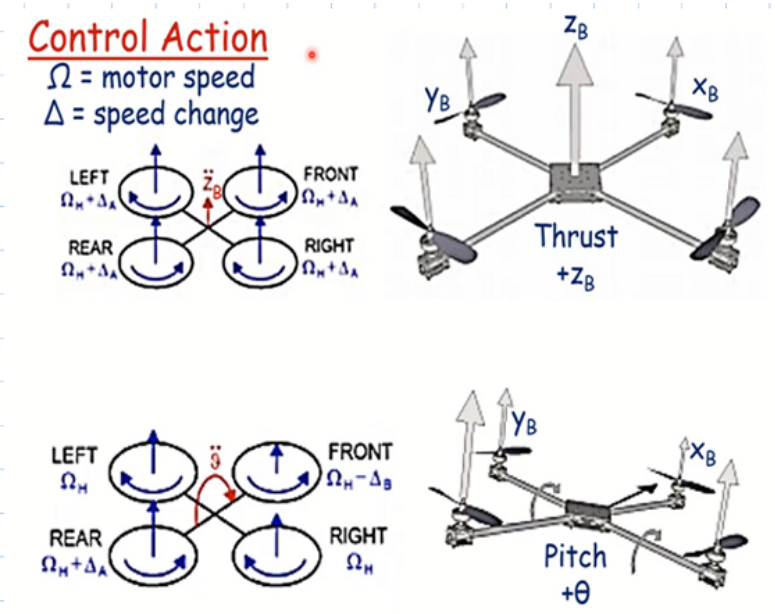


Figure 1.1: Mechanism of generating Thrust and Pitch torque in a standard cross configuration

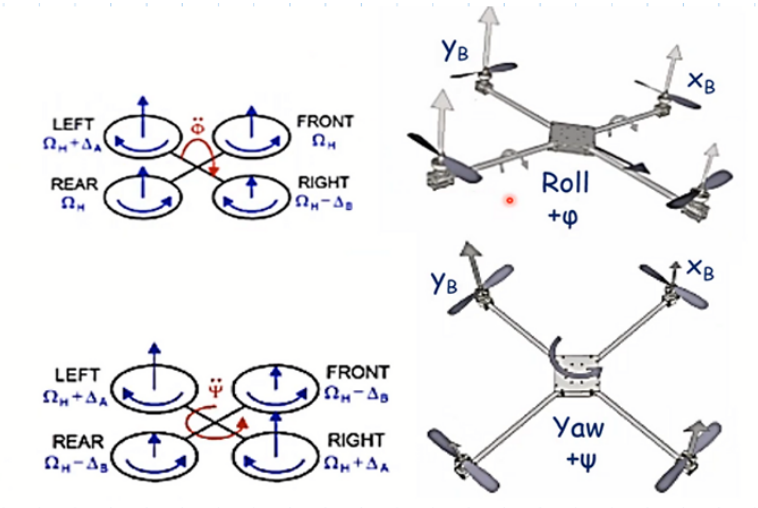


Figure 1.2: Mechanism of generating Roll and Yaw torques in a standard cross configuration

By synthesizing the aforementioned equations, the linearized Mixer Matrix for the standard cross configuration is derived. This matrix is directly implemented in the design of the inner-loop attitude controller:

$$\begin{bmatrix} U_1 \\ U_2 \\ U_3 \\ U_4 \end{bmatrix} = \begin{bmatrix} k_f & k_f & k_f & k_f \\ 0 & -l_y k_f & 0 & l_y k_f \\ -l_{fx} k_f & 0 & l_{rx} k_f & 0 \\ -k_m & k_m & -k_m & k_m \end{bmatrix} \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$

1.3 Project Scope

Based on a systematic engineering development approach, the scope of this project is defined as an integrated workflow that outlines the transition from theoretical design to final implementation. The sequential block diagram of this process, arranged in two continuous lines, is illustrated below:

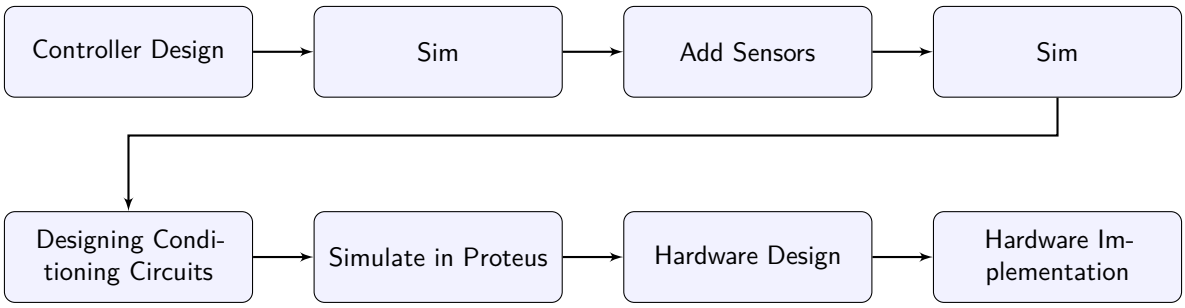


Figure 1.3: Sequential Block Diagram of the Project Scope and Workflow

As depicted in the diagram, the execution of the project follows a continuous path divided into three primary phases:

- **Controller Design and Dynamic Simulation:** The process initiates with the Controller Design to handle the flight dynamics of the xwing quadcopter. Utilizing

the Newton-Euler equations of motion and Euler angles, a cascaded architecture featuring an inner loop for attitude control and an outer loop for position tracking is developed. The system is then simulated (Sim). Subsequently, mathematical models of measurement sensors (Add Sensors) are introduced into the loop to account for real-world noise. The system undergoes a second simulation (Sim) to evaluate the robustness of the filtering algorithms and control responses.

- **Signal Conditioning and Electronic Simulation:** Following the validation of the control algorithms, the workflow transitions to the second line of the flowchart. Because raw sensor signals are often inadequate for direct microcontroller processing, specific signal Conditioning Circuits are designed. These circuits—comprising hardware filters, amplifiers, and level shifters—are then modeled and validated (Simulate in Proteus) to ensure reliable electronic functionality prior to fabrication.

- **Hardware Design and Implementation:** Finally, guided by the simulation results, the Hardware Design is finalized, encompassing the optimal selection of microcontrollers, sensors, and actuator driving interfaces. The workflow culminates in Hardware Implementation, which involves physical assembly on the drone frame, sensor integration, and real-world functional testing of the Data Acquisition (DAQ) system.

1.4 Explanation of Operational Modes and Operator Interaction in Automatic and Remote Modes.

The safe and optimal operation of the xwing quadcopter during Data Acquisition (DAQ) missions necessitates a precise definition of flight modes and the level of human intervention within the control loop. The Flight Control System (FCS) is architected to support three distinct operational modes:

1.4.1 System Operational Modes

- **Manual / Remote Mode:** In this mode, the operator utilizes a Radio Controller (RC) to send direct attitude and thrust commands (Roll, Pitch, Yaw, and Throttle) to the aircraft. The inner-loop controller is responsible for attitude stabilization, but the outer position loop is effectively closed by the human operator.

- **Automatic / Autonomous Mode:** Under this condition, the autopilot assumes full authority over both the inner and outer control loops. The quadcopter can independently navigate through predefined waypoints, maintain a specific Position Hold, and execute the DAQ operations automatically without any manual stick input.

- **Local / GCS Mode:** This mode involves the interaction between the drone and the Ground Control Station (GCS) software. High-level supervisory commands—such as Return to Launch (RTL), auto-land, or mission modifications—are transmitted and executed via a telemetry datalink.

1.4.2 Operator Interaction

The operator's physical and cognitive interaction varies significantly depending on the active flight mode:

- **Interaction in Remote Mode:** The operator serves a continuous, direct control role (Human-in-the-loop). This requires maintaining Visual Line of Sight (VLOS) and providing rapid manual responses to counteract environmental disturbances like wind gusts. The operator's cognitive load is at its peak.
- **Interaction in Automatic Mode:** The operator's role shifts from an active controller to a Supervisory Role. Through the Graphical User Interface (GUI) at the GCS, the operator monitors vital telemetry data such as battery voltage, signal integrity, and real-time sensor outputs. The most critical responsibility in this mode is maintaining situational awareness to execute a manual takeover (fail-safe override) should a software anomaly or positional sensor failure (e.g., GPS degradation) occur.

1.5 Safety and Security Requirements

Implementing a Data Acquisition (DAQ) and flight control system for the xwing quadcopter necessitates strict adherence to safety and security requirements within the cascade control structure. These protocols ensure data integrity, prevent physical damage, and maintain stability during critical conditions.

1.5.1 Safety Considerations

Safety considerations are defined to prevent systematic and physical failures:

- **Sensor Health & Redundancy:** Pre-flight checks for the IMU, altitude, and position sensors are vital to evaluate noise and calibration states. Advanced estimators must reject invalid sensor data to maintain stability.
- **Reference Frame Conversion:** Strict validation of ground-to-body frame transformations is required. To prevent mathematical singularities such as Gimbal Lock during aggressive maneuvers, quaternions are often preferred over Euler angles.
- **Command Limitation & Actuator Saturation:** Controller outputs must be rigorously saturated before reaching the Motor Mixer Algorithm (MMA) to prevent generating unrealistic commands for the motors. If the controller commands forces beyond the physical capabilities of the propulsion system, the motors enter a state of saturation. In this state, not only do the motors draw excessive stall currents (leading to rapid battery depletion and ESC overheating), but the system also loses its control degrees of freedom, rendering it incapable of executing stabilization maneuvers. Implementing anti-windup mechanisms in the integral branch of the controller ensures that actuator saturation does not lead to integral accumulation and subsequent loss of control.
- **Motor & Propeller Health:** Since the MMA delivers the final commands to the four motors, the failure of any motor or propeller causes a severe imbalance in

torques. Pre-flight physical inspections must verify the correct rotation direction of the motors, the dynamic balance of the propellers to mitigate harmonic vibrations, and the structural integrity of the frame joints. Early detection of thrust imbalances can prevent catastrophic failures.

- **Power Safety & Energy Management:** Voltage drops can cause both inner and outer loop failures because the controller's response relies heavily on stable motor performance. To prevent unexpected power depletion, a **Supervisory Control System** is implemented for energy optimization. This system continuously monitors the battery's State of Charge (SoC), voltage, current draw, and temperature. It dynamically calculates the energy required to return to the launch point (factoring in distance and wind speed) to optimize the flight path. Furthermore, to prevent the deep discharge of LiPo batteries, strict thresholds are defined for low-battery warnings, autonomous Return to Launch (RTL), and ultimately, in-place Auto-Land sequences. Additionally, utilizing separate isolated voltage regulators (BECs) for the processing system and motors prevents microcontroller resets induced by sudden motor voltage spikes.

- **Environmental Conditions:** Factors such as crosswinds, turbulence, moisture, dust, and environmental noise can cause errors in attitude estimation and reduce the accuracy of the position control loop. Missions must be executed strictly within allowable environmental limits. In the controller design, specific emphasis is placed on wind disturbance rejection capabilities to ensure the UAV maintains its trajectory amidst atmospheric turbulence.

- **Emergency States Logic:** Systems require robust fail-safes like emergency landing or auto-return mechanisms. These include Return to Launch (RTL) upon signal loss and automatic motor kill-switches upon detecting an unrecoverable flip.

1.5.2 Security Considerations

This section focuses on protecting communication links and software integrity:

- **Data Link Protection:** Wireless links must be protected against eavesdropping and fake command injection. Utilizing encrypted telemetry protocols (e.g., authenticated MAVLink) mitigates hijacking risks.

- **Access Control:** Controller gains and limits must be restricted to authorized users to prevent unauthorized configuration changes.

- **Software Security & Computational Stability:** The code must resist invalid data, noise, and numerical errors. Utilizing watchdogs is critical to autonomously reset the microcontroller in the event of a software freeze.

1.6 Expected Errors and Disturbances

The dynamic performance of quadcopters is continuously subjected to internal (systematic) and external (environmental) disturbances. Accurately identifying, classifying, and prioritizing these errors forms the foundation for designing a robust control system.

1.6.1 Anticipated Disturbances

Disturbances can enter through sensors, coordinate conversions, controllers, the MMA, motors, or the environment:

- **Sensor Errors:** Issues include high-frequency noise, gyroscope drift over time, feedback loss, and miscalibration (e.g., hard/soft iron effects on the magnetometer).
- **Frame Conversion:** Problems may arise from incorrect angles or matrix misalignments when projecting vectors fromurbances include voltage drops, electrical noise, overheating, wind, turbulence, and electromagnetic interference (EMI).

1.6.2 Prioritization & System Response

Based on their impact on flight stability, disturbances are classified into three severity levels:

- **High Priority (Critical):** Motor/propeller failures, severe voltage drops, main sensor loss, frame conversion errors, and loop instability fall into this category because they can cause catastrophic crashes. The system's response must involve immediate emergency termination, such as triggering a motor kill-switch or deploying a parachute.
- **Medium Priority (Degradation):** Sensor noise, slight actuation delays, and moderate wind are categorized here as they impair flight performance. The flight controller must compensate through robust filtering or initiate an autonomous return-to-launch sequence if positional sensors are compromised.
- **Low Priority (Nuisance):** UI delays, logging errors, and short-term transient oscillations are low priority because they are easily correctable by the feedback loop without threatening flight safety.

1.7 Solution to Error/Disturbances and Noise

The presence of sensor noise and aerodynamic disturbances are inherent in flight control systems. To guarantee the stability of the xwing quadcopter, combined strategies are implemented at both the signal filtering and control algorithm levels:

1.7.1 Sensor Noise Management

- **Extended Kalman Filter (EKF):** Given the nonlinear nature of flight kinematics, an EKF is employed for robust sensor fusion. By utilizing the mathematical model of the UAV, the EKF heavily attenuates high-frequency noise (e.g., accelerometer vibrations) and low-frequency gyro drift, providing an optimal estimation of the Euler angles.
- **Outlier Rejection:** Within the EKF framework, by analyzing the innovation covariance matrix, measurement deviations that exceed defined thresholds are identified as impulse noise and rejected from the estimation update step.

- **Low-Pass Filters (LPF):** Applying software-based IIR or FIR filters to raw IMU signals before they enter the controller is essential to eliminate high-frequency harmonics induced by motor rotations.

1.7.2 Disturbance Rejection

- **Robust Control Design:** Environmental disturbances such as wind gusts are modeled as bounded uncertainties. By precisely tuning the derivative gains in the inner-loop controller, the system's bandwidth is expanded, allowing the aircraft to rapidly generate counter-torques to reject wind disturbances.
- **Coaxial Interference Compensation:** In the xwing architecture, the tail motors are coaxially mounted. The aerodynamic downwash from the top propeller degrades the efficiency of the bottom propeller. This structural disturbance is compensated by introducing an aerodynamic correction factor into the Mixer Matrix for the lower motor, ensuring a linear thrust distribution.

1.8 Analyzing and Proposing Solutions for Foreseeable Operational Defects(Fault in the System)

The occurrence of technical faults during flight, especially in Unmanned Aerial Vehicles (UAVs), is inevitable. Designing a Fault-Tolerant System requires a rigorous analysis of failure points and the implementation of robust compensatory strategies.

1.8.1 Actuator Faults

Fundamentally, quadcopters cannot survive a complete motor or ESC failure due to the lack of true actuator redundancy. However, the xwing architecture presents unique characteristics:

- **Front Motor Failure:** If one of the front arms fails, the system instantly loses torque symmetry and flips. The solution involves rapid fault detection via the IMU, immediately triggering a motor kill-switch and deploying an emergency parachute to mitigate impact.
- **Coaxial Tail Analytical Redundancy:** If one of the two coaxially mounted tail motors fails, the remaining motor can still generate critical lift to prevent a free-fall. The Fault-Tolerant Control (FTC) algorithm rapidly reconfigures the Control Allocation (Mixer) matrix, utilizing the excess torque from the front motors to counter the induced yaw spin, thereby facilitating a controlled emergency landing.

1.8.2 Sensor and Navigation Failures

- **GPS Signal Loss:** In the event of GPS signal degradation (high HDOP) or complete loss, the control algorithm exits absolute position tracking. Relying on short-term estimation achieved by fusing IMU and Barometer data, the system initiates a safe hover mode until the signal is recovered or an auto-land command is executed.

- **Magnetometer Failure:** Severe electromagnetic interference can corrupt magnetometer readings. In such cases, the EKF dynamically reduces the magnetometer's measurement weight to zero and temporarily relies on gyro integration (Dead Reckoning) and kinematic models for yaw estimation.

1.8.3 Communication and Power Failures

- **Telemetry/RC Loss:** If the communication link between the Ground Control Station and the UAV is severed, a Fail-Safe protocol is triggered. The aircraft autonomously adjusts its altitude and executes a Return to Launch (RTL) sequence.
- **Critical Low Battery:** A dedicated power monitoring module continuously evaluates the battery voltage. Upon reaching a critical threshold, the flight controller automatically overrides the current mission and initiates either a Return to Home (RTH) or an immediate Auto-Land, depending on the current altitude and distance.

Chapter 2

Design of the Flight Controllers

2.1 Chapter Introduction

This chapter documents the design, implementation, and simulation of the flight-control system for a Vertical-Take-Off-and-Landing (VTOL) quadcopter. The vehicle is an under-actuated rigid body: it has six degrees of freedom (three translational, three rotational) but only four independent actuators (the four rotor speeds). Consequently, horizontal motion cannot be commanded directly — it can only be produced by tilting the airframe so that a component of the rotor thrust acts in the horizontal plane. This physical coupling is the central fact that shapes the entire control architecture.

The plant model used throughout is the nonlinear Newton–Euler model derived in the former chapter, summarised by the six coupled equations of motion [1]–[6] and the thrust relation $T = K(\omega_1^2 + \omega_2^2 + \omega_3^2 + \omega_4^2)$. The controller is organised as a two-level cascade: a fast inner loop that regulates attitude and altitude, and a slower outer loop that regulates horizontal position by generating the attitude references the inner loop tracks. Sections 2.1–2.3 develop this architecture, design each controller by pole placement, and validate it in Simulation. Section 2.4 introduces a wind disturbance, upgrades the outer loop to PID, and tunes the gains against a square-trajectory benchmark using RMSE.

Nomenclature

Symbol	Meaning	Value / Unit
m	Vehicle mass	1.2 kg
g	Gravitational acceleration	9.81 m/s^2
I_{xx}, I_{yy}	Roll / pitch moments of inertia	0.01 kg $\cdot$ m^2
I_{zz}	Yaw moment of inertia	0.02 kg $\cdot$ m^2
ϕ, θ, ψ	Roll, pitch, yaw (Euler angles)	rad
p, q, r	Body-frame angular rates	rad/s
u_1	Total thrust command	N
u_2, u_3, u_4	Roll, pitch, yaw torque commands	N $\cdot$ m
ω_i	Rotor angular speeds	rad/s
ℓ, K, b	Arm length, lift factor, drag factor	—
K_p, K_d, K_I	Proportional, derivative, integral gains	—
ω_n, ζ	Natural frequency, damping ratio	rad/s, —
T_p, P.O.	Peak time, percent overshoot	s, %

Table 2.1: Principal symbols used in this Chapter.

2.2 The Cascaded Control Architecture

A single controller that maps the desired trajectory directly to the four rotor speeds is extremely difficult to design, because the rotor speeds enter the dynamics nonlinearly and every output (position, attitude, altitude) depends on all four inputs simultaneously. The design is made tractable by decomposing it into stages that each solve one well-posed sub-problem. Two decompositions are used: a control-allocation stage (the motor-mixing algorithm) that separates the physically meaningful control efforts from the individual motor commands, and a cascade of an inner attitude loop inside an outer position loop.

2.2.1 Motor-Mixing Algorithm

Rather than command four motor speeds directly, the controller is designed to produce four physically meaningful quantities: total thrust u_1 and the three body torques u_2 (roll), u_3 (pitch), u_4 (yaw). A fixed linear map — the mixing matrix M — relates these efforts to the squared rotor speeds:

$$\begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix} = M \cdot \begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix}$$

with the rows of M formed from the lift factor, arm length, and drag factor.

$$M = \begin{bmatrix} b & b & b & b \\ 0 & -b\ell & 0 & b\ell \\ -b\ell & 0 & b\ell & 0 \\ -d & d & -d & d \end{bmatrix}$$

Because M is constant and invertible, the required rotor speeds are recovered by a single inversion,

$$\begin{bmatrix} \omega_1^2 \\ \omega_2^2 \\ \omega_3^2 \\ \omega_4^2 \end{bmatrix} = M^{-1} \cdot \begin{bmatrix} u_1 \\ u_2 \\ u_3 \\ u_4 \end{bmatrix}.$$

This is the key simplification of the actuator interface: the controller reasons in the intuitive space of thrust and torque, and the mixer translates that into motor commands. In the present simulation study the plant is driven directly in u-space, so M^{-1} is implicit; on hardware it becomes an explicit block between the controller and the electronic speed controllers.

2.2.2 Inner Loop — Attitude and Altitude

The inner loop is the fast core of the autopilot. It accepts four references — ϕ_{ref}, θ_{ref}, ψ_{ref} and the altitude reference z_{ref} — and produces the four control efforts u_1–u_4. Under the near-hover linearisation (small angles, hover thrust), the four attitude/altitude channels decouple: roll responds to u_2, pitch to u_3, yaw to u_4, and altitude to u_1. Each channel therefore reduces to an independent second-order plant that can be regulated by its own PD controller. This loop is what makes the vehicle stabilisable: it holds a commanded orientation and height. What it cannot do by itself is decide which orientation is needed to reach a horizontal target — that is the outer loop's task.

2.2.3 Why an Outer Loop is Needed

A human or a mission planner naturally specifies where the vehicle should be — a horizontal position (x_{ref}, y_{ref}) — not the precise roll and pitch angles required to get there. Yet for an under-actuated quadcopter, horizontal translation is produced only by tilting: from the translational equations of motion, the horizontal accelerations satisfy

$$\ddot{x} = (u_1/m)(\cos\phi \sin\theta \cos\psi + \sin\phi \sin\psi),$$
$$\ddot{y} = (u_1/m)(\cos\phi \sin\theta \sin\psi - \sin\phi \cos\psi).$$

Providing ϕ_{ref} and θ_{ref} by hand to achieve a desired position is therefore impractical — the mapping is nonlinear and coupled to yaw. The purpose of the outer loop is precisely to close this gap: it takes the position references (x_{ref}, y_{ref}), computes the horizontal acceleration required to drive the position error to zero, and converts that acceleration demand into the tilt references (ϕ_{ref}, θ_{ref}) that the inner loop then tracks. In short, the outer loop generates the attitude set-points that a pilot would otherwise have to guess.

2.2.4 Frame Rotation of the Acceleration Command

The position error is expressed in the global (inertial) frame, whereas roll and pitch tilt the vehicle about its own body axes. The two frames differ by the yaw angle ψ. A desired global horizontal acceleration must therefore be rotated by ψ before it can be interpreted as a body-axis tilt command; otherwise the vehicle would tilt in the wrong direction whenever its heading is non-zero. Linearising the translational equations about

hover ($u_1 \approx mg$, small angles) gives $\ddot{x} \approx g\theta$ and $\ddot{y} \approx -g\phi$, so the desired tilts follow from the desired accelerations through a $(1/g) \cdot R(\psi)$ mapping:

$$\begin{bmatrix} \theta_{des} \\ \phi_{des} \end{bmatrix} = \frac{1}{g} \cdot \begin{bmatrix} \cos\psi & \sin\psi \\ \sin\psi & -\cos\psi \end{bmatrix} \cdot \begin{bmatrix} \ddot{x}_{des} \\ \ddot{y}_{des} \end{bmatrix}$$

The block that performs this rotation is the interface between the two loops. It is important to state precisely what it does: it does not translate the position reference into a body-frame position; it rotates the horizontal acceleration command by the current heading ψ so that the resulting roll/pitch set-points point the vehicle correctly regardless of which way it faces. When $\psi = 0$ the rotation reduces to the identity and the coupling vanishes, which is why heading is held at zero in the nominal tests of this chapter.

2.2.5 System Inputs and Outputs

With the architecture established, the signal interface of the complete system can be stated. The nested structure is: mission references → outer loop → attitude references → inner loop → control efforts → mixer → rotor speeds → plant → state.

Block	**Inputs**	**Outputs**
Outer loop (position)	x_{ref}, y_{ref} (global); measured x, y, ψ	ϕ_{ref}, θ_{ref}
Inner loop (attitude/altitude)	$\phi_{ref}, \theta_{ref}, \psi_{ref}, z_{ref}$; measured state	u_1, u_2, u_3, u_4
Motor mixer	u_1, u_2, u_3, u_4	$\omega_1^2, \omega_2^2, \omega_3^2, \omega_4^2$
Plant (Newton–Euler)	rotor speeds (via thrust & torques)	12-state vector
Overall system	$x_{ref}, y_{ref}, z_{ref}, \psi_{ref}$	x, y, z, ψ (and full state)

Table 2.2: Input/output interface of the controllers and the overall system.

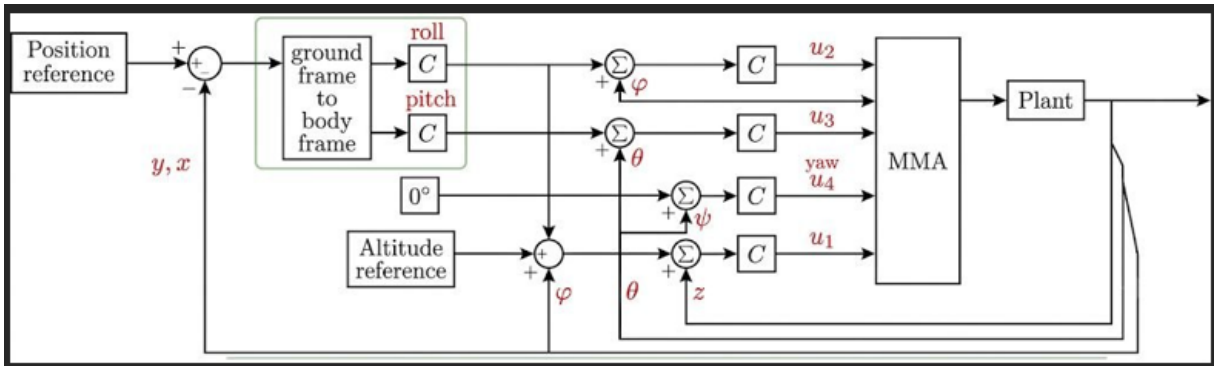


Figure 2.1: The Control Architecture Diagram

2.3 Design of the Inner-Loop Controllers

2.3.1 Controller Design

The inner loop is derived first. Under the hover linearisation — $\phi \approx 0, \theta \approx 0, \psi \approx 0$ and $p \approx q \approx r \approx 0$ — the roll equation of motion $\ddot{\phi} = [u_2 - qr(I_{zz} - I_{yy})]/I_{xx}$ simplifies to the double integrator

$$\ddot{\phi} = u_2/I_{xx} \Rightarrow G(s) = \phi(s)/u_2(s) = 1/(I_{xx}s^2).$$

The pitch and yaw channels are identical in form ($\ddot{\theta} = u_3/I_{yy}, \ddot{\psi} = u_4/I_{zz}$). A PD compensator $C(s) = K_d \cdot s + K_p$ is applied to each. Closing the loop on the double integrator gives the second-order transfer function

$$T(s) = \frac{K_d s + K_p}{I_{xx} s^2 + K_d s + K_p}.$$

Matching the denominator to the standard form $I_{xx}s^2 + K_d s + K_p$ yields the design equations $K_p = \omega_n^2 I_{xx}$ and $K_d = 2\zeta\omega_n I_{xx}$. Selecting a peak time $T_p = 0.3$ s and overshoot P.O. = 5 % gives $\zeta = 0.7$ and $\omega_n = 14.7$ rad/s, i.e. closed-loop poles at $s = -10.29 \pm j10.50$. Substituting the roll inertia:

$$K_p = (14.7)^2(0.01) = 2.16, \quad K_d = 2(0.7)(14.7)(0.01) = 0.21.$$

The roll and pitch controllers are thus $C(s) = 0.21s + 2.16$. The yaw channel uses $I_{zz} = 0.02$, which doubles both gains to give $C_{yaw}(s) = 0.42s + 4.32$.

The altitude channel is treated the same way but about a non-zero operating point. Linearising $\ddot{z} = -g + (u_1/m)\cos\phi\cos\theta$ at hover ($\cos\phi \approx \cos\theta \approx 1, u_1 = mg + \delta u_1$) gives $\delta\ddot{z} = \delta u_1/m$, i.e. $G(s) = 1/(ms^2)$. A smoother response is chosen here ($T_p = 1$ s, P.O. = 5 % $\rightarrow \omega_n = 4.4$ rad/s, $\zeta = 0.7$), giving

$$K_p = m\omega_n^2 = (1.2)(4.4)^2 = 23.2, \quad K_d = 2\zeta\omega_n m = 7.4, \quad C_{alt}(s) = 7.4s + 23.2,$$

with a feed-forward term $u_{1eq} = mg$ added so the controller only regulates the deviation from hover thrust.

Channel	**Plant**	ω_n **(rad/s)**	ζ	K_p	K_d
Roll (ϕ)	$1/(I_{xx}s^2)$	14.7	0.7	2.16	0.21
Pitch (θ)	$1/(I_{yy}s^2)$	14.7	0.7	2.16	0.21
Yaw (ψ)	$1/(I_{zz}s^2)$	14.7	0.7	4.32	0.42
Altitude (z)	$1/(ms^2)$	4.4	0.7	23.2	7.4

Table 2.3: Consolidated inner-loop PD gains.

2.3.2 Implementation in Simulink

Each PD controller is realised from primitive blocks. Following the identity $\delta = K_d \cdot \frac{d}{dt}(\phi_{ref} - \phi) + K_p(\phi_{ref} - \phi) = K_p(\phi_{ref} - \phi) - K_d \cdot \dot{\phi}$, the derivative of the reference is dropped and the measured body rate is used directly for the derivative term (recall $\dot{\phi} = p$ near hover). This avoids differentiating the reference signal and uses the gyro rate that is directly available in the state vector. The error $(\phi_{ref} - \phi)$ is scaled by K_p, the measured rate is scaled by K_d, and the two are subtracted to form the roll effort u_2. The pitch, yaw, and altitude channels replicate this structure, and their outputs are multiplexed into the control-effort vector $U_{vector} = [u_1, u_2, u_3, u_4]$.

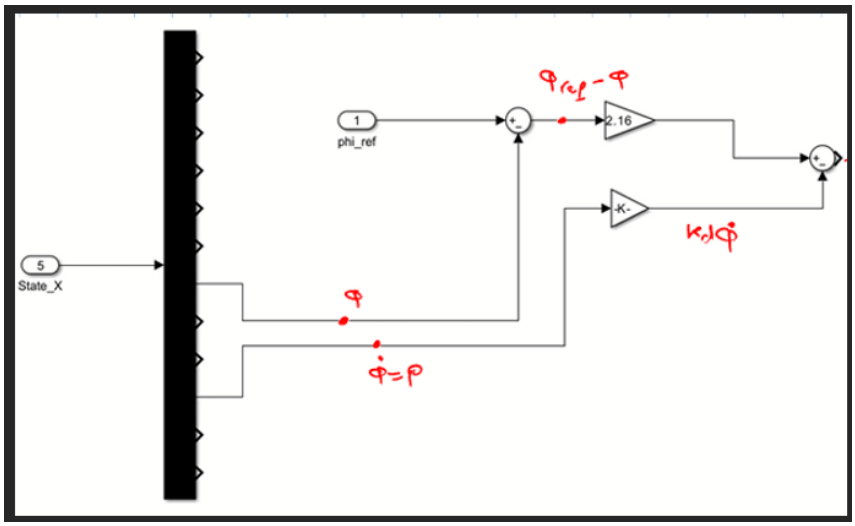


Figure 2.2: Single-channel PD implementation: the roll error is scaled by $K_p = 2.16$ and the measured rate by K_d, then combined to form u_2.

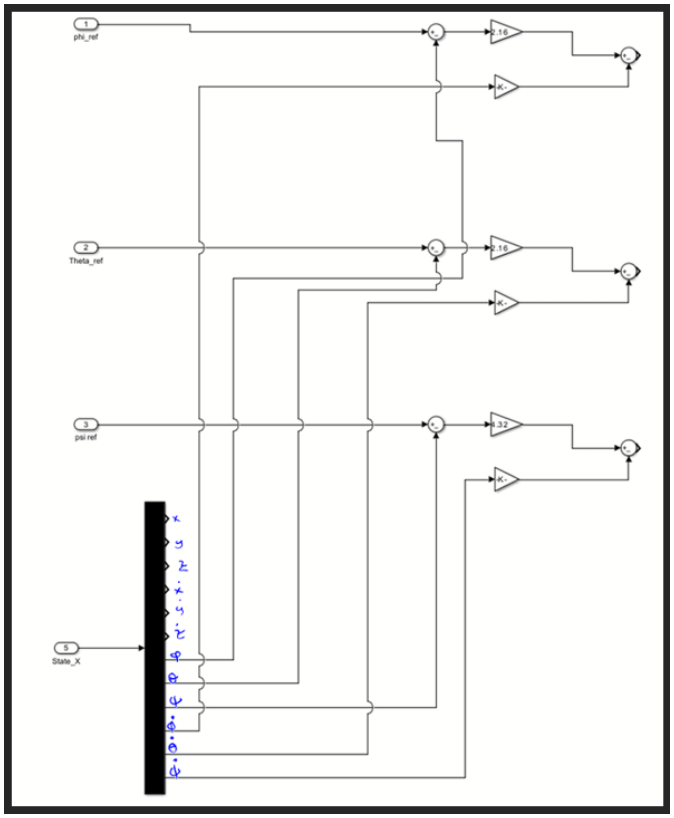


Figure 2.3: Complete inner-loop controller: four PD channels (roll, pitch, yaw, altitude) fed by the reference inputs and the State_X bus, multiplexed into U_{vector}.

2.3.3 Simulation

The inner loop is verified by commanding the altitude reference to $z_{ref} = 1$ m with the attitude references held at zero. The altitude response rises to the set-point with a small overshoot and settles cleanly. Reading the response: Percent overshoot: P.O.

$= (1.041 - 1)/1 \times 100 \approx 4.1$ % — consistent with the 5 % design target ($\zeta = 0.7$). Peak time: $T_p \approx 0.918$ s — consistent with the 1 s design target.

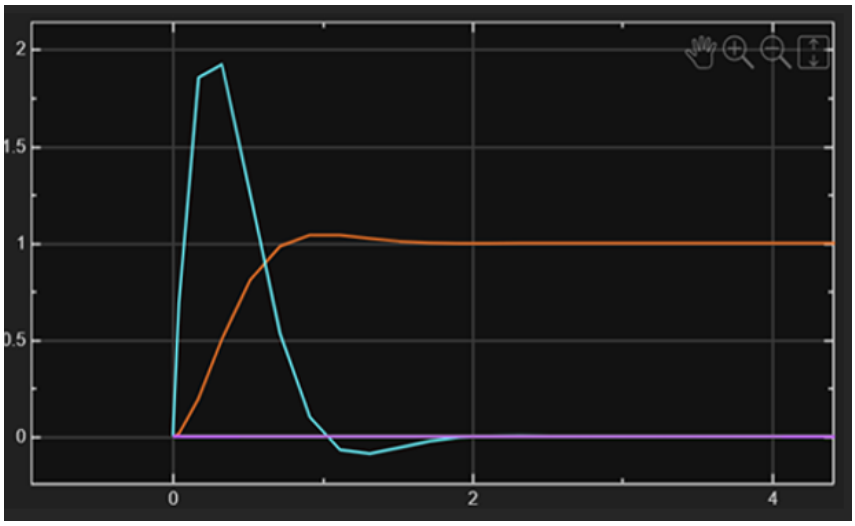


Figure 2.4: Altitude step response to $z_{ref} = 1$ m: measured P.O. ≈ 4.1 %, $T_p \approx 0.918$ s (design targets 5 %, 1 s).

The roll channel is checked separately with a step of $\phi_{ref} = 0.1$ rad ($\approx 5.7°$) applied at $t = 2$ s; because roll, pitch, and yaw share the same controller form, testing roll alone is sufficient. The measured response gives $T_p = 2.29 - 2 = 0.29$ s and P.O. $= (0.105 - 0.1)/0.1 \times 100 = 5$ %, both in close agreement with the design specification and confirming the pole-placement calculation.

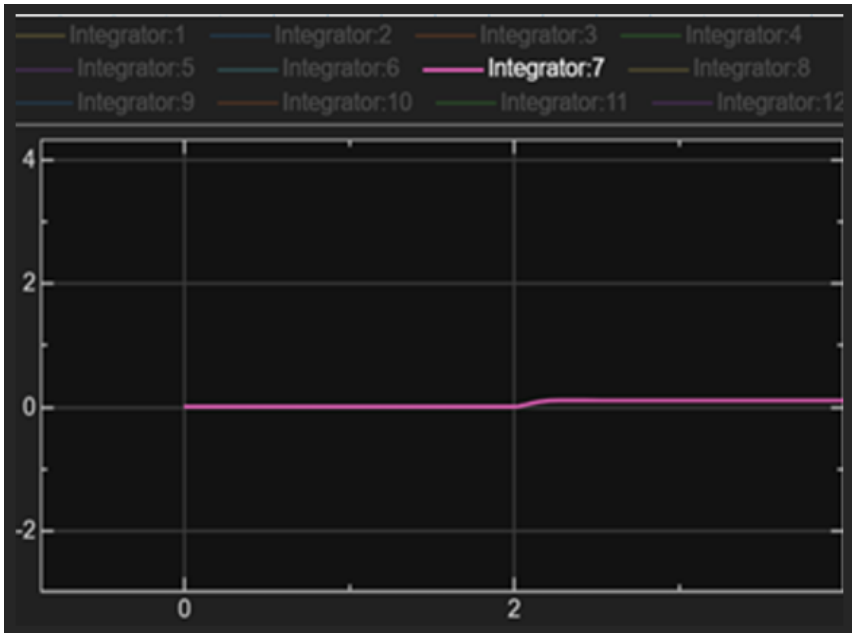


Figure 2.5: Roll step response to $\phi_{ref} = 0.1$ rad at $t = 2$ s: measured P.O. $= 5$ %, $T_p = 0.29$ s.

2.4 Design of the Outer-Loop Controllers

2.4.1 Controller Design

With the inner loop treated as an ideal, fast tilt-tracking element, the outer loop is designed on the horizontal-position dynamics. Assuming hover thrust and small angles, and taking heading $\psi = 0$ so that $R(\psi) = I$, the linearised horizontal dynamics reduce to

$$\ddot{x} = g\theta, \quad \ddot{y} = -g\phi.$$

The outer-loop controller produces a desired acceleration; the $(1/g) \cdot R(\psi)$ rotation block then converts it into the tilt commands $(\theta_{cmd}, \phi_{cmd})$. Because the g produced by the plant ($\ddot{x} = g\theta$) cancels the $1/g$ inside the rotation block, the effective plant seen by the outer-loop compensator is a pure double integrator,

$$G(s) = 1/s^2.$$

A PD compensator is again applied, giving the second-order closed loop $T(s) = (K_d s + K_p)/(s^2 + K_d s + K_p)$. To keep the outer loop well inside the inner loop's bandwidth (time-scale separation), a slow, well-damped response is chosen: $\zeta = 0.9$ and $\omega_n = 2$ rad/s. Hence

$$K_p = \omega_n^2 = 4, \quad K_d = 2\zeta\omega_n = 3.6, \quad C_x(s) = C_y(s) = 3.6s + 4.$$

2.4.2 Implementation in Simulink

The outer loop wraps around the inner loop. The position controller subsystem takes x_{ref}, y_{ref} and the measured position, forms the PD acceleration command for each horizontal axis, and passes the pair into the Rotation block. That block implements the $(1/g) \cdot R(\psi)$ transformation — building the rotation matrix from $\cos\psi$ and $\sin\psi$ and applying it to the two acceleration inputs $(a_{x,inertial}, a_{y,inertial})$ — and outputs theta_cmd and phi_cmd, which become the references for the inner loop. Saturation blocks limit the commanded tilt to keep the vehicle within the small-angle regime for which the design is valid.

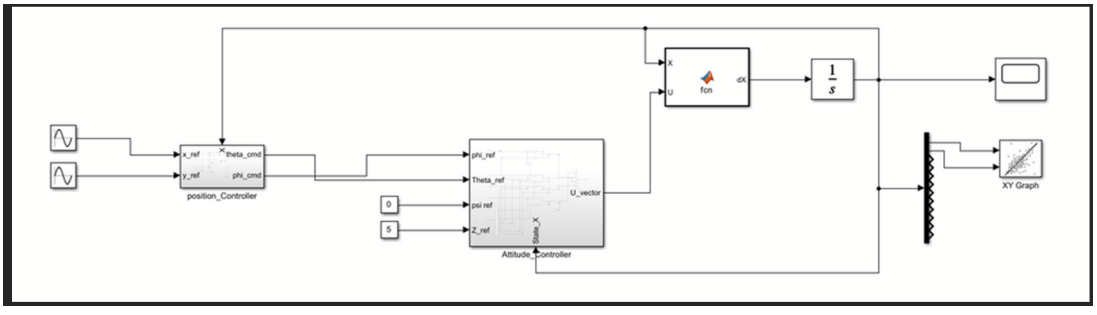


Figure 2.6: Full cascaded model: two sources drive the position controller, whose tilt commands feed the Attitude_Controller and the Newton–Euler plant; outputs are visualised on the XY graph.

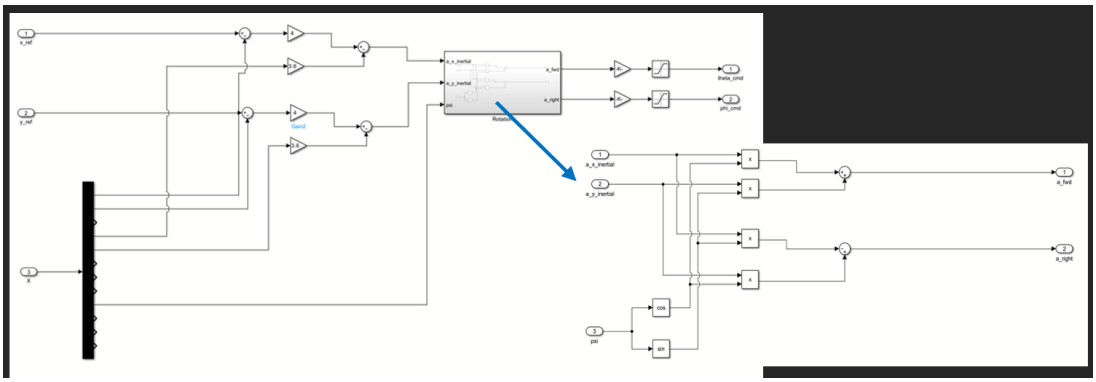


Figure 2.7: Interior of the position controller and the $R(\psi)$ Rotation block, which converts the global acceleration command into body-referenced tilt set-points.

2.4.3 Simulation

Circular-trajectory tracking is used to validate the complete cascade. The references are set to

$$x_{ref} = 5\sin(2\pi \cdot 0.5 \cdot t), \quad y_{ref} = 5\cos(2\pi \cdot 0.5 \cdot t),$$

which command the vehicle to follow a circle of radius 5 m. After an initial transient in which the drone acquires the path from the origin, the XY plot shows the trajectory settling onto the commanded circle and tracking it continuously. The time histories of $x(t)$ and $y(t)$ are the expected pair of quadrature sinusoids of amplitude 5, confirming that the outer loop correctly generates the tilt references needed to produce the circular motion and that the two loops interact stably.

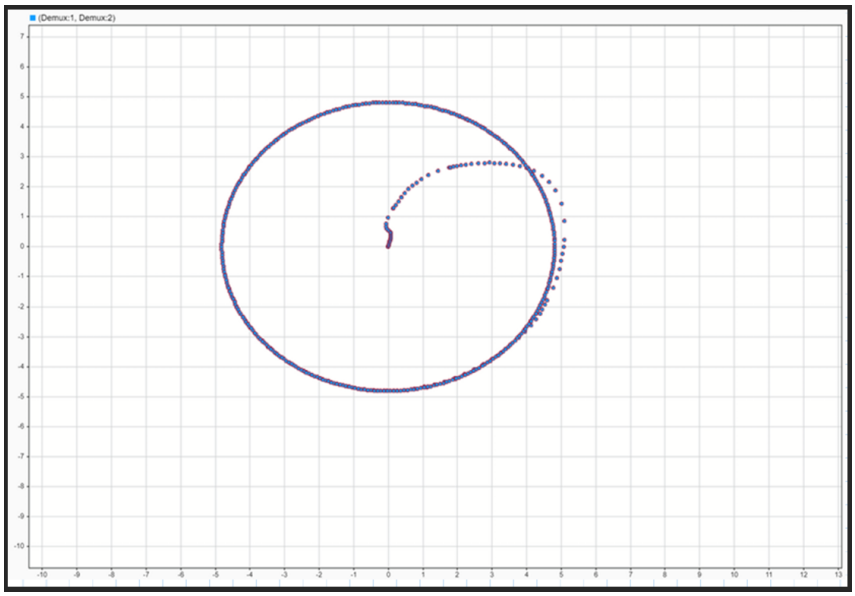


Figure 2.8: Circular tracking (radius 5 m): after the acquisition transient, the vehicle follows the commanded circle in the XY plane.

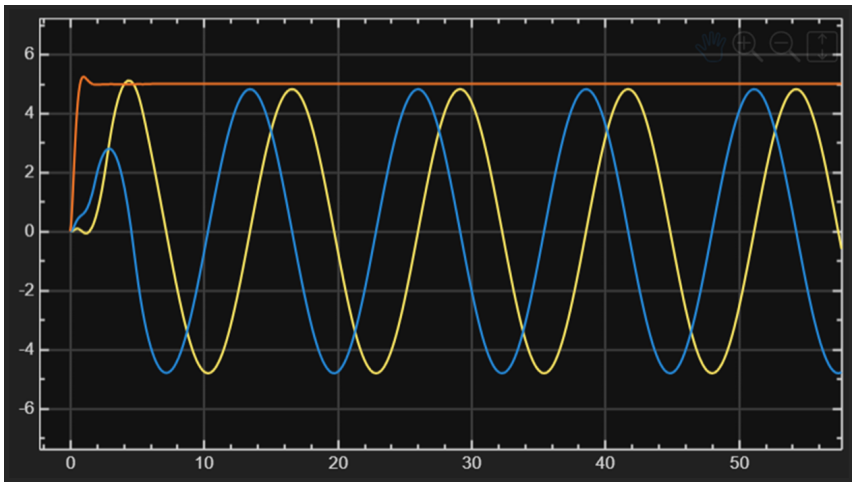


Figure 2.9: Corresponding $x(t)$ and $y(t)$ time histories — quadrature sinusoids of amplitude 5 m.

2.5 Disturbance Rejection: Wind and PID Tuning

To approximate real-world operating conditions, a constant horizontal disturbance is introduced by adding a 2 m/s^2 acceleration to the x channel, representing a steady wind. With the PD outer loop, this produces a persistent steady-state position error: the drone drifts and holds at approximately $x = 2$ m instead of returning to its reference. This is the expected behaviour — a PD controller cannot reject a constant disturbance without a residual error, because a non-zero error is required to generate the opposing control effort.

The remedy is to add integral action, upgrading each outer-loop controller from PD to PID:

$$u = K_p \cdot e + K_d \cdot \dot{e} + K_I \cdot \int e dt.$$

The integrator accumulates the standing error until the control effort exactly cancels the disturbance, driving the steady-state error to zero. An integral block and gain are added to each horizontal channel and the gain is tuned; $K_I = 2$ is found to give the fastest well-behaved recovery. With the integrator in place the drone initially drifts under the wind, then the integral term corrects the offset and returns the vehicle to its reference position. By sweeping the disturbance magnitude it is found that the vehicle can reject horizontal accelerations up to about 4 m/s^2; this ceiling is set by the tilt-saturation limit ($a_{max} = g \cdot \tan \theta_{sat}$), not by the integral action.

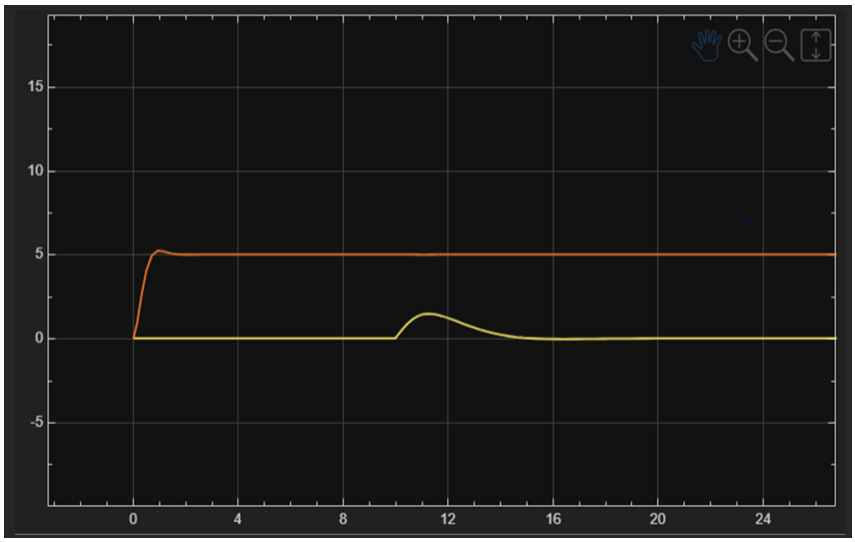


Figure 2.10: Wind test: PID outer loop. The x channel is disturbed at $t \approx 10$ s; the integral term returns the position to its reference, whereas a PD loop would hold a steady offset.

2.5.1 Final Benchmark — Square-Trajectory Tuning by RMSE

The controller coefficients are finalised with a demanding benchmark: the vehicle is commanded to trace a square path (generated with Signal Editor sources), and the tracking quality is quantified by the root-mean-square error (RMSE) between the reference and the actual XY trajectory. The square is the harder test because its corners are velocity discontinuities that no smooth linear controller can follow without overshoot, so RMSE is dominated by corner behaviour and is a sensitive discriminator between gain sets.

Three PID designs are compared, each parameterised by a natural frequency and damping ratio that map to $K_p = \omega_n^2$ and $K_d = 2\zeta\omega_n$ (with $K_I = 2$ held throughout). The results are summarised below.

Design	ω_n **(rad/s)**	ζ	K_p	K_d	K_I	**RMSE (m)**
Run 1	2.0	0.9	4.00	3.6	2	3.891
Run 2	2.5	0.9	6.25	4.5	2	3.515
Run 3 (final)	3.0	0.9	9.00	5.4	2	3.244

Table 2.4: Square-trajectory tuning: three PID gain sets and their RMSE.

RMSE decreases monotonically as the outer-loop bandwidth is raised, from 3.891 m at $\omega_n = 2$ to 3.244 m at $\omega_n = 3$. The final design is therefore $K_p = 9$, $K_d = 5.4$, $K_I = 2$, which gives the minimum RMSE of the three variants. The robustness against wind is re-checked with these coefficients and, as expected, integral action continues to remove the steady-state error — the faster gains do not compromise disturbance rejection.

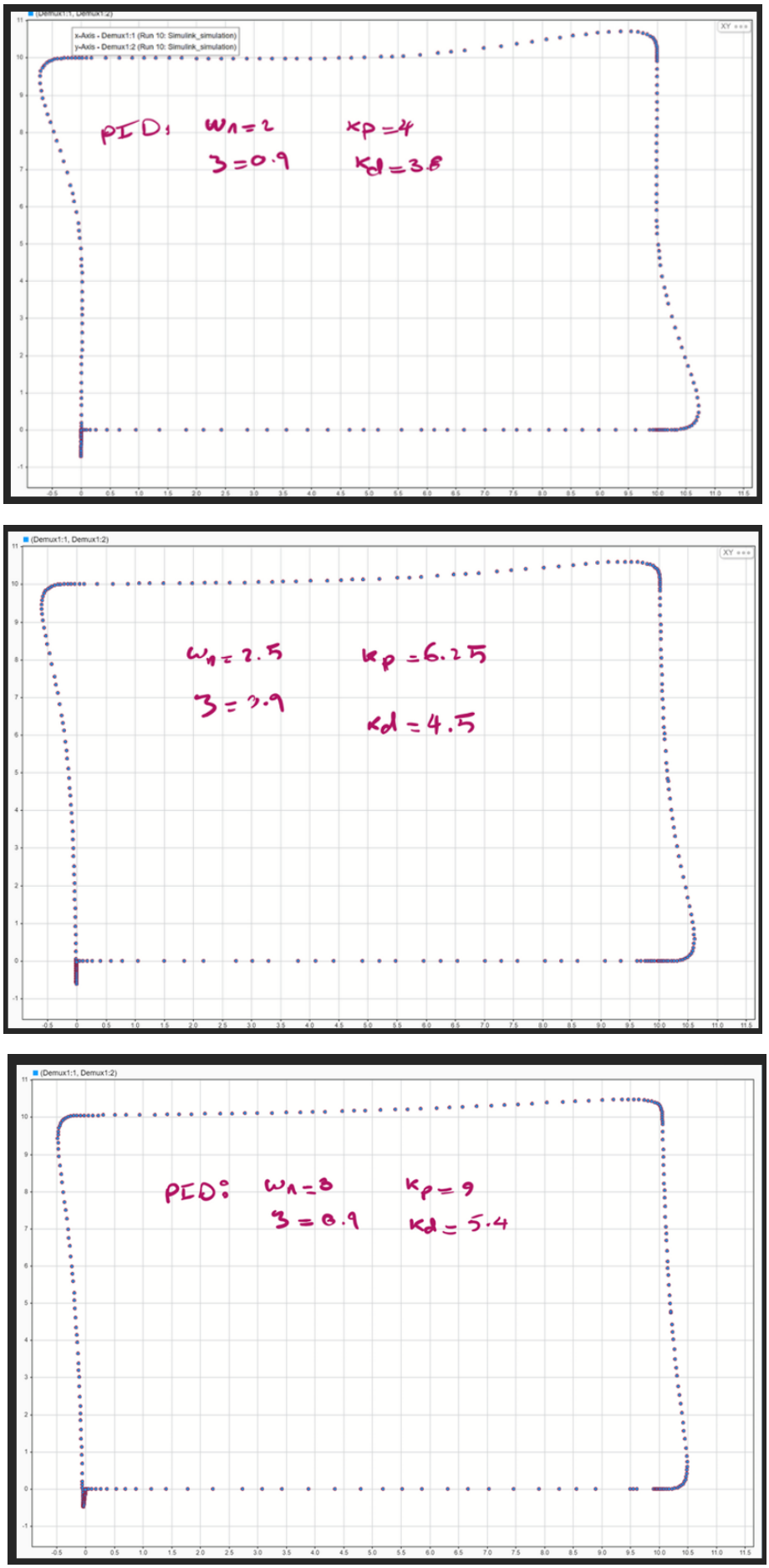


Figure 2.11: Square-trajectory tracking for the three PID variants, annotated with gains and RMSE (3.891 → 3.515 → 3.244 m).

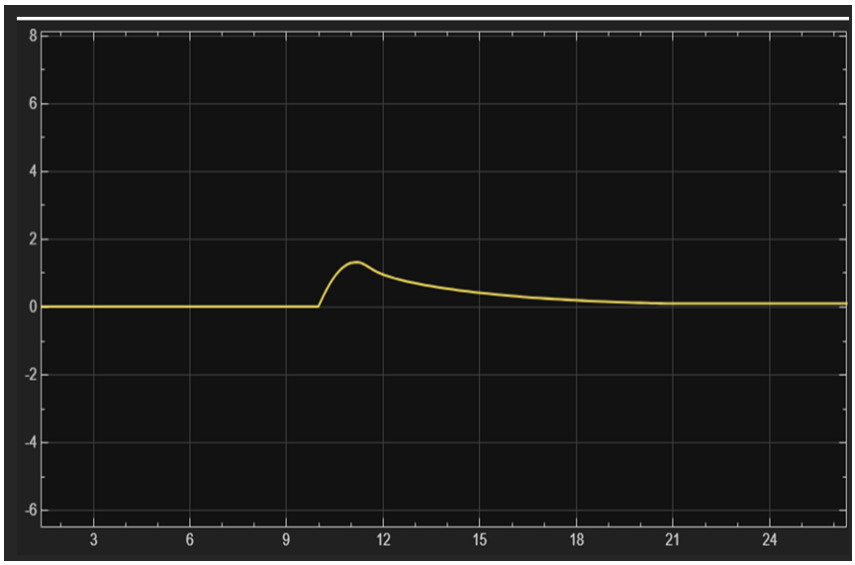


Figure 2.12: Wind robustness re-verified with the final coefficients ($K_p = 9$, $K_d = 5.4$, $K_I = 2$): the disturbance is again rejected to zero steady-state error.

2.6 Chapter Summary

A cascaded autopilot was designed for the under-actuated quadcopter. A motor-mixing stage decouples the intuitive thrust/torque efforts from the individual rotor commands. The inner loop regulates attitude and altitude with PD controllers designed by pole placement (roll/pitch 2.16 + 0.21s, yaw 4.32 + 0.42s, altitude 23.2 + 7.4s with hover feed-forward), achieving the specified 5 % overshoot and peak-time targets in Simulation. The outer loop closes the position problem the inner loop cannot: it converts a global horizontal-acceleration demand — rotated into body reference by $(1/g) \cdot R(\psi)$ — into tilt set-points, and was validated on circular tracking. Finally, a wind disturbance motivated the upgrade to PID; square-trajectory RMSE tuning fixed the outer-loop gains at $K_p = 9$, $K_d = 5.4$, $K_I = 2$. These controllers form the foundation for the sensing, state-estimation, and hardware chapters that follow.

Chapter 3

Sensing and State Estimation

The controllers of Chapter 1 were designed and validated using the true, noise-free state of the vehicle. On real hardware that state does not exist as a directly readable quantity: it must be reconstructed from physical sensors, each of which measures only part of the state and does so with noise, bias, and a limited update rate. This chapter selects the sensor suite, analyses its limitations, and develops the Extended Kalman Filter (EKF) that fuses the sensors into a usable state estimate. Where relevant, material is drawn from the group's sensor-selection and redundancy study.

3.1 A Look at the Necessary Sensors

The response quality of the quadcopter depends directly on the feedback provided by the sensor subsystem. In the cascade architecture, the outer loop needs horizontal position and the inner loop needs attitude, angular rate, and altitude. Four measurement sources — a GPS, an IMU, a magnetometer, and a barometer — supply these quantities, with an additional short-range altimeter added for the low-altitude regime the barometer cannot serve.

3.1.1 The GPS Sensor

A Global Positioning System (GPS) receiver determines absolute position and velocity by trilaterating the signals of orbiting satellites. In this project it supplies the horizontal position (x, y) and the three velocity components that the outer loop and the estimator depend on — the only sensor providing an absolute, non-drifting position reference.

The chosen model is the u-blox NEO-6M. which has a horizontal position accuracy of about 2.5m CEP, a velocity accuracy near 0.1 m/s, and an update rate of 1–5 Hz over a UART/NMEA interface. Its full-scale range is effectively global in the horizontal plane, with an operating altitude ceiling of roughly 50 km and a velocity limit near 500 m/s — far beyond any regime this vehicle will encounter, so the receiver never saturates on our trajectories.

It was chosen for its very low cost, easy microcontroller integration, and sufficiency for general position estimation once fused with the IMU in an EKF. On the price/performance axis it sits firmly at the low-cost end: its accuracy and 1–5 Hz update rate are modest compared with modern multi-constellation receivers, and it performs poorly with limited sky view or during fast manoeuvres — but from an educational standpoint whose GPS is corrected by higher-rate inertial data, that trade is entirely acceptable.

3.1.2 The IMU Sensor

An Inertial Measurement Unit (IMU) measures the vehicle's own motion: a 3-axis accelerometer senses specific force and a 3-axis gyroscope senses angular rate. These are body-frame quantities available at high rate, making the IMU the backbone of the fast inner loop and the driving input of the estimator.

The chosen model is the InvenSense MPU6050, a 6-axis device combining accelerometer and gyroscope with a digital I^2C interface. Relevant characteristics: a user-selectable accelerometer full-scale range of $\pm 2/\pm 4/\pm 8/\pm 16$ g and a gyroscope full-scale range of $\pm 250/\pm 500/\pm 1000/\pm 2000$ °/s, sampled at up to 1 kHz — far faster than the GPS. For this vehicle the ± 4 g and ± 500 °/s ranges comfortably contain the accelerations and rates seen in flight while preserving resolution.

It was chosen because it is one of the most common, low-cost IMUs in robotics and educational control, provides exactly the accelerometer and gyroscope data the inner loop and EKF need, and runs much faster than the GPS. Its weaknesses are equally clear and shape the rest of the chapter: it has no internal magnetometer (so it cannot determine absolute heading alone), its gyroscope drifts over time, it is sensitive to motor/propeller vibration, and it cannot measure position or altitude directly. On price/performance it is a low-cost part with more noise, bias instability, and drift than industrial IMUs — accepted here because the EKF and complementary sensors correct precisely these deficiencies.

3.1.3 The Barometer

A barometer infers altitude from atmospheric static pressure, which falls monotonically with height. It supplies the vertical channel — the altitude measurement used by the altitude-hold controller and the estimator.

The chosen model is the MEAS MS5611, a high-resolution absolute pressure sensor. Relevant characteristics: a pressure full-scale range of 10–1200 mbar (covering roughly -500 m to $+9000$ m of altitude), an altitude resolution as fine as $\sim$10 cm, temperature compensation, low power and weight, and a digital I^2C/SPI output. It was chosen for its precision in detecting small altitude changes, its suitability for altitude-hold, and its EKF compatibility.

Its weaknesses are environmental: readings drift with weather and wind, it is disturbed by propeller wash, filtering it introduces delay, and it cannot measure true distance to the ground. This last point is critical near landing. Below roughly 1 m above the ground, ground effect and propeller-induced pressure fluctuations corrupt the barometric reading, and the sensor also cannot distinguish height-above-terrain from height-above-sea-level. Because the barometer is only reliable from about the 1 m threshold upward, a separate short-range sensor is required for altitudes below it — the rangefinder of the next subsection. On price/performance the MS5611 is an inexpensive, high-resolution part whose limitations are environmental rather than intrinsic, making it excellent value for the altitude-hold role.

3.1.4 The Ultrasonic Rangefinder (MB1240)

An ultrasonic rangefinder measures true distance to the nearest surface by timing an emitted acoustic pulse's echo. It provides the height-above-ground the barometer cannot, covering the low-altitude landing regime.

The intended model is the MaxBotix MB1240 (XL-MaxSonar-EZ4). Relevant characteristics: an analog voltage output scaled at Vcc/1024 per cm, a full-scale range of about 20–765 cm, a 10 Hz update rate, and a narrow beam suited to a downward-facing altimeter. It was chosen to complement the barometer precisely where the barometer fails — below ∼1 m — giving a third, independent altitude source with a different failure mode, which also strengthens the vertical channel of the EKF. On price/performance the MB1240 is a mid-cost sensor whose real, physical distance measurement justifies its price for the landing phase.

For the hardware implementation this sensor was specified, but the MB1240 could not be sourced from the local market. It was therefore replaced with the Sharp GP2Y0A21YK, an infrared triangulation distance sensor. This substitute measures distance from the parallax angle of a reflected IR beam and outputs an analog voltage. Its relevant characteristics differ in important ways: a full-scale range of 10–80 cm (shorter than the MB1240), an update period of ∼38 ms, and — most importantly — a nonlinear, non-monotonic transfer curve whose output rises as objects approach but folds back below ∼10 cm, making very short distances ambiguous. It was chosen simply because it was the available analog distance sensor; on price/performance it is inexpensive but its nonlinearity shifts part of the conditioning burden into firmware (a linearisation curve) and its fold-back requires a guard near the ground. These consequences are treated in the hardware chapter. The table provided by the datasheet of the GP2Y0A21YK is shown below:

Distance	**V_out (typ)**
10 cm	2.30 V
15 cm	1.65 V
20 cm	1.30 V
30 cm	0.90 V
40 cm	0.70 V
50 cm	0.58 V
60 cm	0.50 V
80 cm	0.40 V

Table 3.1: GP2Y0A21YK distance vs. output voltage

This table is then fitted to the function below:

$$d = 27.86 \cdot V^{-1.15} - 0.42$$

3.1.5 The Magnetometer

A magnetometer measures the local magnetic field vector and, from it, the vehicle's absolute heading. It exists in the suite for one essential reason: the gyroscope alone yields yaw only by integration, which drifts, and — as shown later — yaw is weakly observable from GPS. A direct heading measurement is therefore needed.

The chosen model is the Honeywell HMC5883L, a 3-axis digital magnetometer. Relevant characteristics: a selectable full-scale range from ±0.88 to ±8.1 gauss (the ±1.3 gauss setting comfortably contains the Earth's ∼0.25–0.65 gauss field), a digital I^2C output, and heading resolution of 1–2°. It was chosen to provide the absolute yaw reference the MPU6050 lacks, to arrest long-term yaw drift, and for its EKF compatibility. Its weaknesses are significant: it is highly sensitive to magnetic noise from motors, batteries,

and ESCs, it requires careful hard- and soft-iron calibration, and it is too slow and noisy to serve the fast inner loop alone. On price/performance it is a low-cost sensor whose value is entirely in supplying the one quantity — absolute heading — that nothing else in the suite can.

Sensor Suite Summary

Sensor	Model	Measures	Full-scale range	Interface
GPS	NEO-6M	x, y, velocity	global; ≤50 km alt	UART/NMEA
IMU	MPU6050	accel, angular rate	±4 g / ±500 °/s (set)	I^2C
Barometer	MS5611	pressure → altitude	10–1200 mbar	I^2C/SPI
Rangefinder	MB1240 → GP2Y0A21YK	height above ground	20–765 cm → 10–80 cm	analog
Magnetometer	HMC5883L	magnetic field → yaw	±1.3 gauss (set)	I^2C

Table 3.2: Table 3.1. Sensor suite: measured variables, chosen models, and full-scale ranges (Added).

3.2 Redundancy

Because no single sensor is trustworthy at all times, the suite is built so that critical quantities are measured by more than one independent source and fused, so that the loss or degradation of any one sensor does not disable the vehicle. The redundancy scheme follows the group's structural-requirements study.

Altitude redundancy

Vertical state is over-determined by three independent sources — the MS5611 barometer, the GPS altitude, and the IMU vertical acceleration — fused in the EKF, with the ultrasonic/IR rangefinder covering the sub-1 m regime where the barometer is unreliable. Their failure modes are uncorrelated (weather for the baro, sky view for GPS, integration drift for the IMU), so the fusion is robust to any one of them degrading.

Attitude redundancy

Orientation is reconstructed by fusing three sources: the gyroscope (fast but drifting), the accelerometer (which senses the gravity direction and so bounds roll and pitch), and the HMC5883L magnetometer (which fixes absolute yaw). No single one is sufficient — the gyroscope drifts, the accelerometer is corrupted by linear acceleration, the magnetometer is slow and magnetically noisy — but together they span the attitude state.

Navigation redundancy and fail-safes

The NEO-6M GPS is backed by short-term IMU/barometer dead-reckoning: if GPS is lost, the estimator coasts on inertial and barometric data long enough to trigger a fail-safe. The study specifies concrete fail-safe logic — loss of GPS initiates hover or landing, and low battery initiates return-to-home or landing — and an EKF innovation check that rejects measurement outliers whose deviation exceeds a defined threshold, protecting the estimate against transient sensor faults.

Single points of failure

One limit must be stated honestly: redundancy covers sensing, not actuation. A quadcopter cannot survive a complete motor or ESC failure — it has exactly four actuators and no spare control authority — so true actuator redundancy requires a hexacopter or octocopter. This is an architectural limit of the airframe, not something the sensor suite can remedy.

3.3 The Simulink Sensor Subsystem

Now that we now the sensors, we should take a look at how these sensors have simulated in Simulink. This subsystem is the bridge between the ideal plant of Chapter 1 and the estimator of chapter 3.4. it receives the clean state, passes it through a model of each physical sensor, and produces the two measurement buses the Extended Kalman Filter consumes. The complete block is shown in Figure 3.0.

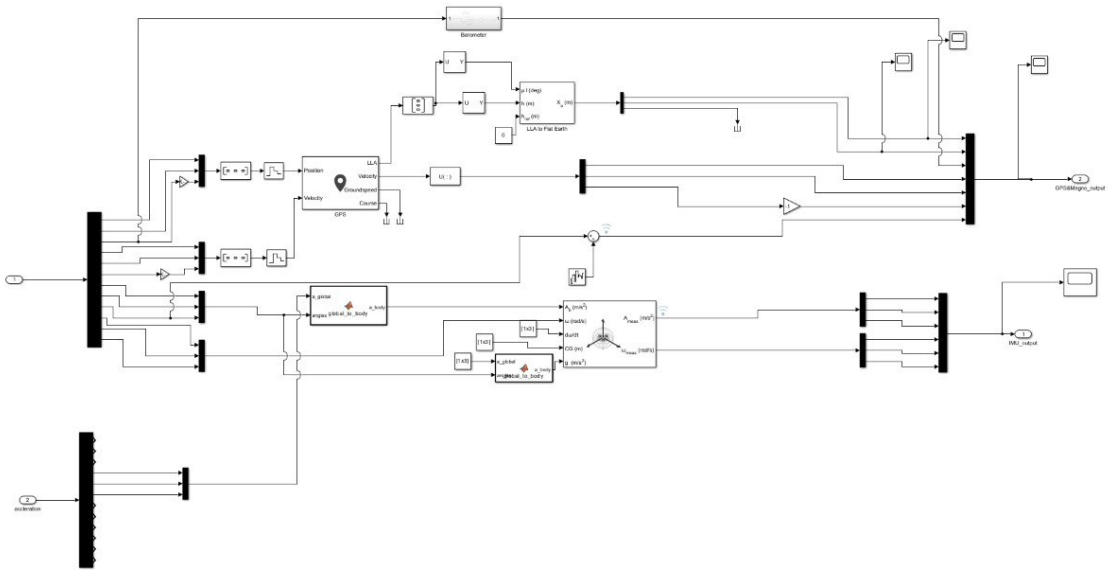


Figure 3.1: The Simulink sensor subsystem: the true state (input 1) is distributed to the GPS, barometer, IMU, and magnetometer models, producing the IMU_output and GPS&Magno_output measurement buses.

Note: This subsystem models sensors, it does not estimate. Its purpose is to degrade the perfect state into measurements with the correct noise, bias, quantisation, and — crucially — the correct coordinate frames and sample rates that each real device would impose. The estimator that recovers a clean state from these measurements is the separate subject of section 3.4.

3.3.1 Inputs and Signal Distribution

The subsystem has two inputs. Input 1 (upper-left) is the full true state vector of the vehicle, produced by the plant. Input 2 (lower-left, labelled acceleration) carries the true linear acceleration, which the IMU accelerometer model requires but which is not part of the integrated state vector. Immediately after each input, a bank of Demux and Selector blocks fans the vectors out into their individual components - position, velocity, Euler angles, body rates, and accelerations - so that each sensor model can be fed exactly the quantities it measures. This distribution network is the array of black bars on the left of the diagram, it performs no computation, only routing.

3.3.2 The GPS Path and the LLA-to-Flat-Earth Conversion

The GPS path is the most intricate part of the subsystem and merits step-by-step treatment.

Down-sampling and frame preparation into the GPS block

The true position and velocity are first passed through a gain and a Zero-Order Hold. Two things happen here. First, a -1 gain is applied to the vertical components: the Simulink plant works in a z-up (ENU-family) frame, whereas the GPS sensor block interprets its inputs in the aerospace-standard North-East-Down (NED) frame, where the vertical axis points downward. The sign flip reconciles the two conventions on the way in. Second, the Zero-Order Hold samples the continuous signal down to the GPS update rate (0.1 s, i.e. 10 Hz), a real receiver reports only at this slow rate, and the GPS block itself rejects continuous-time inputs, so this discretisation is mandatory.

The GPS sensor block and its LLA output

The prepared position and velocity enter the GPS block. This block models a real receiver, it adds position and velocity noise with the characteristic slow, correlated (random-walk) behaviour of GPS, and it outputs four signals — LLA, Velocity, Groundspeed, and Course. The important one is LLA, which stands for Latitude, Longitude, Altitude. This is the crux of the whole path: a real GPS does not report position as local metres — it reports geodetic coordinates, an angular latitude and longitude on the Earth's ellipsoid plus an altitude. The block faithfully reproduces this, emitting a three-element LLA vector in [degrees, degrees, metres] rather than the [x, y, z] metres the controllers and estimator expect.

Why the LLA must be converted to Flat Earth

Because the rest of the system — the outer-loop controller, the estimator, the plant — reasons in local Cartesian metres, the geodetic LLA output is unusable directly. Latitude and longitude are angles on a curved surface, a controller cannot subtract a reference position from a latitude in degrees and obtain a metre-valued error. The LLA must therefore be converted back into a local flat-Earth position — a tangent-plane approximation in which a small patch of the Earth's surface is treated as flat and positions are expressed as [x, y, z] metres from a reference origin. This round trip — true metres $\rightarrow$ (into GPS) $\rightarrow$ noisy LLA $\rightarrow$ (out of GPS) $\rightarrow$ noisy metres — is deliberate: it is precisely how the noise a real receiver introduces in the geodetic domain is injected into the local position estimate.

The LLA-to-Flat-Earth block, step by step

The conversion is performed by the LLA to Flat Earth block (upper-middle of the diagram), fed through two small U$\rightarrow$Y Selector/reshape blocks that split the three-element LLA vector into the shapes the conversion block's ports require:

- **l (deg) port.** This receives the latitude and longitude together as a 2×1 vector. One Selector extracts elements 1–2 (lat, lon) of the LLA output and orients them as the required column vector.
- **h (m) port.** This receives the altitude — element 3 of the LLA vector — as a scalar.
- **href (m) port.** This is the reference altitude, supplied by the constant 0 (or a fixed reference such as 1200 m) beneath the block. It defines the altitude of the flat-Earth tangent plane's origin.

The block outputs Xe (m), the position in the local flat-Earth frame as a 3×1 vector of metres. Internally the block assumes the NED convention, i.e. its z output is down-positive. To return this to the subsystem's z-up frame, a further -1 gain is applied to

the vertical component of Xe on the output side. This is the counterpart of the input sign flip: NED is entered, NED is exited, and the two flips together keep the GPS path consistent with the z-up convention used everywhere else.

Note: The barometer, not the GPS, ultimately supplies the altitude used by the filter. The GPS's own vertical channel is comparatively poor, so although the LLA-to-Flat-Earth block produces a z value, the estimator's vertical measurement is taken from the dedicated barometer path described below. The GPS path's primary contribution is the horizontal position x and y.

The GPS velocity output

In parallel with the position conversion, the GPS block's Velocity output is taken directly (through the U(:) reshape block that flattens it to a plain vector) and routed to the output bus. Because it originates in the NED-configured GPS block, its vertical component is likewise sign-corrected (the -1 gain on the lower velocity branch) to match the z-up frame. The Groundspeed and Course outputs are not used and are terminated.

3.3.3 The Barometer Path

The Barometer block (top of the diagram) receives the true altitude directly and models the pressure-derived altitude measurement: it adds noise and bias and samples at the barometer rate. Because altitude is naturally "up-positive," and the barometer is treated in the subsystem's native z-up frame, this path requires no sign flip — a clean contrast with the GPS path, and the reason the barometer, rather than the GPS, is the preferred altitude source. Its output is routed to the GPS&Magno bus as the vertical measurement.

3.3.4 The IMU Path

The IMU path (lower-centre) models the accelerometer and gyroscope together. The accelerometer does not measure kinematic acceleration in the world frame; it measures specific force in the body frame. Two coordinate rotations therefore feed the IMU block, both implemented by global_to_body MATLAB Function blocks:

- The true linear acceleration (from input 2), expressed in the world frame, is rotated into the body frame by the first global_to_body block using the current Euler angles, and fed to the block's Ab (m/s^2) port as the kinematic body acceleration.
- The gravity is rotated into the body frame by the second global_to_body block and fed to the g (m/s^2) port, so that the block can form the specific force correctly (specific force = kinematic acceleration minus gravity).

The remaining ports are the true body rates on (rad/s), the angular acceleration on d /dt (a [1×3] input), and the centre-of-gravity offset on CG (m). The block then outputs the two noisy inertial measurements — A_meas (m/s^2), the body-frame specific force, and _meas (rad/s), the measured body rates — with realistic accelerometer and gyroscope noise, bias, and the small wireless-style noise symbol indicating the added stochastic component. These two three-element signals become the IMU_output bus.

3.3.5 The Magnetometer (Yaw) Path

Yaw is supplied by a simplified magnetometer model. The true yaw is taken from the state, and a band-limited white-noise source (the noise-generator symbol) is added to it at

a summing junction, producing a noisy heading measurement at the magnetometer rate. This stand-in captures the essential character of a real compass — an absolute heading corrupted by noise — without modelling the full three-axis magnetic field. The result is appended to the GPS&Magno bus as its seventh element.

3.3.6 The Two Output Buses

The subsystem terminates in two Mux blocks that assemble the measurements into the buses the estimator consumes:

Output bus	**Elements**	**Source**	**Rate**
IMU_output	ax, ay, az, p, q, r	IMU block (A_meas, ω_meas)	0.005 s (200 Hz)
GPS&Magno_output	x, y, z, vx, vy, vz, yaw	GPS (x,y,v), barometer (z), magnetometer (yaw)	0.1 s (10 Hz)

Table 3.3: Table 3.0a. The two measurement buses produced by the sensor subsystem.

The multirate structure is deliberate and physically faithful: the inertial measurements are fast (200 Hz), while the GPS-derived measurements are slow (10 Hz). This is exactly the fast-IMU / slow-GPS pattern the Extended Kalman Filter is built around — predicting at the IMU rate and correcting at the GPS rate — and it is established here, at the sensor level, so that the estimator inherits it directly. The various scope blocks scattered around the diagram are for inspection during development and play no role in the signal path.

3.4 The Problems with the Sensors

Two specific deficiencies of the raw sensor suite motivate the estimator and must be stated precisely, because the EKF is designed to answer exactly these two problems.

3.4.1 No Direct Sensor for Roll and Pitch

There is no sensor that outputs the roll angle ϕ or the pitch angle θ directly. The gyroscope measures angular rate, not angle — integrating it yields attitude but accumulates drift without bound. The accelerometer senses the gravity vector, from which tilt can in principle be inferred, but only when the vehicle is not accelerating; in flight its output is a mixture of gravity and manoeuvre acceleration. The magnetometer fixes only yaw. Consequently roll and pitch are unmeasured states that must be estimated by fusing gyroscope, accelerometer, and the position/velocity dynamics — a task no single sensor can perform and which is a principal reason the EKF exists.

3.4.2 Noise on the Sensors

Every measurement is corrupted by noise, bias, and quantisation. GPS position carries metre-level noise; the accelerometer is polluted by motor and propeller vibration; the magnetometer is disturbed by the currents of the motors and ESCs; the barometer fluctuates with propeller wash. Feeding these raw signals to a controller designed for the clean state would inject that noise straight into the motor commands. The severity is visible in the simulation of §3.5, where the raw GPS position signal spans roughly ± 10 m of noise about a true value of a few metres. The estimator's second job is therefore to filter this noise

— to extract a smooth, low-variance estimate from measurements whose instantaneous values are nearly unusable.

3.5 The Solution: The Extended Kalman Filter

3.5.1 The Theory

The Kalman filter is a recursive state estimator that fuses a model-based prediction of how the state evolves with noisy measurements, weighting each by its trustworthiness to produce an estimate more accurate than either alone. It maintains not only a state estimate $\hat{x}$ but also a covariance P describing the estimate's uncertainty, and propagates both over time.

The linear Kalman filter

For a linear system $x_k = Fx_{k-1} + Bu_{k-1} + w$ with measurement $z_k = Hx_k + v$, where $w \sim \mathcal{N}(0, Q)$ is the process noise and $v \sim \mathcal{N}(0, R)$ is the measurement noise, the filter alternates two steps. The predict step advances the state and grows its uncertainty:

$$\hat{x} \leftarrow F\hat{x} + Bu, \quad P \leftarrow FPF^T + Q.$$

The update step, executed when a measurement arrives, computes the Kalman gain, corrects the state toward the measurement, and shrinks the uncertainty:

$$K = PH^T(HPH^T + R)^{-1}, \quad \hat{x} \leftarrow \hat{x} + K(z - H\hat{x}), \quad P \leftarrow (I - KH)P.$$

The gain K is simply a trust weighting: a large R (noisy sensor) drives K toward zero so the measurement is largely ignored, while a small R lets the measurement dominate.

The Q and R matrices

These two covariance matrices are the heart of the filter's tuning and deserve precise treatment. R is the measurement-noise covariance: a matrix whose diagonal entries are the variances (σ^2) of each sensor's noise. It answers "how much do I distrust each measurement?" and is read essentially from the sensor datasheets — a GPS with $\sigma = 2.5$ m contributes 6.25 to the corresponding diagonal of R. Q is the process-noise covariance: it captures everything the prediction model fails to represent — un-modelled dynamics, discretisation error, and, in an IMU-driven filter, the noise entering through the inertial inputs. It answers "how much do I distrust my own model?". Unlike R, Q cannot be read cleanly from a datasheet; it is seeded from the inertial noise densities and then tuned. The ratio of Q to R sets the filter's character: a large Q/R trusts measurements and tracks quickly but noisily; a small Q/R trusts the model and smooths heavily but lags.

The extension to nonlinear systems

The quadcopter is nonlinear — its dynamics contain products of trigonometric functions of the attitude and a body-to-navigation rotation — so constant matrices F and H do not exist. The Extended Kalman filter resolves this by keeping the full nonlinear functions to propagate the state — predicting with $\hat{x} \leftarrow f(\hat{x}, u)$ and the expected measurement with $\hat{z} = h(\hat{x})$ — while linearising only for the covariance arithmetic, using the Jacobians $F = \partial f/\partial x$ and $H = \partial h/\partial x$ evaluated at the current estimate. Every other equation is identical to the linear filter. In short, the EKF is the linear Kalman filter with f, h and their Jacobians substituted for F, H.

3.5.2 Updated Project Scope

Based on the aforementioned points, the updated block diagram is as follows:

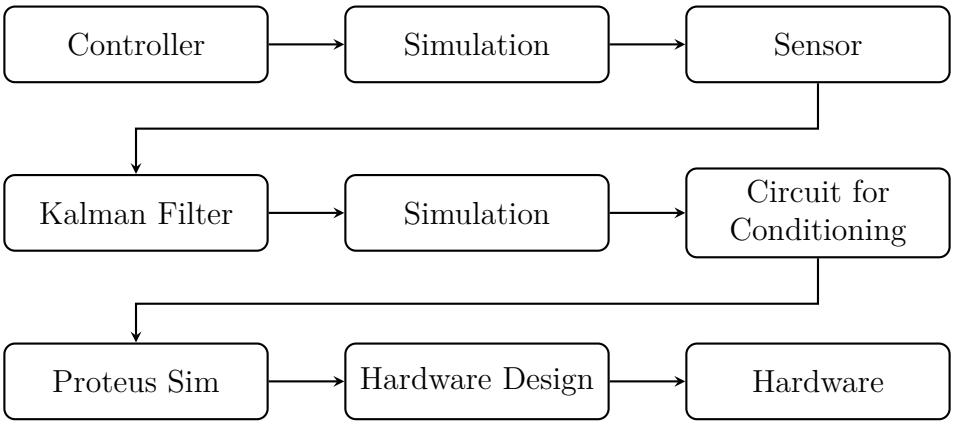


Figure 3.2: Updated project scope block diagram.

3.5.3 The Implementation

(a) Two possible measurement structures

There are two standard ways to arrange a quadcopter EKF, and the choice determines the entire filter structure.

In the first approach — all sensors as measurements — a simple kinematic model propagates the state and every sensor, including the accelerometer and gyroscope, is treated as a measurement corrected in the update step. It is conceptually simple, but it forces the process model to predict body specific force and angular rate, quantities set by thrust and un-modelled control action; the model must then invent a fictitious "constant acceleration" assumption and fights it continuously.

In the second approach — IMU as input, GPS and magnetometer as measurements (the standard strapdown INS/GPS structure) — the accelerometer and gyroscope are treated as the input u that drives the prediction, while the slow, absolute GPS position, GPS velocity, barometric altitude, and magnetometer yaw are the measurements that correct the inertial drift. This mirrors how real flight controllers operate and cleanly matches the physics: the IMU tells the model how the vehicle is accelerating and rotating, and the absolute sensors bound the resulting drift.

The second approach was chosen. It avoids the fictitious process model of the first, it places the IMU's noise where it physically enters (through the input, i.e. into Q), and it lets the datasheet-derived GPS/magnetometer noise define R directly. This partition — IMU into the state-transition function, GPS/baro/magnetometer into the measurement function — is precisely what the two MATLAB functions below implement.

(b) The MATLAB functions

The filter is realised with the Control System Toolbox Extended Kalman Filter block, which requires two user functions: a state-transition function $f(x, u)$ and a measurement function $h(x)$. The reported version uses a single measurement vector of seven elements — the three GPS positions (with the altitude actually supplied by the barometer), the three GPS velocities, and the magnetometer yaw. The nine estimated states are

$$x = [x, y, z, v_x, v_y, v_z, \phi, \theta, \psi]^T.$$

State-transition function. This encodes the rigid-body kinematics in the z-up (ENU) frame: position integrates velocity; velocity integrates the accelerometer specific force rotated into the navigation frame plus gravity; and the Euler angles integrate the body rates through the standard rate-transformation matrix. The step Ts is fixed at the 200 Hz IMU rate. The single extra input $u = [a_x, a_y, a_z, p, q, r]$ carries the IMU data.

```
function xk1 = ekfStateTransition(xk, u)
% xk = [px py pz vx vy vz phi theta psi]'
% u  = [ax ay az p q r]'
Ts = 0.005;
phi = xk(7); theta = xk(8); psi = xk(9);
a_b = u(1:3);  w_b = u(4:6);

cph=cos(phi); sph=sin(phi);
cth=cos(theta); sth=sin(theta); cps=cos(psi); sps=sin(psi);


R = [ cth*cps, sph*sth*cps-cph*sps, cph*sth*cps+sph*sps;
cth*sps, sph*sth*sps+cph*cps, cph*sth*sps-sph*cps;
-sth,    sph*cth,             cph*cth ];
g = [0; 0; -9.81];

% body rates -> Euler rates
T = [ 1, sph*tan(theta), cph*tan(theta);
0, cph,           -sph;
0, sph/cth,        cph/cth ];

xdot = [ xk(4:6);          % p_dot = v
R*a_b + g;        % v_dot
T*w_b ];          % Theta_dot
xk1  = xk + Ts*xdot;       % forward Euler
end
```

Line by line: the roll, pitch, yaw are extracted from the state; the accelerometer and gyro are split out of u; the body-to-navigation rotation matrix R (ZYX Euler order) and the Euler-rate matrix T are built from the current attitude; the three physical relations — $\dot{p} = v$, $\dot{v} = R \cdot a_b + g$, $\dot{\Theta} = T \cdot w_b$ — are assembled into the derivative xdot; and a forward-Euler step advances the state. Note the process noise w is not added here — with additive process noise the EKF block adds it internally.

Measurement function. The GPS measures position and velocity directly and the magnetometer measures yaw directly, so the measurement model is a simple selection of states — the nonlinearity lives entirely in f, not h. The seven-element output picks states 1–6 (position and velocity) and state 9 (yaw).

```
function y = ekfMeasurement(xk)
% 7-element measurement:
%   GPS position (x,y,z=baro), GPS velocity, magnetometer yaw
y = [ xk(1:6); xk(9) ];
end
```

The measurement Jacobian is therefore the constant $H = [I_6 \quad 0_{6\times 3}; \text{row selecting } \psi]$, a 7×9 matrix of ones and zeros. Two consequences of this structure are worth noting. First, the third measurement is labelled a GPS position but is physically the barometer altitude, exploiting that both report the same state z — the filter simply receives a better-quality vertical measurement. Second, because roll and pitch appear in no row of h, they are never measured directly; they are observed only indirectly through the way an attitude error rotates the accelerometer and corrupts the velocity prediction that GPS then corrects. This is the formal expression of the problem raised in §3.3.1, and it is why the estimator — not a sensor — is what ultimately delivers ϕ and θ.

Covariance settings

Matrix	**States / measurements**	**Diagonal values (variance)**
R (7×7)	GPS x,y,z(baro); GPS vx,vy,vz; mag yaw	[6.25, 6.25, 0.04, 2.5e-3, 2.5e-3, 2.5e-3, 1.2e-3]
Q (9×9)	position; velocity; attitude	[1e-6 ×3, 1e-2 ×3, 1.5e-6 ×3] (tuned)
P_0 (9×9)	initial uncertainty	[6.25, 6.25, 0.04, 2.5e-3 ×3, 3e-3, 3e-3, 1.2e-3]

Table 3.4: Table 3.2. EKF covariance settings for the seven-measurement filter (Added).

3.6 Simulation Results of the Kalman Filter

The filter is verified in closed simulation by comparing, for each state, the true value, the raw sensor measurement, and the EKF estimate. The results confirm the two jobs set out in §3.3: recovering the unmeasured roll and pitch, and filtering the heavy measurement noise.

Roll and pitch estimation

Figures 3.1 and 3.2 place the true and estimated roll and pitch side by side. Despite roll and pitch never being measured directly, the estimate reproduces the true trajectory almost exactly — the aggressive, high-frequency manoeuvres of ϕ and θ (excursions of ± 0.2 rad) are tracked with only slight smoothing. This is the estimator delivering the states that no sensor provides, and it is the strongest single result of the chapter.

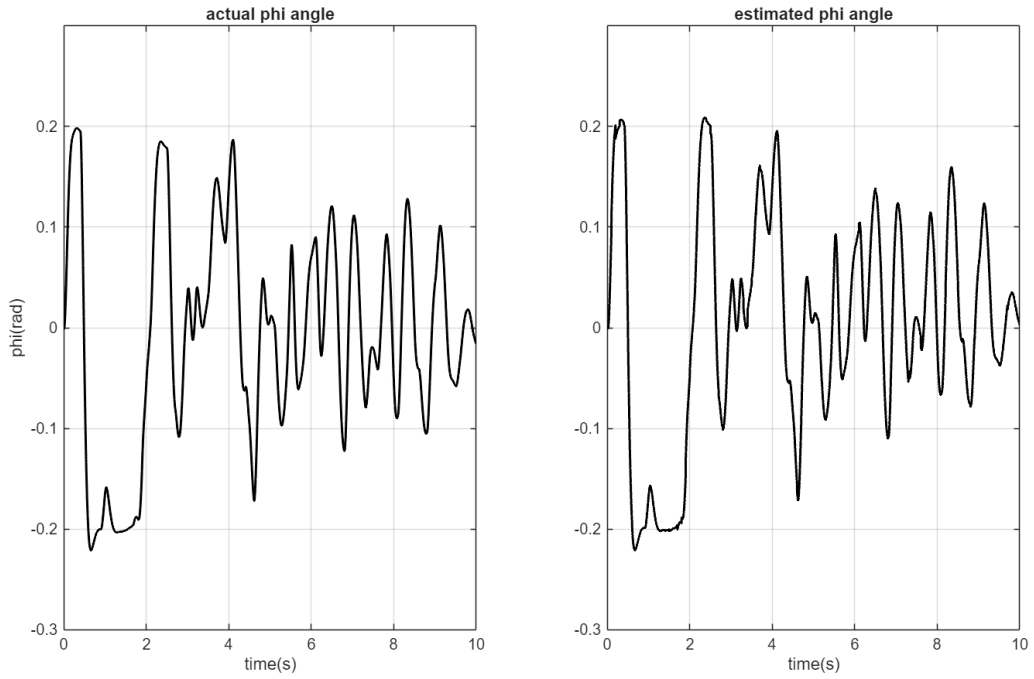


Figure 3.3: Roll: true ϕ (left) versus EKF-estimated ϕ (right). The unmeasured roll angle is reconstructed almost exactly.

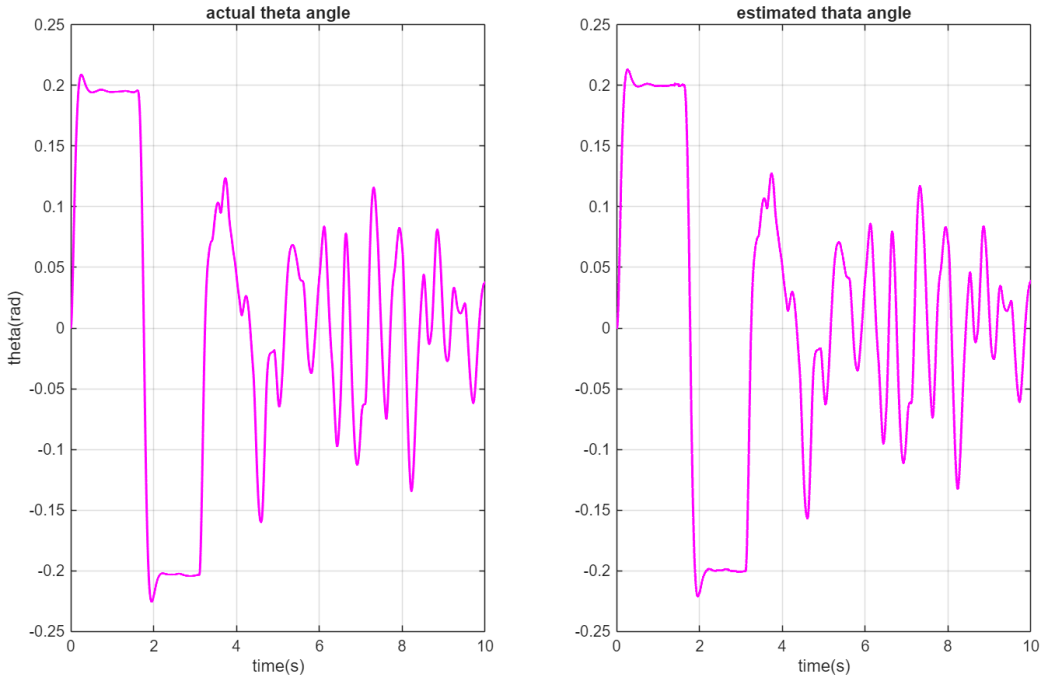


Figure 3.4: Pitch: true θ (left) versus EKF-estimated θ (right). Like roll, pitch is recovered despite never being measured directly.

Horizontal position filtering

Figures 3.3 and 3.4 show the horizontal channels in three panels each — true, raw sensor, and filtered. The contrast is dramatic: the raw GPS signal swings across roughly ±10–15 m of noise, rendering the instantaneous measurement almost unusable, yet the filtered

estimate tracks the true trajectory to within a few tenths of a metre. The estimator reduces the position noise by roughly one to two orders of magnitude, converting an unusable sensor into a usable state.

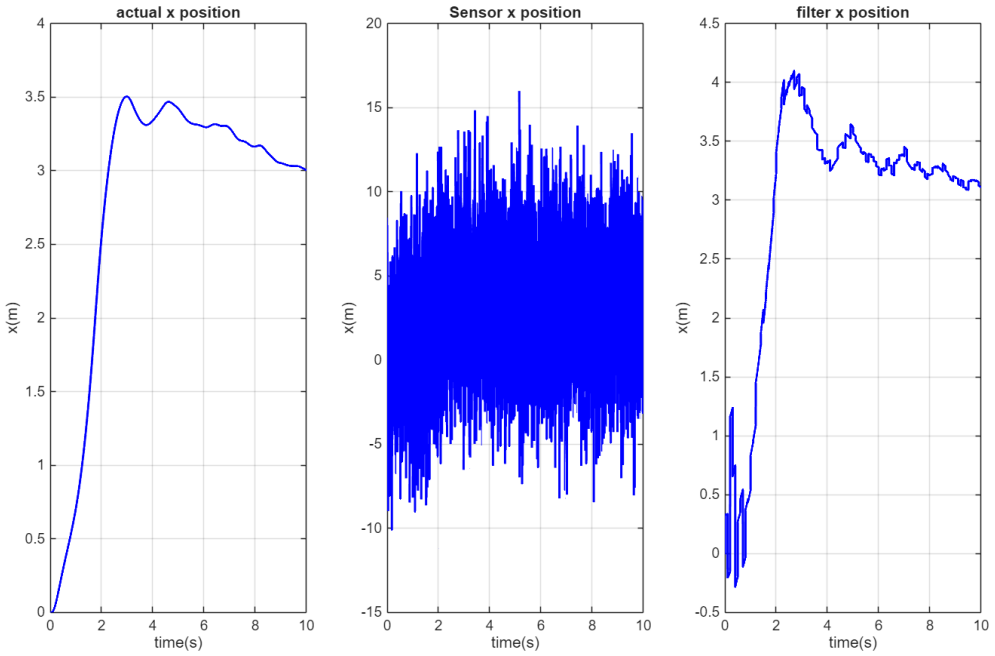


Figure 3.5: x position: true (left), raw GPS sensor spanning ±10 m of noise (centre), and EKF estimate tracking truth to a few tenths of a metre (right).

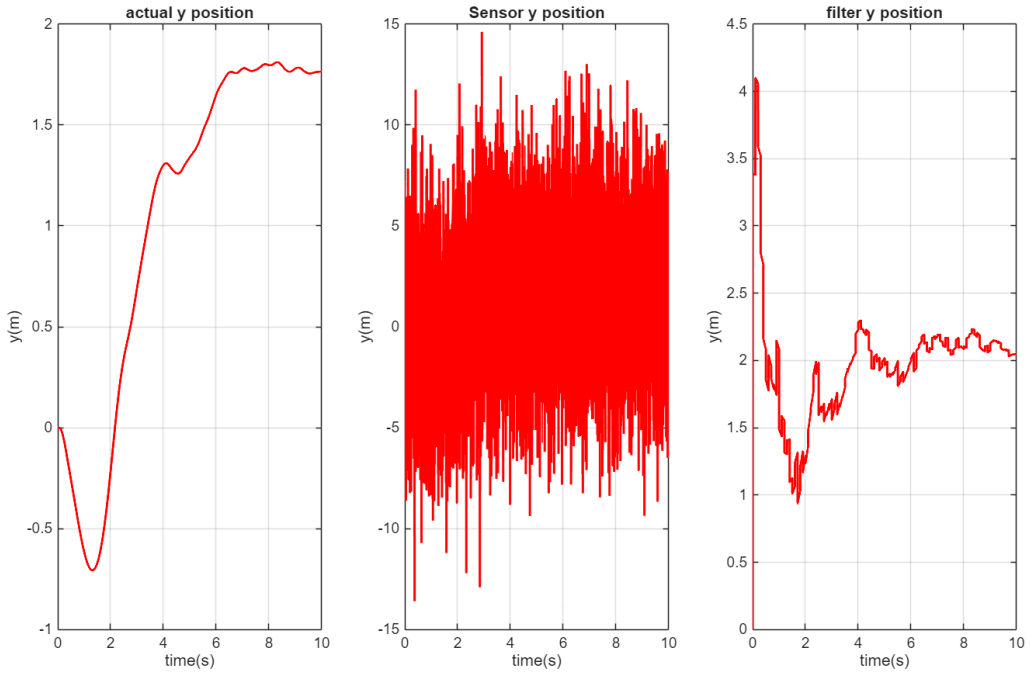


Figure 3.6: y position: true (left), raw sensor (centre), EKF estimate (right). Same one-to-two order-of-magnitude noise reduction as x.

Altitude and yaw

The altitude channel (Figure 3.5) is the cleanest: the barometer-based measurement is

already low-noise, so true, sensor, and filtered traces all sit tightly on the 3 m set-point — the filter smooths the residual ripple. Yaw (Figure 3.6) shows the magnetometer measurement, which is noisier (±0.1 rad) than the small true yaw (±0.03 rad); the filter recovers the true heading from it, confirming that the seventh measurement performs its intended role of anchoring yaw.

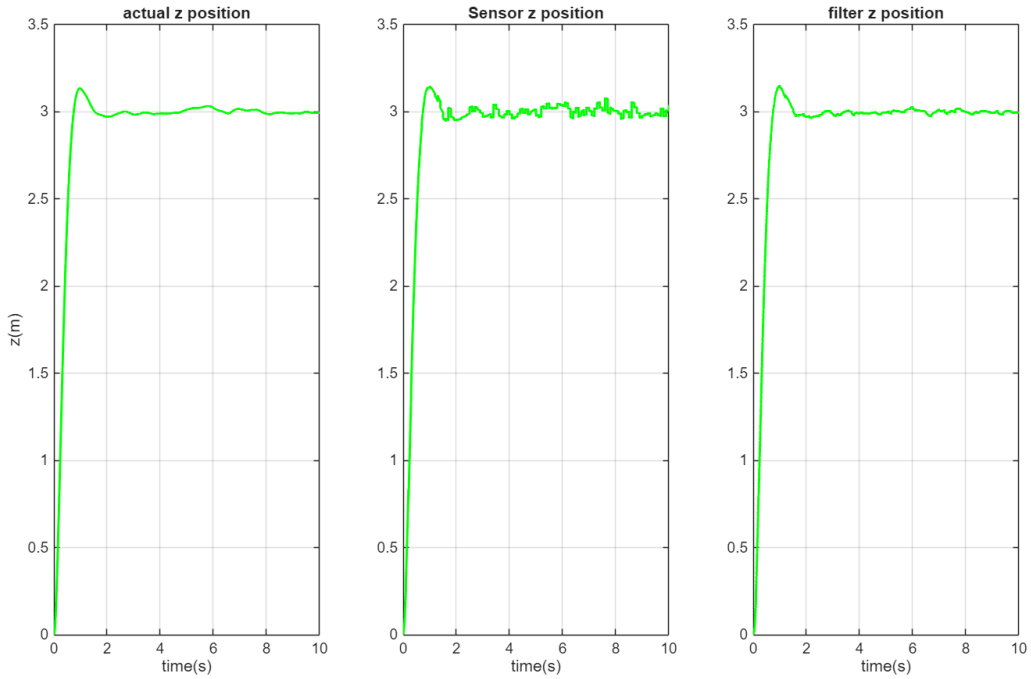


Figure 3.7: z altitude: true, barometer sensor, and EKF estimate. The barometric measurement is already clean; all three agree closely on the 3 m set-point.

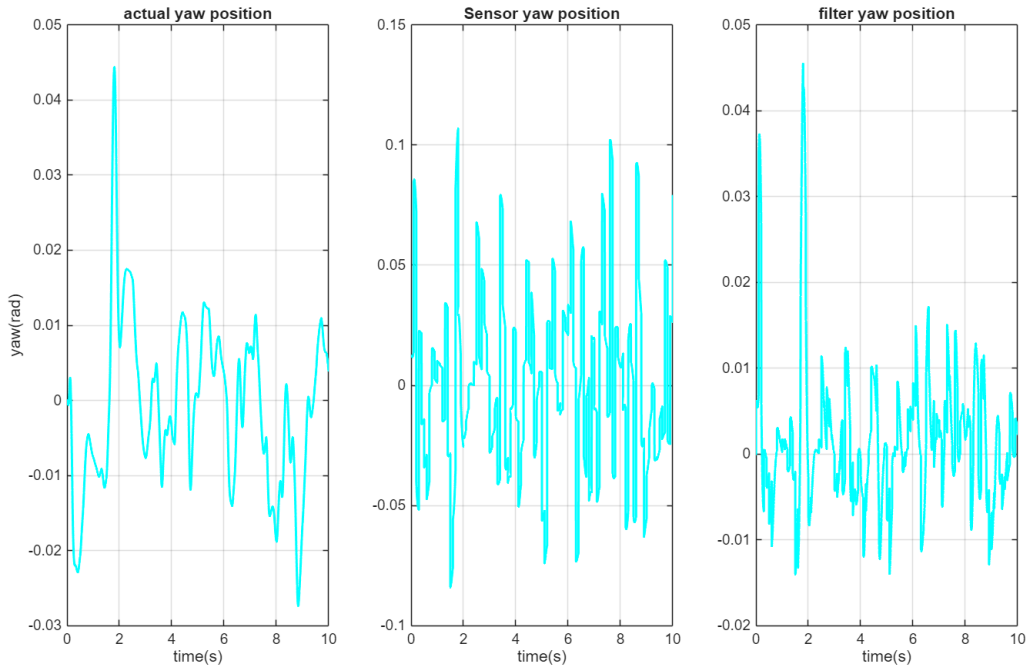


Figure 3.8: Yaw: true (left), magnetometer sensor (centre, ±0.1 rad noise), EKF estimate (right). The filter recovers the small true heading from the noisier magnetometer signal.

3.7 Chapter Summary

Added for completeness: Short closing summary added to consolidate the chapter.

A four-source sensor suite was selected — NEO-6M GPS, MPU6050 IMU, MS5611 barometer, and HMC5883L magnetometer, with an MB1240 rangefinder (substituted in hardware by a GP2Y0A21YK) for the sub-1 m regime the barometer cannot serve. Critical quantities are made redundant by fusing independent sources, though actuator redundancy remains beyond a quadcopter's reach. Two deficiencies motivate estimation: roll and pitch are measured by no sensor, and every measurement is heavily noisy. Both are answered by an Extended Kalman Filter built in the IMU-as-input / GPS-and-magnetometer-as-measurement structure, implemented through the `ekfStateTransition` and `ekfMeasurement` functions with a seven-element measurement vector. Simulation confirms the filter reconstructs the unmeasured attitude almost exactly and reduces the raw position noise by one to two orders of magnitude, producing the clean state estimate the controllers of Chapter 1 require.

Chapter 4

The Conditioning Circuits

Having selected the sensor suite in Chapter 3, this chapter addresses the electrical interface between those sensors and the flight-control microcontroller: the conditioning circuits that carry each sensor's signal to the processor cleanly and safely. The circuits are presented in the order in which they were developed, each validated in Proteus. A short note on the target processor is warranted first.

Note on the target microcontroller

The intended flight-control processor is the ESP32-WROOM-32 (30-pin) module, chosen for its 240 MHz dual-core processor and floating-point unit, which are necessary to execute the Extended Kalman Filter of Chapter 3 in real time. However, the ESP32-WROOM-32 has no simulation model in Proteus, so the interface circuits could not be simulated around it. For the Proteus work the ATmega328P was used instead — the microcontroller of the Arduino Uno, which Proteus models reliably and completely. This substitution is acceptable because the circuits validated here exercise the bus electrical and protocol layers and the power and level-translation stages, none of which depend on the specific processor; the results transfer directly to the ESP32, whose I^2C and UART peripherals implement the same standards. Where the ESP32 differs — for example its native 3.3 V logic, which removes the need for the level shifter of section 4.5 — this is noted in the relevant section.

4.1 The Processing of the Sensors

With the single exception of the rangefinder, every sensor selected in Chapter 3 is a digital device: the accelerometer, gyroscope, magnetometer, and barometer each integrate their transducer, amplifier, and analog-to-digital converter on-chip and communicate the result as digital data. Only the ultrasonic/IR rangefinder produces an analog voltage requiring the classical conditioning chain. This split has an important consequence for the conditioning task.

Of the four digital sensors, three — the MPU6050 IMU, the HMC5883L magnetometer, and the MS5611 barometer — communicate over the I^2C bus, while the NEO-6M GPS communicates over a USART (UART) serial link using NMEA text. The two protocols are summarised below, as they underpin the circuits that follow.

4.1.1 The I²C Protocol

I²C (Inter-Integrated Circuit) is a two-wire, multi-device, synchronous serial bus. One line, SDA, carries data; the other, SCL, carries the clock generated by the master. Its defining feature is that every device drives the lines open-drain: a device can only pull a line low; the line is returned high by external pull-up resistors. This makes it electrically impossible for two devices to conflict - the bus reads low if any device pulls it low, and high only when all release - so many devices share the same two wires. Each device carries a unique 7-bit address, after a start condition the master transmits the address, and only the device whose address matches responds by acknowledging (pulling SDA low on the ninth clock). This selective addressing is what allows the three sensors to coexist on one bus. Because the pull-up resistors alone perform every low-to-high transition, their value directly governs the bus rise time — the subject of section 4.3.

4.1.2 The USART Protocol

USART (Universal Synchronous/Asynchronous Receiver-Transmitter) is a point-to-point serial link. In the asynchronous mode used by the GPS, there is no shared clock; instead both ends agree in advance on a baud rate (9600 bit/s for the NEO-6M), and each byte is framed by a start bit and a stop bit so the receiver can re-synchronise on every character. The GPS simply transmits ASCII NMEA 0183 sentences continuously — no command or addressing is required. Because timing is agreed rather than shared, correct operation depends critically on both ends running at the same effective clock, a point that becomes practically important in section 4.6.

4.2 Circuit 1 — Power Supply Conditioning

Why it was needed. The sensors are powered from the vehicle's electronic speed controllers, whose switching regulators produce a supply rail heavily contaminated with high-frequency noise. Feeding this directly to the sensors would corrupt their internal measurements and ADC conversions. Circuit 1 conditions the raw 5 V rail into a clean, regulated 3.3 V supply.

What it does. The chain is: 5 V input → ferrite bead → LC low-pass filter → LM1117 3.3 V low-dropout regulator → decoupling capacitors → sensor load. The 100 μH inductor together with the 10 μF bulk capacitor forms an LC low-pass with cutoff

$$f_c = \frac{1}{2\pi\sqrt{LC}} = \frac{1}{2\pi\sqrt{100\mu\text{H} \cdot 10\mu\text{F}}} \approx 5.0 \text{ kHz},$$

which attenuates the regulator's ∼100 kHz switching ripple by roughly 52 dB. The ferrite bead (≈300 Ω at 100 MHz) targets the higher-frequency conducted EMI that the LC filter cannot reach, and the LM1117's power-supply rejection (∼60 dB at low frequency) removes what remains. The sensor load — dominated by the GPS at ∼40 mA of the ∼44 mA total — is represented by a resistor.

Why it matters. Clean power is the foundation of every downstream measurement. Supply noise appearing on a sensor's reference directly becomes measurement error, the value of a precise conditioning chain is wasted if the sensor is fed a dirty rail. This circuit is therefore built first, because every subsequent circuit depends on it.

We model the sensors with a 75 Ω resistor. **Why 75 Ω?** It fakes your sensors drawing current.Add them up:

Sensor	**Current**
MPU6050	3.9 mA
HMC5883L	0.1 mA
MS5611	∼0.01 mA
NEO-6M	∼40 mA
Total	**≈ 44 mA**

3.3 V ÷ 0.044 A = **75** Ω. — the GPS dominates by 10×, which is worth noticing.

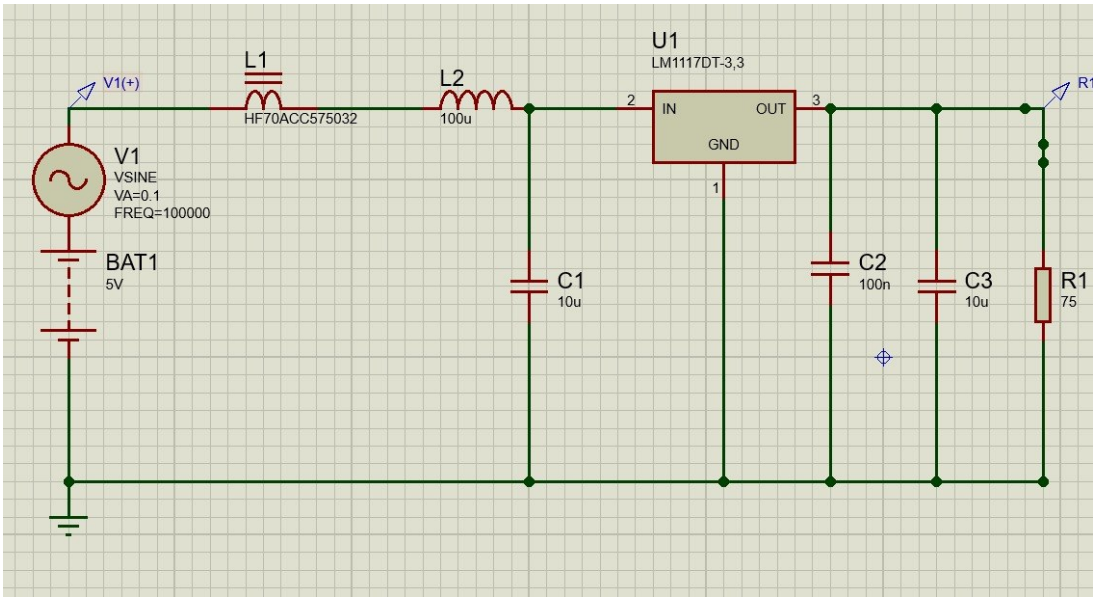


Figure 4.1: Circuit 1 — power-supply conditioning: ferrite bead, LC filter, LM1117 regulator, and decoupling.

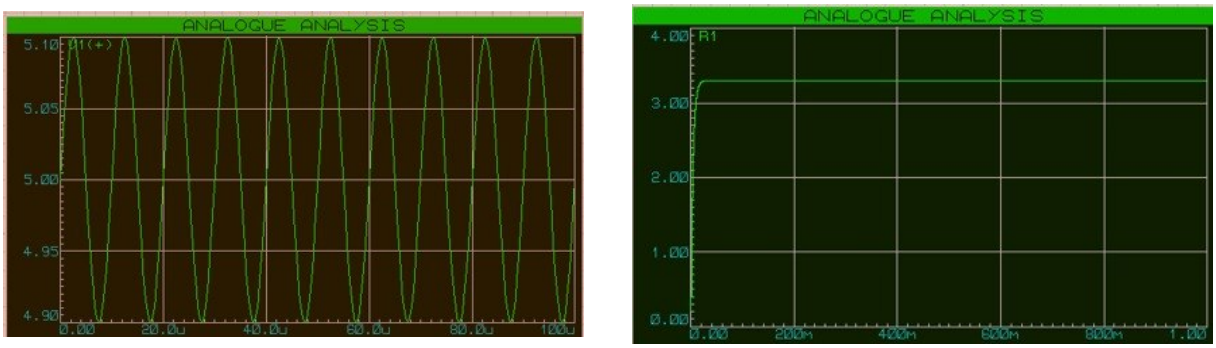


Figure 4.2: Circuit 1 simulation: oscilloscope showing the noisy input rail (Channel A) versus the clean regulated output (Channel B).

4.3 Circuit 2 — I²C Pull-up Rise-Time Characterisation

Why it was needed. The three I²C sensors share one bus whose high-going edges are produced entirely by pull-up resistors charging the bus capacitance. If the resistors are too large, the edges are too slow and a receiving device samples a bit before it has settled, corrupting data intermittently — the hardest kind of fault to diagnose. Circuit 2 designs the pull-up value from specification and verifies it by measurement.

What it does. The pull-up value is bracketed by two constraints. The minimum is set by the current a device can sink at a valid low level, $R_{p(min)} = (V_{dd} - V_{ol})/I_{ol} = (3.3 - 0.4)/3\ \text{mA} \approx 967\ \Omega$. The maximum is set by the rise-time budget: for 400 kHz Fast-mode the rise time must stay below 300 ns, and with three sensors plus wiring the bus capacitance is about 150 pF, giving $R_{p(max)} = 300\ \text{ns}/(0.8473 \cdot 150\ \text{pF}) \approx 2.36\ \text{k}\Omega$. Between this maximum and minimum, a value of 1.8 kΩ is chosen. The circuit drives the SDA line through an open-drain output (mimicking a real I²C device) and measures the rising edge on an oscilloscope.

The predicted rise time follows $t_r = 0.8473RC$.

Pull-up	**Calculated t_r (30–70%)**	**400 kHz limit (300 ns)**	**Verdict**
1.8 kΩ (chosen)	229 ns	≤ 300 ns	Pass
4.7 kΩ	597 ns	≤ 300 ns	Fail — 2× over

Table 4.1: Table 4.1. Measured/predicted I²C rise time: the designed value passes; the common 4.7 kΩ default fails at 400 kHz on a three-device bus.

Why it matters. This is the clearest instance in the project of designing a component from a specification rather than copying a common value — and demonstrating, with numbers, that the common value violates the specification for this faster, more heavily loaded bus. A note on measurement is worth recording: the I²C specification defines rise time between the 30 % and 70 % thresholds, not the more familiar 10–90 %; using the wrong threshold definition would make a compliant bus appear to fail.

As mentioned before, we simulated the microcontroller in Proteus, using the ATMEGA328P, but we use the ARDUINO IDE to write the code for the microcontroller, so it is of importance to know how the pins on an Arduino UNO map to pins of an ATMEGA328P. That is shown in the table below:

Arduino label	**This symbol**	**Pin #**
A4 (SDA)	PC4/ADC4/SDA	**27**
A5 (SCL)	PC5/ADC5/SCL	**28**
D0 (RX)	PD0/RXD	30
D1 (TX)	PD1/TXD	31
RESET	PC6/RESET	29

Table 4.2: Arduino UNO to ATMEGA328P pin mapping

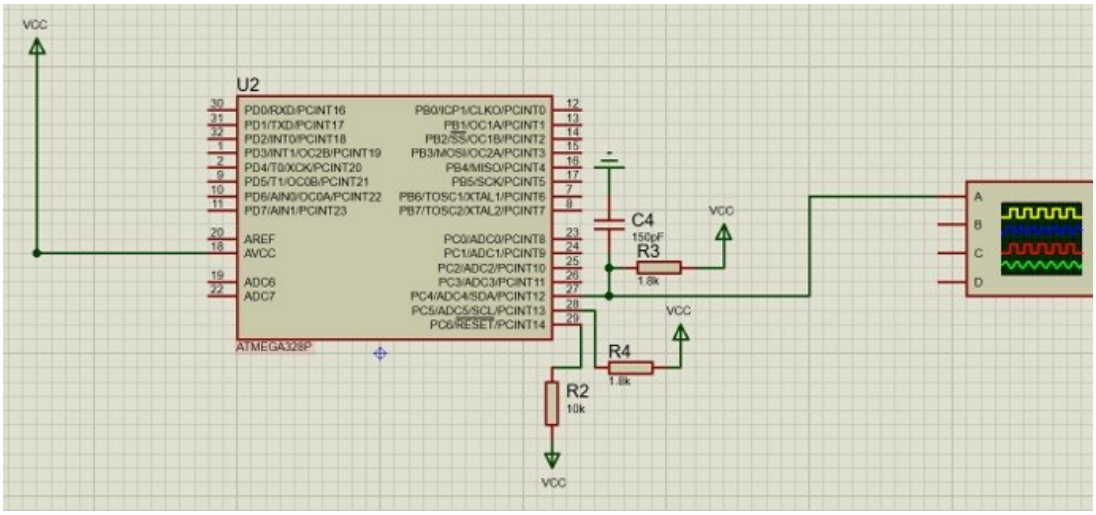


Figure 4.3: Circuit 2 — I^{2}C pull-up test: open-drain driver, 1.8 kΩ pull-up, and 150 pF bus-capacitance model.

4.4 Circuit 3 — Multi-Device I^{2}C Bus Validation

Why it was needed. Circuit 2 proved the bus is electrically fast enough; Circuit 3 proves the protocol works — that several devices can share the two wires and be addressed individually without conflict. These are independent failure modes: a bus can have perfect edges yet still fail because of an address clash or a broken acknowledge.

What it does. Because Proteus has no models of the actual sensor dies, three 24C02 EEPROMs — real I^{2}C slaves with configurable addresses — are placed on the bus at distinct addresses (0x50, 0x51, 0x52) as electrical and protocol stand-ins. An I^{2}C scanner running on the microcontroller probes all 127 addresses, and an I^{2}C debugger instrument captures the raw bus traffic. The scanner reports the three devices; the debugger shows the start condition, address byte, and acknowledge/not-acknowledge for each, confirming that present devices answer and absent addresses do not.

Why it matters. It validates the core architectural claim of Option A from Chapter 3 — three sensors on one two-wire bus — at the protocol level: selective addressing, the acknowledge handshake, and bus sharing all function with the pull-up network designed in Circuit 2. The substitution of EEPROMs for sensor dies is legitimate because what is being tested is the bus, not the sensor: an EEPROM acknowledges at its address in exactly the way an MPU6050 acknowledges at 0x68; only the data exchanged afterward differs.

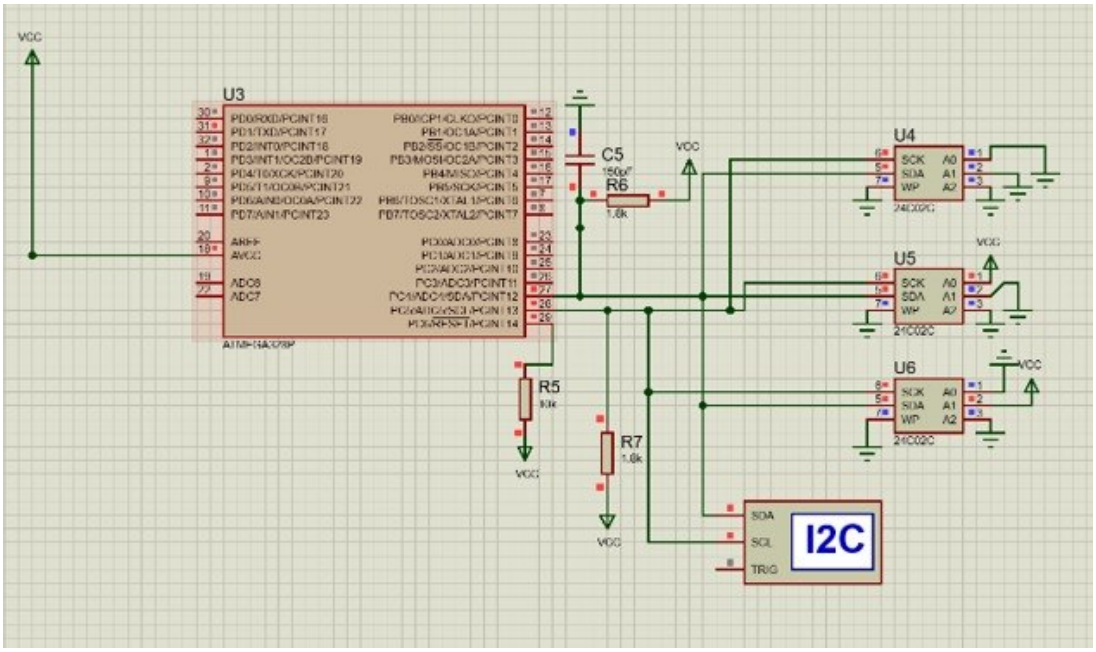


Figure 4.4: Circuit 3 — three 24C02 EEPROMs sharing the I^{2}C bus at addresses 0x50–0x52, with the microcontroller, pull-ups, and I^{2}C debugger.

```
#include <Wire.h>

void setup() {
Wire.begin();
Wire.setClock(400000);
Serial.begin(9600);
delay(500);
Serial.println("Scanning I2C bus...");

int found = 0;
for (byte addr = 1; addr < 127; addr++) {
Wire.beginTransmission(addr);
byte error = Wire.endTransmission();
if (error == 0) {
Serial.print("Device found at 0x");
if (addr < 16) Serial.print("0");
Serial.println(addr, HEX);
found++;
}
}
Serial.print("Total devices: ");
Serial.println(found);
}

void loop() {}
```

How it works, in one line: it tries every address 1–126, sends just an address byte, and checks whether anything ACKs. `endTransmission()` returns 0 only if a device pulled SDA low to acknowledge. That's the entire mechanism of I^{2}C device discovery.

Note `Wire.setClock(400000)` — this ties Circuit 3 back to Circuit 2. You're running the bus at the speed your pull-up calculation assumed.

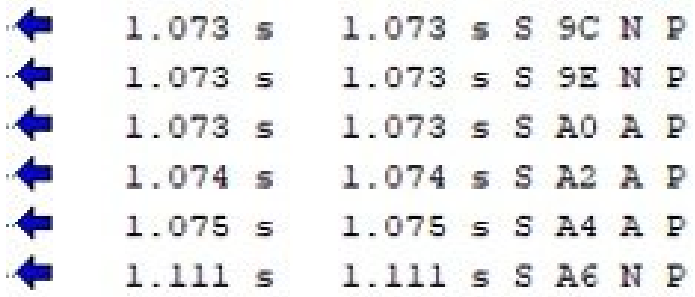


Figure 4.5: Circuit 3 simulation: the I²C scanner terminal output listing the three devices, and the I²C-debugger transaction trace.

4.5 Circuit 4 — Logic-Level Translation

Why it was needed. The simulation microcontroller (ATmega328P / Arduino Uno) operates at 5 V logic, whereas the bare sensor dies operate at 3.3 V with an absolute-maximum input near 3.6 V. Driving a 3.3 V sensor pin with 5 V exceeds its rating and can destroy it. Circuit 4 translates logic levels between the two rails.

What it does. Because I²C is bidirectional and open-drain, a simple resistor divider is unsuitable — it is one-directional and would load the bus. The standard solution, a single BSS138 MOSFET translator per line (with a pull-up to 3.3 V on the low side and to 5 V on the high side), is used. When either side pulls low, the MOSFET conducts — via its channel when the 3.3 V side is driven, and via its body diode followed by the channel when the 5 V side is driven — so the low level is transferred in both directions while the idle-high levels are set independently by the two pull-ups. The simulation drives one side with a pulse and confirms the other follows at the correct voltage.

Why it matters. It protects the sensors from over-voltage while preserving the bidirectional open-drain behaviour I²C requires — something a resistor divider cannot do. It is worth noting that this circuit is an artifact of the 5 V simulation platform: the target ESP32 is natively 3.3 V, so on the final hardware the level shifter is unnecessary for the sensors that share its rail. The circuit is nonetheless a genuine and reusable design, required whenever a 5 V and a 3.3 V I²C device must interoperate.

How the circuit Works

Three cases, and all three work:

- **Case 1 — bus idle, both sides high.**
  Source sits at 3.3 V (its pull-up). Gate is at 3.3 V. So $\mathbf{V_{GS} = 0}$ → FET off. Drain floats up to 5 V via its own pull-up. Both sides high. (OK)

- **Case 2 — a 3.3 V device pulls the low side down.**
  Source goes to ∼0 V. Gate is still 3.3 V. Now $\mathbf{V_{GS} = 3.3}$ V, well above the threshold (∼1 V) → FET turns **on** → it conducts, dragging the drain down too. High side follows low. (OK)

- **Case 3 — the 5 V device (your Arduino) pulls the high side down.**
  Drain goes to ∼0 V. The FET is off, so how does the low side know? **The body**

diode. Every N-MOSFET has an intrinsic diode from source to drain. When the drain drops below the source, that diode conducts, pulling the source down to about 0.7 V. That drop is enough to make V_{GS} exceed the threshold → the FET turns properly on → the channel now conducts and both sides go to near 0 V. (OK)

- Note that this level shifter is bidirectional; it works when the micro is deriving the voltage and when the micro is receiving the voltage.

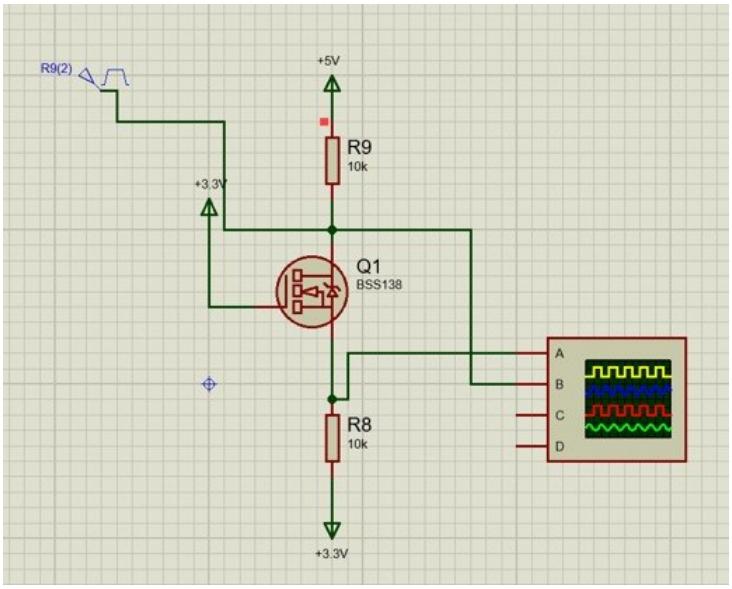


Figure 4.6: Circuit 4 — BSS138 bidirectional level shifter between the 5 V and 3.3 V I^{2}C rails.

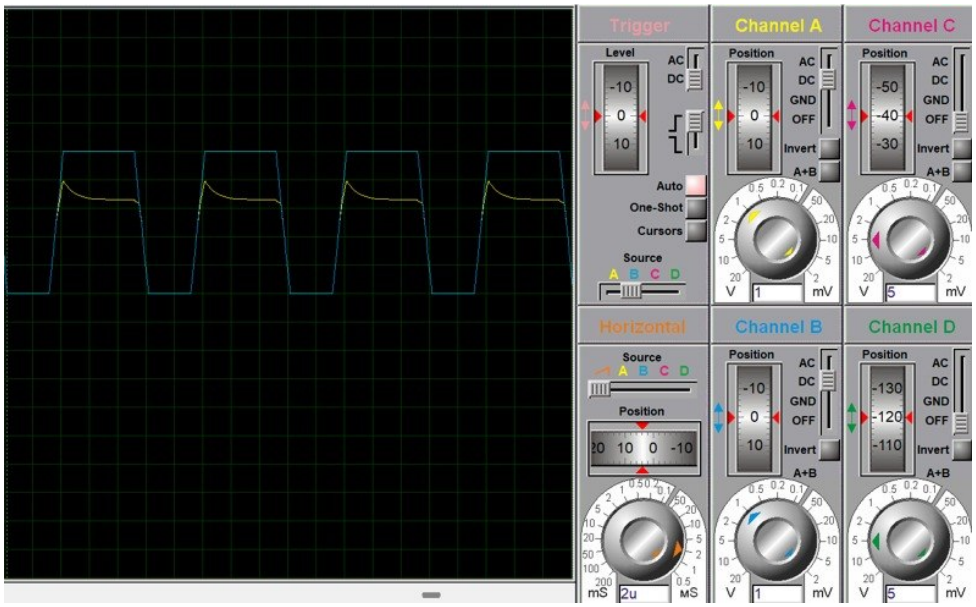


Figure 4.7: Circuit 4 simulation: the 3.3 V side and 5 V side tracking each other in both translation directions.

4.6 Circuit 5 — GPS Reception and LCD Display (UART/NMEA)

Why it was needed. The GPS is the one digital sensor on a UART rather than I^2C, and its data is human-readable NMEA text. Circuit 5 validates the entire GPS interface — reception, parsing, and display — and doubles as the sensor-value display required by the project (LCD read-out).

What it does. The microcontroller receives NMEA sentences on its UART at 9600 baud, parses the $GPGGA sentence to extract latitude and longitude, and displays them on a 16×2 character LCD in 4-bit mode. Because Proteus has no NEO-6M model, a virtual terminal injects real NMEA sentences into the UART — the exact ASCII a NEO-6M transmits — so the protocol, baud rate, parsing, and display are validated end-to-end. Only the radio front-end is absent; everything on the wire is genuine. A no-fix sentence (quality field 0) is also injected to confirm the firmware detects loss of fix and displays a warning rather than stale coordinates.

Why it matters. This is the most complete of the digital-sensor circuits: nothing is approximated except the radio itself. It confirms the UART link, NMEA framing, field parsing, and LCD output all function, and it demonstrates the fault-detection logic (loss-of-fix) that feeds the fail-safe behaviour described in Chapter 3. A practical lesson emerged here: because UART is asynchronous, correct reception depends on the microcontroller's clock being configured to the true 16 MHz — a mismatched clock corrupts the baud rate and yields unreadable output, whereas the synchronous I^2C bus is immune to this class of error.

For the simulation the NMEA code
`$GPGGA,123519,4807.038,N,01131.000,E,1,08,0.9,545.4,M,46.9,M,,*47` was given to the virtual terminal, once you press enter the terminal starts transmitting data to the micro, the micro reads the data and deciphers the code to get the latitude and longitude.

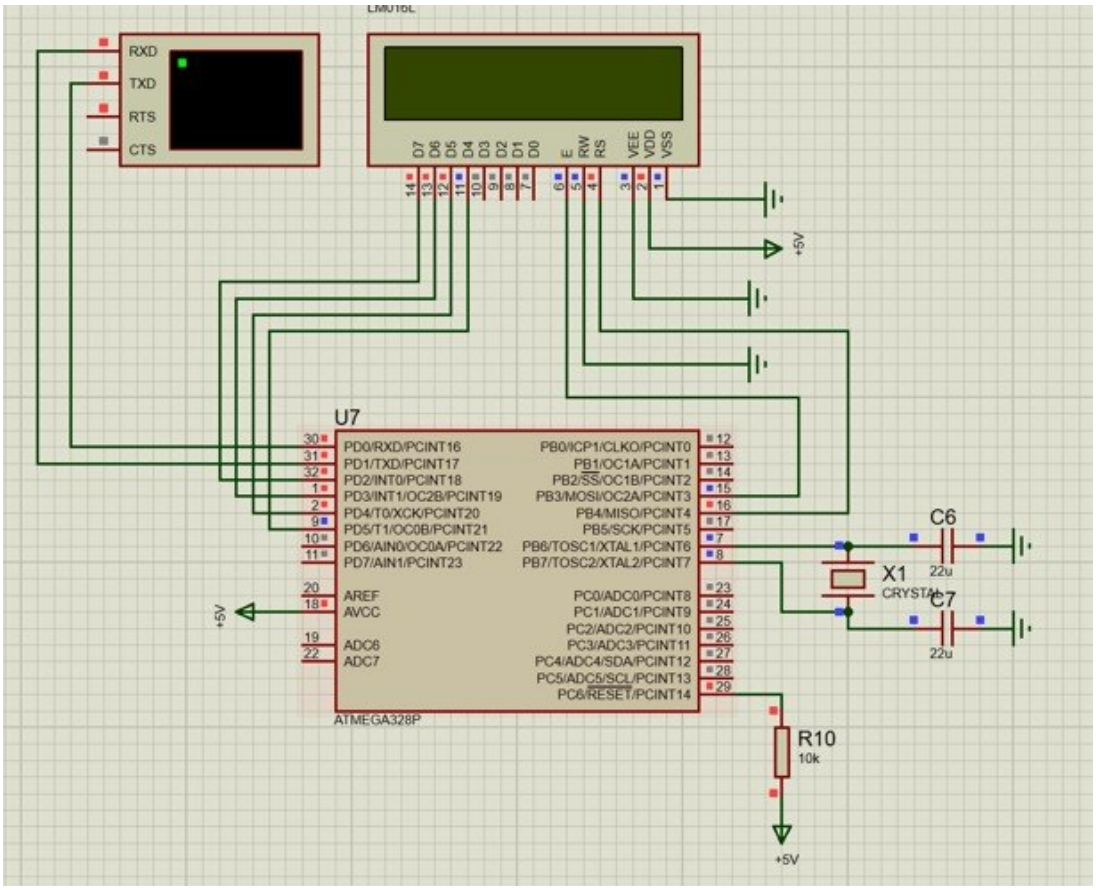


Figure 4.8: Circuit 5 — GPS/UART interface: microcontroller, 16×2 LCD in 4-bit mode, and virtual terminal injecting NMEA sentences.

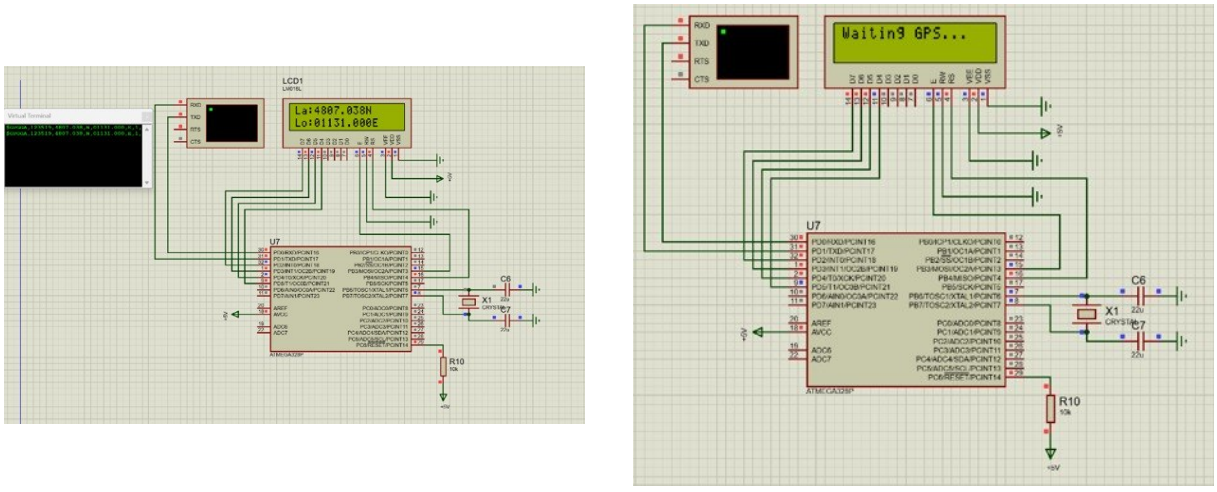


Figure 4.9: Circuit 5 simulation: the LCD displaying parsed latitude and longitude from an injected $GPGGA sentence (and the no-fix warning case).

The microcontrollers code is as followed:

```
#include <LiquidCrystal.h>
LiquidCrystal lcd(12, 11, 5, 4, 3, 2);
String line = "";
```

```
void setup() {
lcd.begin(16, 2);
Serial.begin(9600);
lcd.print("Waiting GPS...");
}
void loop() {
while (Serial.available()) {
char c = Serial.read();
if (c == '\n' || c == '\r') {
if (line.startsWith("$GPGGA")) {
String f[15];
int idx = 0, start = 0;
for (int i = 0; i < line.length() && idx < 15; i++) {
if (line[i] == ',') {
f[idx++] = line.substring(start, i);
start = i + 1;
}
}
if (f[6] == "0") {                    // fix quality field
lcd.clear();
lcd.print("NO FIX");
} else {
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("La:"); lcd.print(f[2]); lcd.print(f[3]);
lcd.setCursor(0, 1);
lcd.print("Lo:"); lcd.print(f[4]); lcd.print(f[5]);
}
}
line = "";
} else {
line += c;
}
}
}
```

The code starts by reading the serial input pin, when ever c equals “\n” or “\r” we know that the transmition is over and we must read the line if c is not either of those, means it’s a genuine character, so it is added to the “line” string. Once we reach “\n” or “\r”, we check the “line” string for the key string ”$GPGGA” which indicated the GPS is sending data, then we go through the line, separating each individual phrase in the code. (each part of the code is separated with “,”) then once we have successfully divided the code into the needed sections we take the latitude as the combination of f[2] and f[3] and the longitude as f[4] and f[5] and if f[6] is zero, then there is a problem and we output no fix.

4.7 Circuit 6 — Analog Conditioning of the Rangefinder

Why it was needed. The rangefinder is the only analog sensor, and it is therefore the only true analog signal-conditioning problem in the project. Its small, slowly-varying output voltage must be scaled to fill the microcontroller's ADC range and filtered before sampling. This section develops the conditioning for the MaxBotix MB1240 ultrasonic rangefinder — the sensor originally specified — comprising an anti-aliasing Butterworth filter and a zero-suppression-and-span amplifier.

Note: As recorded in Chapter 3, the MB1240 could not be sourced and was replaced in hardware by the nonlinear GP2Y0A21YK, whose conditioning drops the zero/span stage in favour of a firmware linearisation. This section documents the MB1240 design.

4.7.1 The Sensor and its Signal

The MB1240 outputs an analog voltage scaled at $V_{cc}/1024$ per cm — about 4.88 mV/cm at a 5 V supply — over a full-scale range of roughly 20–765 cm, updated at 10 Hz. For the landing regime of interest (20–200 cm) the raw output spans only

$$V(20\text{ cm}) = 20 \cdot 4.88\text{ mV} = 97.6\text{ mV} \quad \text{to} \quad V(200\text{ cm}) = 200 \cdot 4.88\text{ mV} = 976\text{ mV},$$

i.e. about 878 mV out of the ADC's 5 V range — under 18 %. Two conditioning stages are required: a low-pass filter to prevent aliasing, and a zero/span amplifier to expand this narrow band to the full ADC range.

4.7.2 Design of the Anti-Aliasing Butterworth Filter

Transfer function and its features

A second-order low-pass Sallen-Key filter with unity gain is used. Its transfer function is

$$H(s) = \frac{\omega_c^2}{s^2 + (\omega_c/Q) \cdot s + \omega_c^2},$$

where ω_c is the cutoff frequency and Q the quality factor. The Butterworth case is defined by $Q = 1/\sqrt{2} \approx 0.707$, which gives the maximally flat passband — no ripple and no peaking before the cutoff — at the cost of a moderate transition slope. This flatness is the reason the Butterworth response is preferred for a measurement channel: it does not distort the in-band signal. Above cutoff the second-order response rolls off at 40 dB/decade. For the equal-resistor Sallen-Key realisation, matching the denominator to the standard form yields

$$\omega_c = \frac{1}{\sqrt{R_1 R_2 C_1 C_2}}, \quad Q = \frac{\sqrt{R_1 R_2 C_1 C_2}}{C_1(R_1 + R_2)}.$$

Setting $R_1 = R_2 = R$ simplifies the quality factor to $Q = \frac{1}{2}\sqrt{C_2/C_1}$, so the Butterworth condition $Q = 0.707$ is met precisely when $C_2 = 2 \cdot C_1$ — the shunt capacitor is twice the feedback capacitor. This single ratio is the heart of the design.

Parameter design at two cutoff frequencies

The lower bound on the cutoff is the signal bandwidth (the descent produces a ∼1–2 Hz signal); the upper bound is the Nyquist frequency of the ADC sampling. Two designs are carried out. In each, the capacitor values are chosen first (their tolerances are coarse, ±10 %) and the resistor is solved for (resistors are available to ±1 %):

Design A — $f_c = 5$ Hz. Choosing $C_1 = 1$ μF (feedback) and $C_2 = 2.2$ μF (shunt, nearest standard value to $2 \cdot C_1$):

$$R = \frac{1}{2\pi \cdot f_c \cdot \sqrt{C_1 C_2}} = \frac{1}{2\pi \cdot 5 \cdot \sqrt{1\mu \cdot 2.2\mu}} \approx 21.5 \text{ k}\Omega, \quad Q \approx 0.74.$$

Design B — $f_c = 10$ Hz. Keeping the same capacitors:

$$R = \frac{1}{2\pi \cdot 10 \cdot \sqrt{1\mu \cdot 2.2\mu}} \approx 10.7 \text{ k}\Omega, \quad Q \approx 0.74.$$

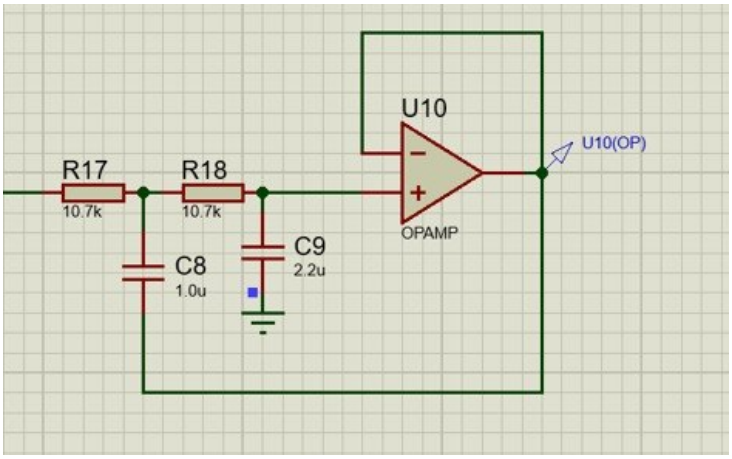
The realised $Q \approx 0.74$ (against the ideal 0.707) arises from rounding C_2 to the standard 2.2 μF rather than an exact 2.0 μF; the resulting $\sim$0.1 dB of passband peaking is negligible, and the design is tolerance-limited rather than ratio-limited.

Trade-off between the two cutoffs

The choice between 5 Hz and 10 Hz is a genuine trade-off between noise rejection and delay:

- **Anti-alias rejection.** At the ADC's Nyquist frequency the 5 Hz design attenuates more strongly than the 10 Hz design (for a given sample rate, halving the cutoff adds roughly 12 dB of rejection at Nyquist). The 5 Hz filter is therefore the safer anti-alias barrier.
- **Group delay.** A second-order Butterworth has group delay $\approx \sqrt{2}/(2\pi \cdot f_c)$, giving about 45 ms at 5 Hz versus 22 ms at 10 Hz. The lower cutoff doubles the lag, which the altitude loop must tolerate as added phase.
- **Signal fidelity.** Both cutoffs sit comfortably above the $\sim$1–2 Hz descent signal, so neither distorts the measurement; the distinction is purely rejection-versus-delay.

In summary, 5 Hz is preferred when clean sampling matters most and the extra $\sim$20 ms of delay is affordable, while 10 Hz is preferred when minimising phase lag in the altitude loop is the priority. For this vehicle the 5 Hz design is the reasonable default, with 10 Hz held as the lower-delay alternative.



4.7.3 Design of the Zero-Suppression and Span Circuit

The filtered signal still occupies only 97.6 mV–976 mV of the 5 V ADC range. The zero/span stage first suppresses the 97.6 mV offset (so that 20 cm maps to 0 V) and then

applies a span gain to stretch the remaining band to the full 5 V (so that 200 cm maps to 5 V). The required span gain is

$$\text{Gain} = \frac{5.000\ \text{V}}{(0.976 - 0.0976)\ \text{V}} = \frac{5.000}{0.878} \approx 5.7,$$

implemented as a difference amplifier that subtracts a 97.6 mV reference (derived from a buffered divider) from the signal and amplifies the difference by 5.69. The transfer characteristic is $V_{out} = (V_{in} - 0.0976) \cdot 5.69$, verified at the endpoints: 20 cm $\rightarrow$ 0 V, 200 cm $\rightarrow$ 5.00 V, 110 cm $\rightarrow$ 2.50 V.

To get the 5.7 gain, we use an operational amplifier with $R_1 = 10$ kΩ and $R_2 = 57$ kΩ, which gives a gain of $R_2/R_1 = 5.7$. For the creation of the 0.0976V reference, we take a 2.5V input voltage and use a voltage divider circuit to get a 0.0976V reference, and we use a buffer to transfer the voltage without any distortions to the gain amplifier. We can use an LM4040-2.5 from Texas Instruments for the 2.5V reference.

The question is why we used a 2.5V reference voltage?

The answer is that we want the voltage to precisely become 0.0976V, but the motor load would cause massive voltage sag and spikes on the 5V rail, which in turn would change the 0.0976V voltage and the entire zero and span circuit will be in jeopardy. So, we use the LM4040-2.5 which gives a precise 2.5V. This expands the usable resolution by the span factor of 5.69, improving the effective quantisation from about 1.0 cm/count to 0.18 cm/count on a 10-bit ADC. A unity-gain buffer precedes the amplifier so that the sensor's output impedance does not corrupt the gain, and the reference divider is likewise buffered to hold the zero point stable.

Why it matters. Together the two stages embody the classical instrumentation chain: the Butterworth filter guarantees the sampled signal is free of aliasing, and the zero/span amplifier ensures the ADC's full resolution is devoted to the range of interest rather than wasted on unused input span. This is the one place in the project where the analog signal-conditioning techniques of the course are applied in full.

Note that the input of circuit is generated using a PWLIN block in proteus. We used a MATLAB code to generate a .txt file representing the sensor values and gave the .txt file to the PWLIN block.

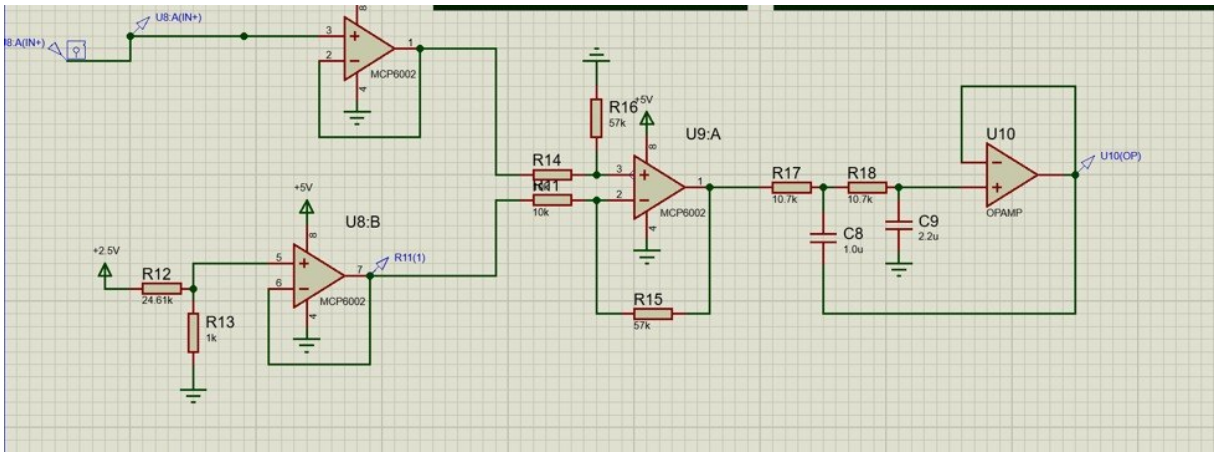


Figure 4.10: Circuit 6 — MB1240 conditioning chain: input buffer, zero-suppression-and-span difference amplifier, and second-order Sallen-Key Butterworth low-pass filter.

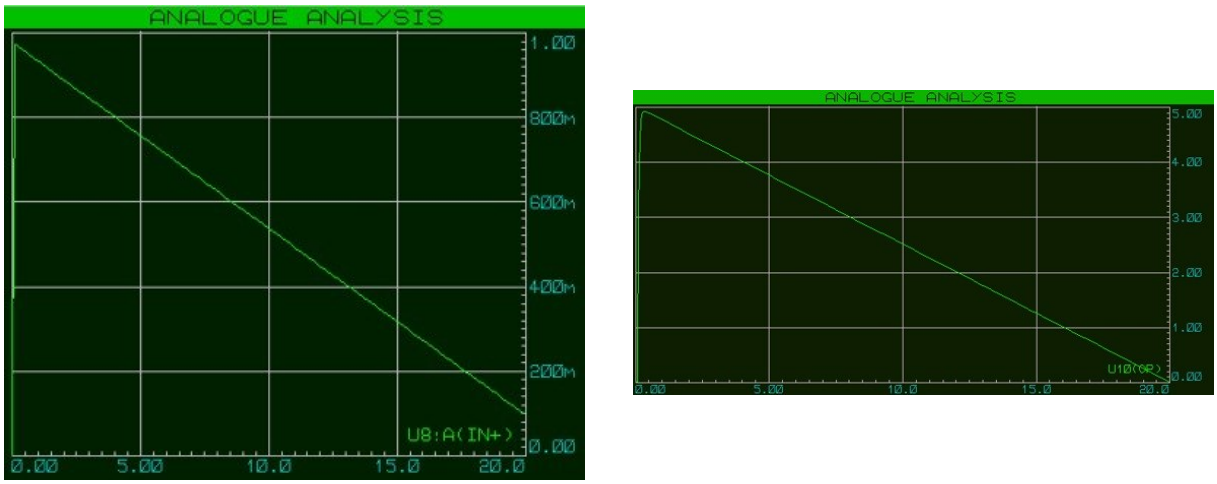


Figure 4.11: Circuit 6 simulation: the raw sensor voltage (≈98–976 mV) and the conditioned output spanning the full 0–5 V ADC range; and the filter's frequency response showing −3 dB at the design cutoff.

4.8 Chapter Summary

#	**Circuit**	**Purpose**	**Key result**
1	Power supply	Clean 3.3 V from a noisy rail	∼52 dB ripple rejection (LC) + LDO
2	I^2C pull-up	Size pull-ups from spec	1.8 kΩ → 229 ns (pass); 4.7 kΩ fails
3	Multi-device bus	Validate 3-device addressing	3 slaves enumerated; ACK verified
4	Level shifter	5 V↔3.3 V translation	Bidirectional transfer confirmed
5	GPS + LCD	UART/NMEA reception & display	Lat/lon parsed; loss-of-fix detected
6	Rangefinder	Analog zero/span + filter	0–5 V span; Butterworth at 5/10 Hz

Table 4.3: Table 4.2. Summary of the six conditioning circuits and their principal results.

A note on methodology closes the chapter. Proteus does not include behavioural models for the MPU6050, MS5611, HMC5883L, or NEO-6M, because these are digital sensors with integrated ADCs and no published models. The digital-sensor interfaces were therefore validated in three layers — electrical (pull-up sizing verified against rise-time specification; power-supply noise rejection; level translation), protocol (multi-device I^2C addressing and acknowledge captured with the debugger; UART/NMEA parsing validated end-to-end), and data (sensor values represented by the validated models of Chapter 3). This layered approach turns a tooling limitation into a deliberate validation strategy, and together with the single genuinely analog circuit (the rangefinder), it establishes that every sensor in the suite can be interfaced to the flight controller cleanly, safely, and at the correct signal integrity.

Chapter 5

Selecting the Required Actuators

5.1 Chapter Introduction

For a quadcopter, the primary actuators are not merely a collection of simple isolated components, but rather a complete, integrated electro-mechanical chain dedicated to thrust generation and attitude control. The mathematical commands computed by the flight controllers must be translated into physical aerodynamic forces with minimal delay. This chapter outlines the physical execution layer of the project, detailing the motors, propellers, electronic speed controllers (ESCs), and the software mixing matrix that glues them together into a cohesive actuation system.

5.2 Brushless DC Motors (BLDC)

Brushless DC motors provide the rotational mechanical force necessary for flight. Unlike brushed motors, BLDCs utilize electronic commutation rather than mechanical brushes, resulting in a significantly higher power-to-weight ratio, greater efficiency, and a longer lifespan. In a quadcopter, altering the speed of each individual motor changes the thrust generated at that specific arm, thereby enabling the 6-DOF control of the vehicle's altitude and attitude.

Proposed Model: A2212 1000KV

The A2212 is a highly ubiquitous outrunner motor for educational and mid-sized quadcopters, chosen for its adequate thrust generation, affordability, and reliable torque curve. The "1000KV" rating indicates the motor's velocity constant—meaning the motor will spin at approximately 1000 revolutions per minute (RPM) per volt applied under no-load conditions. The theoretical no-load speed is calculated as:

$$\text{RPM} = KV \times V_{supply}$$

When powered by a standard 3S LiPo battery (11.1 V), it achieves a nominal no-load speed of roughly 11100 RPM. Under load (with a propeller attached), the speed decreases, but the 1000KV rating provides an optimal balance between torque and angular velocity, ensuring rapid response to the controller's step commands without drawing excessive current.

5.3 Propellers and Aerodynamic Linkage

Propellers act as the aerodynamic linkage of the system; they are responsible for converting the rotational mechanical energy of the BLDC motors into downward air displacement, thereby generating upward lift. The thrust T produced by a propeller is highly nonlinear and is generally proportional to the square of its angular velocity ω (per motor):

$$T = k_f \omega^2$$

where k_f is the aerodynamic lift factor. The selection of the propeller's diameter and pitch has a profound impact on this factor, heavily influencing the total thrust, current draw, and overall flight stability.

Proposed Model: 1045 and 1045R

These propellers are exceptionally common for lightweight quadcopters. The designation "10x4.5" implies a diameter of 10 inches and a pitch of 4.5 inches. The 10-inch diameter provides a large disc area, generating substantial thrust at lower RPMs, which is highly efficient for stable hovering. The 4.5-inch pitch (the theoretical forward distance the propeller would advance in one revolution) prevents the motor from stalling and ensures smooth, predictable airflow.

In the X-wing configuration, two standard (tractor) and two reverse (pusher, denoted by "R") propellers are required. Placing them diagonally opposite each other ensures that the reactive aerodynamic drag torques naturally cancel out during a level hover, preventing unwanted yaw rotation.

5.4 Electronic Speed Controllers (ESC)

The Electronic Speed Controller (ESC) serves as the vital intermediary between the flight controller and the motor. A microcontroller cannot drive a 3-phase BLDC motor directly; the ESC receives the low-voltage control signals (typically PWM) from the controller and dynamically adjusts the timing and power of the 3-phase AC signals applied to the motor's stator coils.

Proposed Model: SimonK 30A or BLHeli 30A

These 30-ampere ESCs are equipped with multirotor-specific firmware (SimonK or BLHeli). Standard airplane ESCs often feature heavy internal filtering and low-voltage cutoffs. In contrast, multirotor firmwares bypass these filters to offer highly linear throttle curves and support high-refresh-rate input signals (up to 400 Hz or digital protocols like DShot), drastically reducing the actuator time constant. The 30 A rating provides a generous safety margin over the typical 12–15 A maximum current draw of the A2212 motor under heavy load, ensuring thermal stability and preventing catastrophic mid-air motor shutdowns.

5.5 The Motor Mixer: Virtual to Physical Translation

While the Motor Mixer is not a physical hardware component, it is arguably the most critical executive block in the actuation system. The cascaded flight controllers compute generalized, virtual control efforts: total thrust (U_1), roll torque (U_2), pitch torque (U_3),

and yaw torque (U_4). The Motor Mixer mathematically translates these virtual commands into four distinct, individual squared-speed commands ($\omega_1^2, \omega_2^2, \omega_3^2, \omega_4^2$) for each motor.

Operating within the X-wing (standard cross) configuration, the mixer maps the control inputs symmetrically. The inverse allocation matrix derives the required motor speeds as follows:

- **Thrust (U_1):** Distributed equally across all four motors to adjust altitude.
- **Roll (U_2):** Implemented by creating a speed differential between the left and right motors, leaving the front and rear motors unaffected.
- **Pitch (U_3):** Implemented by creating a speed differential between the front and rear motors, leaving the lateral motors unaffected.
- **Yaw (U_4):** Implemented by speeding up the clockwise-rotating motors and slowing down the counter-clockwise-rotating ones (or vice versa), exploiting the aerodynamic drag to turn the chassis without altering total vertical lift.

This block bridges the conceptual gap between the linear control laws and the physical actuation hardware, executing the commands with precision.

5.6 Chapter Summary

The physical execution of the quadcopter's flight plan relies on a robust and tightly coupled actuation chain. The A2212 1000KV BLDC motors provide the necessary rotational force with optimal torque, which the 1045/1045R propellers efficiently convert into aerodynamic lift. The fast-switching SimonK/BLHeli 30A ESCs ensure these motors respond almost instantaneously to the flight controller's high-frequency demands. Finally, the software-based Motor Mixer serves as the translator, taking the abstract attitude and altitude commands (U_1 to U_4) and decomposing them into precise individual motor speeds, enabling the stable, coordinated, and highly responsive flight of the X-wing quadcopter.

Chapter 6

Graphical User Interface (GUI)

This chapter details the design and implementation of the Ground Control Station (GCS) GUI tailored for the X-Wing VTOL Autopilot Drone Project.

6.1 Listener

To bridge the numerical simulation with the interactive GUI without interrupting the main solver loop, we created a dedicated block called `GUI_Telemetry`. This block was explicitly configured as an atomic subsystem. In Simulink, making a subsystem atomic is a strict architectural requirement because it forces the engine to execute the subsystem as a single, indivisible computational unit during each time step. By making it atomic, we can reliably attach an execution event listener to the block's `PostOutputs` phase. This ensures the listener triggers exactly once per timestep, guaranteeing that all input signals are fully computed, stable, and ready before the GUI attempts to extract them.

6.2 Designing the App

6.2.1 The Design View

The graphical interface was constructed using the MATLAB App Designer environment. To maintain a clean layout and prevent visual clutter, we created different tabs to compartmentalize the telemetry data. We brought multiple plot components into each tab to visualize the incoming data streams. Based on our architecture, the GUI is split into a Controller tab—which visualizes Control Effort (Thrust, Roll, Pitch, Yaw), State Position, State Velocity, and State Attitude—and a Sensor tab—which tracks GPS Position, GPS Velocity, IMU Acceleration, and Magnetometer Yaw.

6.2.2 The Code View

The backend logic of the App Designer GUI is structured into public properties, private properties, and private methods to encapsulate the data handling.

Properties:

- `TimeBuffer`, `UBuffer`, `StatePosBuffer`, `StateVelBuffer`, `StateYawBuffer`, `GPSPosBuffer`, `GPSVelBuffer`, `IMUAccelBuffer`, and `MagYawBuffer` are declared as public proper-

ties. These act as dynamically growing arrays to store the historical data required to draw continuous plots.

- `FastCounter` is implemented to manage the execution rates, bridging the high-frequency 0.005s Simulink sample rate with the slower 0.1s UI refresh rate.
- Private properties include `SerialDevice` for maintaining the live hardware connection, `HardwareTimer` to manage the background polling loop, `CurrentTime` to track the physical hardware run time, and `LivePlotLine` to store the X-axis time numbers for the animated line graph.

```
properties (Access = public)
% Buffers for the plots
TimeBuffer = [];
UBuffer = [];

StatePosBuffer = [];
StateVelBuffer = [];
StateYawBuffer = [];

GPSPosBuffer = [];
GPSVelBuffer = [];
IMUAccelBuffer = [];
MagYawBuffer = [];

% Counter to manage the 0.005s vs 0.1s sample rates
FastCounter = 0;
end

properties (Access = private)
SerialDevice
HardwareTimer
CurrentTime
LivePlotLine
end
```

Functions:

- `startupFcn(app)`: This function executes immediately after component creation. It first uses the `evalin` command to safely check the MATLAB base workspace for the user's `ModeSelection` variable. If the user selected 'Hardware Only' or 'Run Both', the app initializes a serial connection on COM9 at a baud rate of 115200. It resets the time tracker to zero, clears the hardware axes, and prepares a high-speed `animatedline`. It then configures and starts a 50Hz background timer (`HardwareTimer`) that executes the `ReadHardwareData` function every 0.02 seconds. A `try-catch` block wraps this routine to prevent the app from crashing if the physical ESP32 microcontroller is unplugged or unavailable.
- `ReadHardwareData(app)`: This timer callback pulls raw string data from the USB port via `readline` and converts it to a double precision number. If the received data

is a valid number (checked via `~isnan`), the function increments the `CurrentTime` by 0.02 seconds, adds the new coordinate to the animated plot line, and dynamically expands the X-axis limits so the plot scrolls continuously. It concludes with `drawnow limitrate` to force MATLAB to render the graphics immediately while allowing it to drop frames if the rendering pipeline falls behind the data stream.

```
methods (Access = private)

% Code that executes after component creation
function startupFcn(app)
%  Safely check what the user selected in the .m file
if evalin('base', 'exist(''ModeSelection'', ''var'')')
runMode = evalin('base', 'ModeSelection');
else
runMode = 'Run Both';
end

%  Only start the serial connection if requested
if strcmp(runMode, 'Hardware Only') || strcmp(runMode, 'Run Both')
try
% Open the physical connection
app.SerialDevice = serialport("COM9", 115200);
configureTerminator(app.SerialDevice, "LF");
flush(app.SerialDevice);

% Reset time tracker
app.CurrentTime = 0;

% Clear the old graph and create the high-speed animated line
cla(app.HardwareAxes);
app.LivePlotLine = animatedline(app.HardwareAxes, 'Color', 'b', 'LineWidth', 1.5);

% Create and start the 50Hz background loop
app.HardwareTimer = timer('ExecutionMode', 'fixedRate', ...
'Period', 0.02, ...
'TimerFcn', @(~,~)ReadHardwareData(app));
start(app.HardwareTimer);

catch
% Catch errors without crashing
disp('Warning: ESP32 not detected on COM9. Live hardware tab inactive.');
end
end
end

function ReadHardwareData(app)
% Pull the raw text from the USB and convert to a number
rawData = readline(app.SerialDevice);
liveDistance = str2double(rawData);
```

```
% Make sure to plot when data is not NAN
if ~isnan(liveDistance)

% Increment time by the timer period (0.02 seconds)
app.CurrentTime = app.CurrentTime + 0.02;

% Add the new point to the existing line (highly optimized)
addpoints(app.LivePlotLine, app.CurrentTime, liveDistance);

% Expand the X-axis so the line continuously scrolls right
xlim(app.HardwareAxes, [0, max(1, app.CurrentTime)]);

% Command MATLAB to render graphics now, dropping frames if needed to prevent lag
drawnow limitrate;
end
end
end
```

6.3 The Run_CoSimulation File

The `Run_CoSimulation.m` script serves as the master launch sequencer, coordinating both the GUI and the Simulink model.

The sequence operates line-by-line as follows:

1. The script initiates a `questdlg` pop-up window, prompting the user to choose between 'Simulation Only', 'Hardware Only', or 'Run Both'.
2. This choice is pushed to the base MATLAB workspace using `assignin` under the variable name `ModeSelection` so the GUI can access it during boot.
3. An `isempty` check ensures that if the user closes the dialog box without clicking an option, the script aborts via an `error` command.
4. The script instantiates the App Designer interface by calling `DroneGCS`.
5. It utilizes `load_system` to load the Simulink model named `Main_with_GuiG` into memory in the background.
6. A string variable defines the exact path to the target block: `[modelName, '/GUI_Telemetry']`.
7. If the user did not select 'Hardware Only', the script issues a `set_param` command to start the Simulink simulation.
8. A `while` loop continually checks the `SimulationStatus` parameter, utilizing a `pause(0.05)` command to wait in 50-millisecond increments until the model engine has finished compiling and is officially 'running'.
9. Once running, `add_exec_event_listener` successfully attaches the listener to the `PostOutputs` phase of the target block, passing the block data, event data, and the app object into the `updateGUI` callback function.

```
% Prompt the user to select the operating mode
modeChoice = questdlg('Which system do you want to run?', ...
'System Launch', ...
'Simulation Only', 'Hardware Only', 'Run Both', 'Simulation Only');

% Save this choice to the MATLAB workspace so the GUI can see it
assignin('base', 'ModeSelection', modeChoice);

% If the user closes the pop-up without clicking, stop the script
if isempty(modeChoice)
error('Launch canceled by user.');
end

%  Open the App
myApp = DroneGCS;

%  Get the Simulink model
modelName = 'Main_with_GuiG';
load_system(modelName);

%  Define the exact path to the Telemetry block
targetBlock = [modelName, '/GUI_Telemetry'];

%  Start the Simulation FIRST
if ~strcmp(modeChoice, 'Hardware Only')
set_param(modelName, 'SimulationCommand', 'start');
end

%  Wait for the Simulink engine to finish compiling and begin executing
while ~strcmp(get_param(modelName, 'SimulationStatus'), 'running')
pause(0.05);
end

%  NOW we attach the listener to the running block
hListener = add_exec_event_listener(targetBlock, 'PostOutputs', ...
@(block, event) updateGUI(block, event, myApp));
```

The `updateGUI` Callback: This function extracts the live simulation time via `block.CurrentTime` and reads the vectors off all six input ports of the telemetry block (representing Control Effort, States, GPS Position, GPS Velocity, IMU, and Magnetometer Yaw). It increments the `FastCounter` variable to track the 0.005s solver steps.

To optimize performance, a modulo operation (`mod(app.FastCounter, 10) == 0`) is used to down-sample the data, ensuring the UI matrices are only appended and plotted every 10th step (yielding a 20Hz refresh rate). During these updates, the new data points are concatenated to their respective public buffers. The script isolates specific state data by extracting indices 1-3 for XYZ position, 7-9 for XYZ velocity, and index 6 for Yaw from the 12-state vector. Finally, standard `plot` and `legend` commands push the updated buffers to the UI Axes on both the Controller and Sensor tabs, terminating with a `drawnow limitrate` command to execute the visual refresh.

```
% --- The Callback Function ---
function updateGUI(block, event, app)
% Get the current simulation time
currentTime = block.CurrentTime;

% Read the live values off the input ports of the GUI_Telemetry block
currentU      = block.InputPort(1).Data;
currentStates = block.InputPort(2).Data;
currentGPSPos = block.InputPort(3).Data;
currentGPSVel = block.InputPort(4).Data;
currentIMU    = block.InputPort(5).Data;
currentMagYaw = block.InputPort(6).Data;

% Increment the fast counter (runs every 0.005s solver step)
app.FastCounter = app.FastCounter + 1;

% Down-sample: Only update the UI every 10th step (20 Hz refresh rate)
if mod(app.FastCounter, 10) == 0

% ---  Append data to buffers ---
app.TimeBuffer = [app.TimeBuffer, currentTime];

% Transpose data to rows if necessary to match plot formatting
app.UBuffer = [app.UBuffer; currentU'];

% Extract X,Y,Z (Index 1,2,3) and Vx,Vy,Vz (Index 7,8,9) from 12-state vector
app.StatePosBuffer = [app.StatePosBuffer; currentStates(1:3)'];
app.StateVelBuffer = [app.StateVelBuffer; currentStates(7:9)'];
app.StateYawBuffer = [app.StateYawBuffer; currentStates(6)'];

app.GPSPosBuffer   = [app.GPSPosBuffer; currentGPSPos'];
app.GPSVelBuffer   = [app.GPSVelBuffer; currentGPSVel'];
app.IMUAccelBuffer = [app.IMUAccelBuffer; currentIMU'];
app.MagYawBuffer   = [app.MagYawBuffer; currentMagYaw];


% --- Controller Tab Plots ---
% Control Effort
plot(app.UIAxes, app.TimeBuffer, app.UBuffer);
legend(app.UIAxes, {'Thrust', 'Roll', 'Pitch', 'Yaw'}, 'Location', 'best');

% State Position
plot(app.UIAxes2, app.TimeBuffer, app.StatePosBuffer);
legend(app.UIAxes2, {'x', 'y', 'z'}, 'Location', 'best');

% State Velocity
plot(app.UIAxes3, app.TimeBuffer, app.StateVelBuffer);
legend(app.UIAxes3, {'vx', 'vy', 'vz'}, 'Location', 'best');
```

```
% State Attitude (Yaw)
plot(app.UIAxes4, app.TimeBuffer, app.StateYawBuffer);
legend(app.UIAxes4, {'Yaw (\psi)'}, 'Location', 'best');

% --- Sensor Tab Plots ---
% Sensor GPS Position
plot(app.UIAxes_2, app.TimeBuffer, app.GPSPosBuffer);
legend(app.UIAxes_2, {'x_{gps}', 'y_{gps}', 'z_{baro}'}, 'Location', 'best');

% Sensor GPS Velocity
plot(app.UIAxes2_2, app.TimeBuffer, app.GPSVelBuffer);
legend(app.UIAxes2_2, {'vx_{gps}', 'vy_{gps}', 'vz_{gps}'}, 'Location', 'best');

% Sensor IMU Acceleration
plot(app.UIAxes3_2, app.TimeBuffer, app.IMUAccelBuffer);
legend(app.UIAxes3_2, {'ax', 'ay', 'az'}, 'Location', 'best');

% Sensor Magnetometer Yaw
plot(app.UIAxes4_2, app.TimeBuffer, app.MagYawBuffer);
legend(app.UIAxes4_2, {'Yaw_{mag}'}, 'Location', 'best');

% Force the UI to refresh visually without freezing Simulink
drawnow limitrate;
end
end
```

Chapter 7

Hardware Implementation

7.1 The Circuit

- **Stage one The MB102:** this device regulates the 12V adapter voltage to a 5V or 3.3V DC supply.
- **Stage two: the buffer amplifier:** we use a LM358 operational amplifier with unit feedback as a buffer to transfer the voltage.
- **Stage 3: the butterworth filter:** as described in chapter 4, this filter is for filtering noise above 10Hz.
- **Stage 5:** the output of the filter is given to a voltage divider to multiply to output voltage by 2/3 because the pins of the ESP32 shouldn't be higher than 3.3, we use this to make sure the voltage does not get higher than it should.
- **Stage 4:** the output of the voltage divider is given to the analog pin of the ESP32.

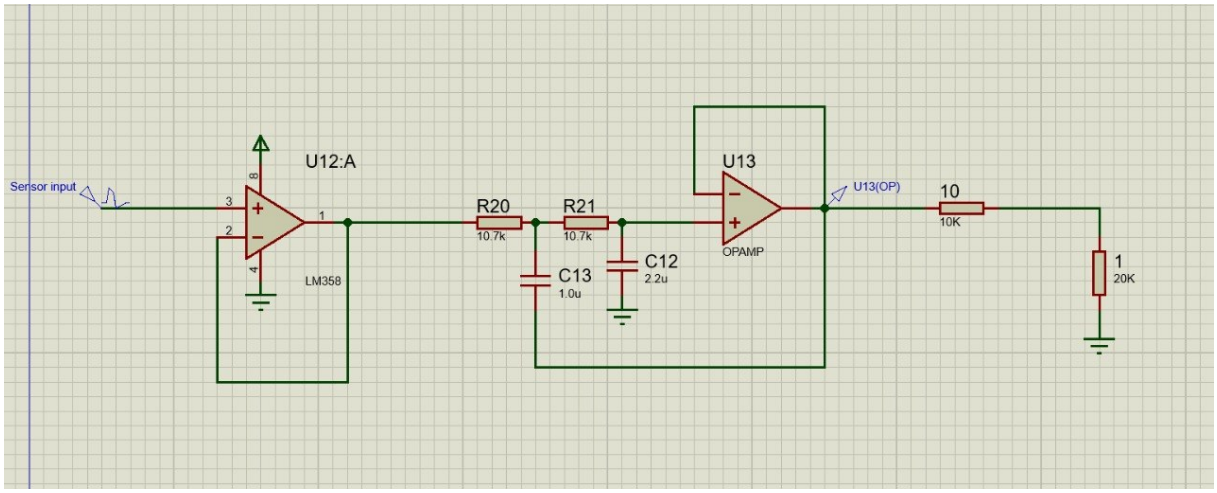


Figure 7.1: The analog conditioning circuit and voltage divider.

The code that runs inside the ESP32 is as followed:

```
// ESP32 IR Sensor Distance Calculator
#include <LiquidCrystal.h>
```

```
#include "BluetoothSerial.h"
const int analogPin = 34;
const int resolution = 4095;
const float refVoltage = 3.3;
const int rs = 13, en = 14, d4 = 25, d5 = 26, d6 = 27, d7 = 33;
LiquidCrystal lcd(rs, en, d4, d5, d6, d7);

BluetoothSerial SerialBT;

void setup() {

Serial.begin(115200);
// start bluetooth
SerialBT.begin("Drone_Telemetry");


analogReadResolution(12);
lcd.begin(16, 2);
lcd.clear();
lcd.print("System Booting..");
delay(1000);
}

void loop() {
//  Read the raw digital value from the pin
int rawADC = analogRead(analogPin);

// Prevent division by zero or errors if the sensor is completely covered
if(rawADC == 0) {
rawADC = 1;
}

//  Convert the raw digital value back into the voltage measured at the pin
float measuredVoltage = (rawADC / (float)resolution) * refVoltage;

//  Reverse the 2/3 voltage divider effect by multiplying by 1.5
float trueSensorVoltage = measuredVoltage * 1.5;

// Formula: d = 27.86 * V^(-1.15) - 0.42
float distance = 27.86 * pow(trueSensorVoltage, -1.15) - 0.42;

lcd.setCursor(0, 0);
lcd.print("Altitude:       ");
lcd.setCursor(0, 1);
lcd.print(distance);
lcd.print(" cm       ");

//  Send the final distance value over the USB cable
```

```
Serial.println(distance);
SerialBT.println(distance);

// Wait 20 milliseconds (runs the loop at 50Hz)
delay(20);
}
```

7.2 The Microcontroller Code

Line by line analysis:

1. we tell the microcontroller which pins are connected to the physical LCD using `LiquidCrystal lcd(rs, en, d4, d5, d6, d7);`.
2. A Bluetooth handle is created by the name of `SerialBT`.
3. in the setup function, we define the baude rate with `Serial.begin(115200)` then start the Bluetooth with the name drone_telementary, begin the LCD command and print system booting.
4. in the loop, the analog data is read from the analog pin(D34) (Note that the ESP32 has an internal 12bit A/D) then we take the A/D output, multiply it by V_ref and divide by 2^12(since the A/D is 12bit) this voltage was multiplied by 2/3 in the voltage divider, we now multiply it by 1.5 to get the initial voltage the sensor output. Then we use the fitted function to calculate the distance. Finally, we take the variable "distance" and we write it on the serial port using `Serial.println(distance);` we write it over Bluetooth with `SerialBT.println(distance);` and we also print on the physical LCD.

7.3 The GUI and the Proteus Simulation

7.3.1 GUI

The serial output of the micro is fed to the laptop using a USB Cable, there it can be seen either in the serial port or serial plot window in Arduino IDE or the dedicated tab on the GUI. but the dedicated tab in the GUI is really slow in plotting the sensor data because the matlab plot function is inherently heavy and the data in being transmitted really fast.

7.3.2 Proteus

Using a COMPIM block in proteus we can recive the sensor data in proteus, then the data is fed to the microcontroller using the RXD pin in both the micro and the COMPIM, then the microcontroller in proteus acts as a translator, taking the data and commanding the LCD to show the data.

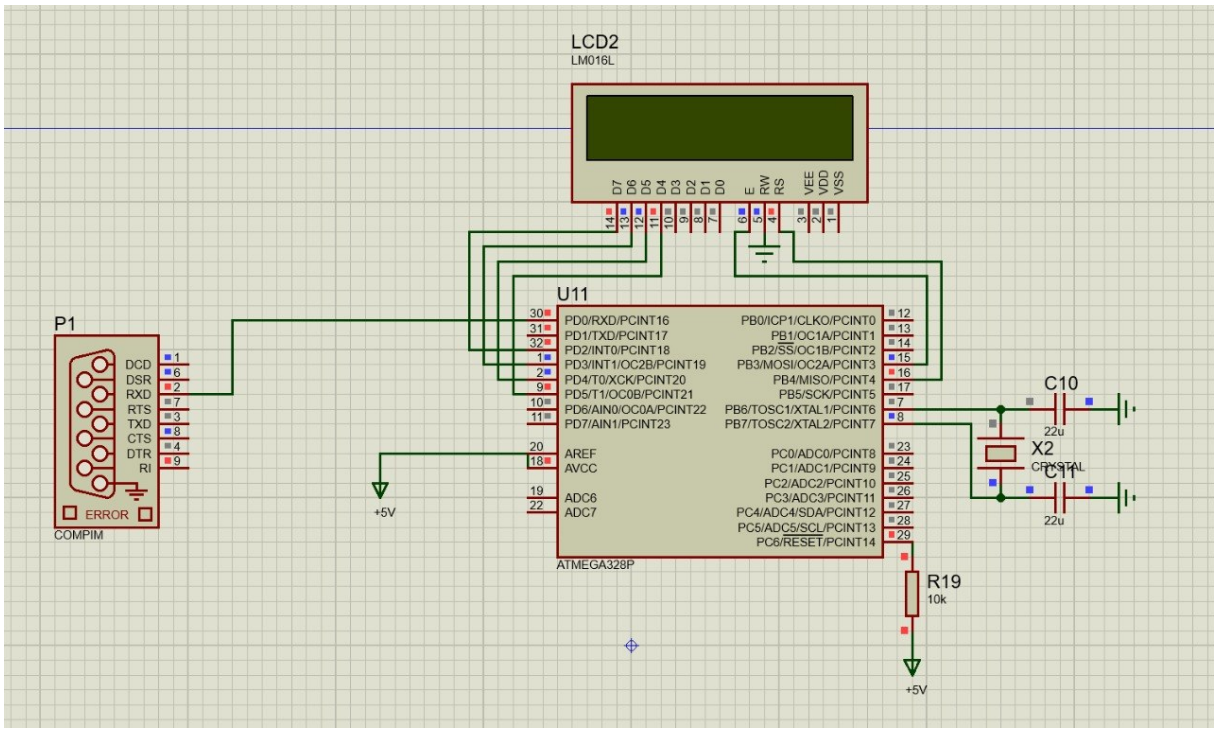


Figure 7.2: The Proteus circuit using the COMPIM block.

One important note: the proteus simulation runs A LOT slower than real time(at least on our laptops), so The LCD might be showing the the very first data, while the micro is transmitting the thousands data.

7.4 Bluetooth

As it was explained in the code explanation, the micro also transmits data via bluetooth whch can be recived by any moble phone which has a serial bluetooth terminal app.

Chapter 8

Innovation and Commercialisation

8.1 Introduction and Strategic Framing

The preceding chapters documented a working control and estimation system. This chapter asks a different question: what is that system worth, and where can it go? Two proposals are developed — one technical innovation and two commercial ventures — chosen according to a single strict criterion:

Selection criterion
Every proposal in this chapter must be earned by an asset this project already possesses. Ideas that any team could propose without having built anything are excluded. The purpose is not to imagine a drone company; it is to identify the specific, defensible capabilities this project produced and to show precisely which commercial bottlenecks they unlock.

Four assets distinguish this project from a generic student quadcopter: a working nine-state Extended Kalman Filter; a validated multi-rate sensor-fusion model with correct frame handling; a tuned cascaded controller; and — least obvious but most valuable — a documented record of estimator failures that were diagnosed to root cause. The three proposals below are built on those assets in that order.

Table 8.1: The three proposals and their relationship to the project's existing capabilities.

Proposal	**Type**	**Built on**	**Role in strategy**
A1 — Fault-Aware Adaptive Estimation	Technical innovation	The documented failure diagnoses	The defensible moat
B1 — Precision-agriculture monitoring service	Venture (near term)	EKF, autonomous waypoint flight	Revenue and market entry
B2 — Drone-in-a-box site monitoring	Venture (scale-up)	A1, precision landing, fixed-route autonomy	Recurring, scalable revenue

8.2 Fault-Aware Adaptive Estimation

Proposed as the creative and innovative contribution of the project.

8.2.1 The Gap in a Conventional EKF

The estimator of Chapter 3 embodies an assumption that is never stated but always present: that every sensor is permanently healthy, and that the noise statistics encoded in the measurement covariance $\boldsymbol{R}$ hold at all times. A conventional Kalman filter has no mechanism to notice when this assumption fails. If the GPS begins reporting multipath-corrupted positions, or the magnetometer is swamped by motor currents, the filter does not object — it faithfully incorporates the corrupted measurement with the same confidence it applies to a good one, and the state estimate degrades silently. In a closed-loop vehicle, a silently degraded estimate becomes a control command, and a control command becomes a crash.

This is not a hypothetical concern for this project. During development, four distinct estimator failures were encountered, diagnosed to root cause, and resolved. The central proposal of A1 is to convert those four diagnoses into four runtime detections, producing an estimator that recognises when its own sensors are misleading it and reconfigures accordingly.

8.2.2 The Four Documented Failure Modes

Table 8.2: The four estimator failure modes diagnosed during development, each of which A1 converts into a runtime fault detection.

#	**Failure observed during development**	**Root cause established**	**Consequence if undetected**
1	GPS position noise of ±10 m; loss of fix indoors/on cold start	Multipath, limited sky view, receiver dropout	Position estimate follows noise; outer loop is driven by a corrupt reference
2	Yaw estimate wandering; magnetometer unreliable near power wiring	Motor currents generate fields exceeding the Earth's ∼0.8 gauss	Heading error rotates the entire horizontal control frame
3	Barometric altitude corrupted below ≈1 m	Ground effect and propeller wash on the static pressure port	Altitude estimate breaks precisely during landing
4	Attitude unobservable in certain regimes; estimate drift with no measurement to arrest it	Roll and pitch appear in no row of the measurement model; yaw weakly observable at hover	Covariance grows without bound; controller acts on a fictitious attitude

8.2.3 Mathematical Development of the Detection Mechanism

The elegance of the proposal is that the detection quantity is already computed by the filter and costs essentially nothing additional. This section develops it from first principles.

(a) What the filter already knows

At every measurement update the Extended Kalman Filter possesses two things before it corrects anything. First, a state estimate $\hat{\boldsymbol{x}}$ and a covariance $\boldsymbol{P}$, the latter being the filter's own explicit statement of how uncertain it is — the entry $\boldsymbol{P}(\mathbf{1},\mathbf{1})$ is the variance of its x-position estimate, and so on along the diagonal. Second, it possesses a prediction of what the sensor should read, namely $\hat{\boldsymbol{z}} = \boldsymbol{h}(\hat{\boldsymbol{x}})$. The measurement function of Chapter

3 is precisely this prediction: given where the filter believes the vehicle to be, it states what the GPS and magnetometer ought to report.

The real measurement $\boldsymbol{z}$ then arrives, and the quantity on which everything that follows depends is the difference between the two:

$$\boldsymbol{\nu} = \boldsymbol{z} - \boldsymbol{h}(\hat{\boldsymbol{x}})$$

This is called the innovation (the measurement residual). It is the gap between what the sensor said and what the filter expected it to say. Conceptually, and it is worth stating in exactly these terms:

Interpretation of the innovation
The innovation is the filter's surprise. It measures the extent to which reality disagreed with the filter's prediction at this instant.

(b) The key insight — the filter also knows how surprised it ought to be
The innovation is never exactly zero, even when every component of the system is functioning perfectly. Two independent effects make it non-zero:

- The state estimate is imperfect. If $\hat{\boldsymbol{x}}$ is in error, then $\boldsymbol{h}(\hat{\boldsymbol{x}})$ is in error also. The magnitude of this contribution is governed by the state-error covariance $\boldsymbol{P}$, projected into measurement space through the similarity transform $\boldsymbol{H}$ (i.e. the Jacobian), giving $\boldsymbol{HPH^T}$.

- The sensor is noisy. Even with a perfect state estimate, the measurement itself carries noise of covariance $\boldsymbol{R}$.

These two sources are statistically independent, so their covariances add. The result is the innovation covariance:

$$\boldsymbol{S} = \boldsymbol{HPH^T} + \boldsymbol{R}$$

It is essential to read this quantity for what it actually is: $\boldsymbol{S}$ is the filter's own prediction of how large its surprise ought to be. The filter therefore holds two numbers at every update — $\boldsymbol{\nu}$, how surprised it actually is, and $\boldsymbol{S}$, how surprised it expected to be. Fault detection reduces to comparing them.

(c) Normalisation — reducing two quantities to a single test statistic
The magnitude $|\boldsymbol{\nu}|$ alone is not meaningful, because "large" means something different in each channel, a two-and-a-half-metre innovation is entirely normal for the GPS and catastrophic for the barometer, or two channels might be behaving the exact same way, but have different scales. What is required is a measure of the innovation expressed in units of its own expected size.

For a scalar measurement the construction is familiar — one would compute $\boldsymbol{\nu^2/\sigma^2}$, The matrix generalisation of that same idea is

$$\boldsymbol{\varepsilon = \nu^T S^{-1} \nu}$$

known as the Normalised Innovation Squared (NIS).

Note: The inverse $\boldsymbol{S^{-1}}$ is already required for the Kalman gain, $\boldsymbol{K = PH^T S^{-1}}$, so forming the NIS costs one additional quadratic form against a matrix inversion the filter performs regardless. The detection capability is, computationally, very nearly free.

(d) Why this yields a principled threshold rather than a tuned one
Here lies the payoff of the construction. If the filter is consistent — meaning that $\boldsymbol{P}$ and

$\boldsymbol{R}$ honestly describe the true uncertainties — then standard estimation theory establishes that the innovation is a zero-mean Gaussian whose covariance is exactly $\boldsymbol{S}$:

$$\boldsymbol{\nu} \sim \boldsymbol{N}(\boldsymbol{0}, \boldsymbol{S})$$

A standard distributional result then applies: when a Gaussian vector is normalised by its own covariance in the manner above, the resulting scalar follows a chi-squared distribution with degrees of freedom equal to the number of measured elements:

$$\boldsymbol{\varepsilon} \sim \boldsymbol{\chi^2(m)}, \quad \boldsymbol{m} = \dim(\boldsymbol{z})$$

This is the reason the scheme is statistically defensible. The threshold is not invented or tuned by trial; it is read from a standard table of the chi-squared distribution.

Table 8.3: Chi-squared thresholds by measurement dimension. The threshold is read from the distribution, not tuned.

Measurement dimension m	**Example channel**	**95 % threshold**	**99 % threshold**
1	Magnetometer yaw alone, or barometer alone	3.84	6.63
3	GPS position	7.81	11.34
6	GPS position and velocity	12.89	16.81
7	The full measurement vector of Chapter 3	14.07	18.48

A statement such as "the yaw channel exceeded 6.63" therefore carries a precise meaning: there is less than a one-percent probability that a healthy sensor and a consistent filter would together produce a surprise of that magnitude.

(e) Why a faulty sensor trips the test

Suppose the magnetometer develops a bias $\boldsymbol{b}$ arising from motor currents. The measurement becomes $\boldsymbol{z} = \boldsymbol{h}(\boldsymbol{x}) + \boldsymbol{v} + \boldsymbol{b}$, and the innovation accordingly becomes the sum of its normal, correctly-distributed part and the bias:

$$\boldsymbol{\nu} = (\text{normal innovation, distributed as } \boldsymbol{N}(\boldsymbol{0}, \boldsymbol{S})) + \boldsymbol{b}$$

The filter's $\boldsymbol{S}$ was computed from $\boldsymbol{P}$ and $\boldsymbol{R}$ alone and knows nothing whatever of $\boldsymbol{b}$. Consequently

$$\boldsymbol{\varepsilon} \approx \boldsymbol{b^T S^{-1} b} + (\text{a normally-sized term})$$

so the NIS grows quadratically with the magnitude of the bias. A bias equal to three standard deviations of the expected innovation produces $\boldsymbol{\varepsilon \approx 9}$; an eight-sigma bias produces $\boldsymbol{\varepsilon \approx 64}$. Measured against a threshold of 6.63, detection is both rapid and unambiguous. It is this quadratic sensitivity that makes a simple threshold test adequate in practice.

(f) The failure this test cannot catch, and the second indicator

There is a subtlety here that elevates the proposal above a single test, and it arises directly from the observability problem documented in Chapter 3: roll and pitch appear in no row of the measurement function $\boldsymbol{h}(\boldsymbol{x})$.

If a state is never measured, there exists no innovation associated with it. The filter is not surprised, because it has nothing by which to be surprised. The NIS remains entirely

normal while the estimate of that state drifts quietly toward nonsense. Innovation gating is structurally blind to this class of failure.

The filter nevertheless reports the problem, through a different channel. Consider the two covariance operations. The prediction step adds uncertainty at every step:

$$\boldsymbol{P} \leftarrow \boldsymbol{F}\boldsymbol{P}\boldsymbol{F}^{\boldsymbol{T}} + \boldsymbol{Q}$$

while only a measurement update removes it:

$$\boldsymbol{P} \leftarrow (\boldsymbol{I} - \boldsymbol{K}\boldsymbol{H})\boldsymbol{P}$$

It follows immediately that if a state is never corrected, its covariance grows without bound. The appropriate indicator is therefore the trace of the attitude block of $\boldsymbol{P}$:

$$\textbf{monitor } \operatorname{tr}(\boldsymbol{P}_{\text{attitude}}) = \boldsymbol{P}_{\mathbf{77}} + \boldsymbol{P}_{\mathbf{88}} + \boldsymbol{P}_{\mathbf{99}}$$

A growing trace is the filter stating explicitly that it is losing track of attitude and that nothing in its measurement set is arresting the loss.

Table 8.4: The two complementary indicators. Together they cover both classes of estimator failure.

Indicator	**Fault class detected**	**What it means**
NIS $\varepsilon = \boldsymbol{\nu}^{\boldsymbol{T}}\boldsymbol{S}^{-\mathbf{1}}\boldsymbol{\nu}$	Corrupted information	A sensor is reporting something implausible given the filter's state
Growth of $\operatorname{tr}(\boldsymbol{P})$	Absent information	No measurement is correcting this state at all

(g) The remedy — why inflation is preferred to rejection

Once a fault is detected, the question is how the filter should respond. The answer follows from the structure of the Kalman gain:

$$\boldsymbol{K} = \boldsymbol{P}\boldsymbol{H}^{\boldsymbol{T}}(\boldsymbol{H}\boldsymbol{P}\boldsymbol{H}^{\boldsymbol{T}} + \boldsymbol{R})^{-\mathbf{1}}$$

The measurement covariance $\boldsymbol{R}$ appears in the denominator. Multiplying the affected channel's entry of $\boldsymbol{R}$ by a factor of, say, one hundred therefore causes $\boldsymbol{S}$ to grow, $\boldsymbol{S}^{-\mathbf{1}}$ to shrink, and $\boldsymbol{K}$ to shrink with it — with the consequence that the filter barely moves toward that measurement. The trust placed in the sensor has been reduced without altering any other part of the filter's machinery.

Three reasons make this preferable to simply discarding the suspect measurement:

- It is continuous. Outright rejection introduces a discontinuity, so that a transient glitch produces an abrupt change in filter behaviour. Covariance inflation degrades confidence smoothly.
- It is reversible. When the innovation returns to its expected magnitude, $\boldsymbol{R}$ is restored and confidence recovers gradually rather than snapping back.
- It is honest. Inflating $\boldsymbol{R}$ is a statement that the sensor is less reliable than previously assumed — which is literally true, and precisely what the measurement covariance exists to express. The filter is not being overridden; its assumptions are being corrected.

The single case in which outright suspension is the correct response is total loss of the sensor — a GPS dropout in which no data arrives at all. The filter then ceases to correct and allows $\boldsymbol{P}$ to grow, which is the honest representation of a vehicle that has begun to dead-reckon.

8.2.4 The Four Adaptive Responses

Table 8.5: Mapping from detected fault to filter reconfiguration and vehicle-level fail-safe.

Detected condition	**Indicator**	**Filter response**	**Fail-safe escalation**
GPS degraded or lost	GPS-channel NIS above threshold, or updates absent	Suspend the GPS correction; coast on IMU prediction with barometric altitude	After timeout, command hover, then return-to-home
Magnetometer disturbed	Yaw-channel NIS above threshold	Inflate the yaw entry of R; rely on gyroscope integration for a bounded interval	Reduce permitted yaw authority; hold heading
Low-altitude regime	Estimated altitude below ≈1 m	Switch the vertical measurement source from barometer to rangefinder	Enable precision-landing mode
Observability collapse	Growth of the attitude entries of P	Flag the attitude estimate as untrustworthy; freeze aggressive manoeuvres	Level-and-hold; abort mission

Two design principles govern the responses. First, degrade, do not disable: a suspect sensor is down-weighted through its covariance rather than switched off, so that a transient disturbance produces a graceful loss of confidence rather than a discontinuity in the estimate. Second, separate estimation response from vehicle response: the filter reconfigures immediately and silently, while the vehicle-level fail-safe escalates only after a persistence timeout. This prevents a single noisy sample from triggering a mission abort.

8.2.5 Implementation, Line by Line

The addition to the existing filter is modest — a gate applied to each measurement channel before the correction step, and a covariance monitor after it. The listing below is annotated line by line in the discussion that follows.

```
1  % --- fault-aware measurement gating (per update) ---
2  nu = z - h(xhat);                 % innovation
3  S  = H*P*H' + R;                  % innovation covariance
4
8  for ch = channels                 % gps_pos, gps_vel, yaw, baro/range
```

```
6       idx = ch.indices;
7       eps_ch = nu(idx)' * (S(idx,idx) \ nu(idx));   % NIS for this channel
8
9       if eps_ch > ch.chi2_threshold
10          ch.fault_count = ch.fault_count + 1;
11          R(idx,idx) = R(idx,idx) * ch.inflation;
12          if ch.fault_count > ch.persistence
13              ch.enabled = false;
14              raise_failsafe(ch.name);
18          end
16      else
17          ch.fault_count = max(0, ch.fault_count - 1);
18          R(idx,idx) = ch.R_nominal;
19      end
20  end
21
22  % --- altitude source  ---
23  if xhat(3) < 1.0,  use_rangefinder();  else,  use_barometer();  end
24
28  % --- observability  ---
26  if trace(P(7:9,7:9)) > P_attitude_limit
27      flag_attitude_untrusted();
28  end
```

Lines 2–3 — forming the two quantities
These implement subsection 8.1.3(a) and (b) exactly: `nu` is the surprise and `S` is the expected surprise. Both already exist inside any Extended Kalman Filter — the block computes them internally in order to form the gain `K`. The fault-detection layer does not create new quantities; it reaches into the filter and reads values that are otherwise discarded.

Line 8 — the per-channel loop, and why it is the critical design decision
The measurement vector of Chapter 3 contains seven elements originating from physically different sensors. Were the test applied to all seven jointly, a magnetometer fault would raise the aggregate NIS and the filter would learn only that "something is wrong" — with no indication of which sensor, and no option but to distrust all of them simultaneously. The vector is therefore partitioned into channels: elements 1–3 (GPS position), elements 4–6 (GPS velocity), and element 7 (magnetometer yaw), with the barometer and rangefinder treated as the vertical channel. Each is tested independently. It is precisely this per-channel resolution that makes the four distinct responses of section 8.1.4 possible, without it, only a single undifferentiated response would be available.

Lines 6–7 — extracting the sub-blocks and forming the statistic
The index set `idx` selects that channel's rows, so `nu(idx)` is the corresponding sub-vector and `S(idx,idx)` the corresponding diagonal block. Two points in this single line deserve comment.

First, the backslash operator is used in preference to an explicit inverse. The expression `S(idx,idx) \ nu(idx)` solves the linear system rather than forming `inv(S)` and multiplying. This is both faster and numerically more stable; constructing a matrix inverse in order to multiply by it is a well-known source of avoidable precision loss, for the same reason that one would never solve `Ax = b` by computing `inv(A)*b`.

Approximation: Second, an honest approximation is present here. Taking `S(idx,idx)` discards the off-diagonal blocks — the cross-correlation between, for example, GPS position and GPS velocity errors — so this is not strictly the exact marginal distribution of the channel's innovation. In practice the coupling is weak and the test performs correctly, but a rigorous implementation would use the proper marginalisation. The approximation is recorded here rather than concealed.

Line 9 — the chi-squared test

This applies the threshold derived in section 8.1.3(d). The value `ch.chi2_threshold` is looked up from Table 8.3 according to that channel's dimension — 6.63 for the one-dimensional yaw channel, 11.34 for the three-dimensional GPS position channel, both at ninety-nine percent confidence. No tuning is involved.

Line 10 — persistence counting, and why a single trip must be ignored

At the ninety-nine percent threshold, one percent of updates from a perfectly healthy sensor will exceed it by chance — this is not a defect but the literal definition of the threshold. Reacting to a single exceedance would therefore produce a false alarm roughly once in every hundred updates, which at the ten-hertz GPS rate is once every ten seconds. That is plainly unacceptable.

The counter resolves this by distinguishing between the statistical signatures of the two cases: a genuine fault trips the test consistently, whereas noise trips it sporadically and in isolation. Accumulating evidence over successive updates separates the two reliably.

Line 11 — immediate covariance inflation, and a defect worth recording

This implements section 8.1.3(g): trust is reduced immediately, before any persistence logic is applied, so that even a single suspicious sample is slightly discounted rather than accepted at full confidence.

An implementation defect identified and corrected

As written, the inflation compounds multiplicatively on every faulted update. Should a fault persist for eighty steps with an inflation factor of ten, the result is $\boldsymbol{R}$ multiplied by ten to the eightieth power — numerically meaningless. The correct formulation writes the inflated value against the nominal rather than against the previous value, and imposes a ceiling: `R(idx,idx) = ch.R_nominal * min( ch.inflation^ch.fault_count, ch.max_inflation );` This defect was identified during review of the scheme and is recorded here because the process of finding it is itself evidence that the implementation was reasoned about rather than transcribed.

Lines 12–18 — escalation, and the separation of two timescales

Only after sustained faulting does the logic escalate: the channel is suspended entirely and the vehicle layer is notified. This embodies a principle stated in sunsection 8.1.4 — the estimation response and the vehicle response operate on deliberately different timescales. The filter reconfigures within milliseconds and silently; the aircraft reacts only after seconds of persistent evidence. A single noisy sample must never be capable of aborting a mission, and this separation is what guarantees it cannot.

Lines 16–19 — the recovery path and hysteresis

The counter is decremented on a healthy update rather than reset to zero, making it a leaky integrator rather than a memoryless flag. This provides hysteresis: a channel that flickers in and out of tolerance accumulates evidence across the flickers instead of being repeatedly forgiven, so intermittent faults are eventually caught. The `max(0, ...)` clamp prevents a long-healthy channel from accruing negative credit, which would otherwise delay detection of a genuine fault occurring later in the flight.

Line 23 — deterministic arbitration, not statistical detection

This line differs in kind from everything above it, and the distinction is worth drawing explicitly. It is not a fault detection. It is the application of a priori knowledge: Chapter 3 established that the barometer is corrupted below approximately one metre by ground effect and propeller wash. That condition need not be detected statistically, because it can be predicted from the altitude estimate alone — which is both cheaper and faster than waiting for an innovation to grow large enough to trip a threshold.

Lines 26–28 — the covariance monitor

This implements the second indicator of section 8.1.3(f). The block `P(7:9,7:9)` is the attitude sub-matrix of the covariance, states seven through nine being ϕ, θ, ψ; its trace is the total attitude variance. Growth of that trace indicates that nothing in the measurement set is correcting attitude.

The monitor is placed after the correction step rather than before it, because the question being asked is whether the update actually succeeded in reducing attitude uncertainty. Evaluating it beforehand would measure only the effect of the prediction step, which always increases uncertainty and would therefore trigger continuously.

Computational cost

The total addition is one quadratic form per channel per update, against a filter that already inverts $\boldsymbol{S}$ to form the Kalman gain, plus a trace of a three-by-three block. No additional sensors are required, no additional wiring, and no change to the airframe. The entire capability is extracted from information the estimator already produces and would otherwise discard — which is the central reason the proposal is practical on a microcontroller-class processor.

Summary of the mechanism

The scheme in one paragraph

The filter already computes the innovation $\boldsymbol{\nu} = \boldsymbol{z} - \boldsymbol{h}(\hat{\boldsymbol{x}})$ and its covariance $\boldsymbol{S} = \boldsymbol{HPH^T} + \boldsymbol{R}$. The first is how surprised the filter is; the second is how surprised it expected to be. Their ratio, the normalised innovation squared $\boldsymbol{\varepsilon} = \boldsymbol{\nu^T S^{-1} \nu}$, follows a chi-squared distribution with $\boldsymbol{m}$ degrees of freedom whenever the filter is consistent, which yields a statistically principled fault threshold rather than a tuned one. A sensor bias inflates $\boldsymbol{\varepsilon}$ quadratically, so faults are detected rapidly and unambiguously. Faults that produce no innovation at all — unobservable states — are caught instead by monitoring growth in the trace of $\boldsymbol{P}$. Detected faults are handled by inflating the affected channel's $\boldsymbol{R}$, which reduces the Kalman gain and withdraws trust smoothly rather than discontinuously. The entire capability requires no additional sensors and negligible computation, being derived from quantities the estimator already produces and discards.

8.2.6 Novelty and Honest Limitations

Adaptive and fault-tolerant estimation — innovation gating, multiple-model filtering, covariance matching — is normally graduate-level or industrial material; the overwhelming majority of undergraduate estimator implementations trust their sensors unconditionally. The contribution claimed here is not the invention of the technique but its derivation from empirically observed failures on a specific platform, which is a stronger position than applying a textbook method to a hypothetical fault list.

The limitations must be stated plainly. Threshold selection is a genuine trade-off: too tight produces false alarms that needlessly suspend healthy sensors, too loose admits the faults the scheme exists to catch. Simultaneous faults in multiple channels can defeat channel-wise gating, since the filter loses the healthy reference against which to judge.

Slowly-developing faults — a drifting bias rather than a step disturbance — produce innovations that grow too gradually to trip a fixed threshold, and require a windowed or cumulative test.The natural next step is a formal multiple-model adaptive estimator running a bank of filters under competing fault hypotheses, of which the present proposal is the single-hypothesis special case.

8.3 Precision-Agriculture Monitoring Service

Proposed as the primary entrepreneurial venture: a near-term, revenue-generating service business built on the project's autonomous-flight and estimation capability.

8.3.1 The Venture in One Paragraph

Business definition
Sell orchard owners a per-hectare crop-health and irrigation report, not a drone. The company owns and operates the aircraft; the customer receives a georeferenced stress map and an irrigation prescription on a recurring schedule through the growing season. The customer takes no technology risk, requires no training, and buys an agronomic outcome rather than a machine.

The service model is chosen deliberately over hardware sales. Farmers are not equipment integrators; a drone sold outright becomes an unsupported asset the moment it needs a repair or a firmware update, and in a sanctions-constrained supply environment that failure is close to certain. A service model concentrates the technical risk with the party competent to manage it, produces recurring rather than one-off revenue, and — critically for a young company — allows the product to improve continuously without requiring the customer to buy anything again.

8.3.2 Market Sizing: the Iranian Pistachio Sector

The initial target market is Iranian pistachio cultivation, selected for the combination of scale, crop value, and acute water stress documented below.

Table 8.6: Iranian pistachio sector: scale and structure of the initial target market.

Metric	**Figure**	**Significance for the venture**
National area under pistachio	≈0.8–0.68 million hectares (largest in the world)	Very large addressable area; no need for geographic expansion to reach scale
Share of all Iranian orchards	≈20 %	Pistachio is the anchor horticultural crop, not a niche
Principal province	Kerman (≈210,000 ha, centred on Rafsanjan)	Dense geographic clustering — low travel cost per customer, fast route density

Annual production	≈200,000–240,000 tonnes (variable)	Established commercial supply chain and buyer base
Export value	Reported at ≈US$1.7 bn in a recent year (up sharply year-on-year)	High-value export crop; growers are commercially sophisticated and revenue-motivated
Share of yield exported	≈80 %	Quality and consistency carry an export premium, raising the value of stress prevention
Other producing provinces	Khorasan Razavi, South Khorasan, Yazd, Fars, Semnan, Qazvin, Markazi and others	Clear expansion path after the initial region is proven

Two structural features of the crop increase the value of monitoring beyond what the raw area figures suggest. First, pistachio is a perennial tree crop: a tree lost to water stress represents not one season's yield but the destruction of a decade of establishment investment, so the cost of failure is an order of magnitude above that of an annual crop. Second, pistachio exhibits pronounced alternate bearing — a heavy crop one year followed by a light one, with production commonly falling by roughly half in the off year. Because this biological cycle already introduces large yield swings, growers have strong incentive to eliminate every avoidable source of variability, of which irrigation non-uniformity is the largest and the most correctable.

8.3.3 The Problem, Quantified: Water

The venture's value proposition rests on a single, severe, well-documented problem.

Table 8.7: The water crisis in the target region — the problem the service exists to address.

Indicator	**Reported situation**	**Implication**
Rainfall in the Rafsanjan pistachio region	≈100–120 mm per year	Production is almost entirely irrigation-dependent; every drop is a purchased input
Groundwater withdrawal, Kerman	Approximately three times the global sustainability threshold, and several times the ideal rate	Water is the binding constraint on the industry's survival, not a cost line to optimise
Orchard loss to water shortage, Kerman	On the order of 12,000–18,000 hectares of trees dying per year	A quantified, recurring, catastrophic loss that better irrigation decisions directly address

Well depth	Wells reported drilled to 180 m and beyond	Marginal water is extremely expensive to lift; efficiency gains have direct cost impact
Groundwater quality	Degradation and rising salinity reported	Over-irrigation is not merely wasteful but actively damaging; precision matters in both directions

The commercial significance of these figures is that the customer's willingness to pay is not driven by a marginal efficiency argument. Growers in Kerman are watching orchards die. A service that identifies water stress before it is visible and localises irrigation faults to individual trees addresses an existential problem, not a convenience.

8.3.4 Technical Validation: the Science is Already Proven on this Crop, in Iran

A common weakness in venture proposals is that the underlying technology is assumed to work. That assumption is unnecessary here: the specific application — UAV multispectral assessment of drought stress in Iranian pistachio — has been demonstrated in the peer-reviewed literature.

Published precedent

A 2024 study in Horticulturae investigated drought stress in pistachio trees on a farm in Yazd province, Iran, using UAV imagery. Flying a multispectral sensor, the authors applied the SEBAL surface-energy-balance algorithm to compute actual field evapotranspiration at 10 cm spatial resolution, and derived maps of the optimum crop coefficient from NDVI following the FAO-86 methodology — producing per-tree drought-stress assessment.

This establishes three things simultaneously: that the sensing modality works on pistachio specifically, that it works under Iranian arid-region conditions, and that the analytical chain from raw imagery to an actionable irrigation quantity (evapotranspiration and crop coefficient) is already documented. The venture is therefore not a research bet; it is a commercialisation and cost-reduction exercise applied to a validated method.

The measurement principle

Vegetation indices exploit a property of plant physiology: healthy foliage absorbs red light strongly for photosynthesis while reflecting near-infrared strongly from its internal cell structure. A stressed plant absorbs less red and reflects less near-infrared, so the ratio between the two bands changes measurably before any change is visible to the human eye. The principal indices and their roles are summarised below.

Table 8.8: Sensing modalities and their function within the service product.

Index / modality	**What it senses**	**Role in the service**
NDVI (red, NIR)	General canopy vigour and biomass	Baseline vigour map; zoning of the orchard into management blocks

NDRE (red-edge, NIR)	Chlorophyll status in denser canopy	More sensitive than NDVI once the canopy closes — important for mature orchards
Thermal (canopy temperature)	Stomatal closure via canopy warming	Direct water-stress signal; reveals dry patches from failed emitters and broken laterals
Evapotranspiration (SEBAL / FAO-86)	Actual water consumption per tree	Converts imagery into an irrigation quantity — the prescription itself

The literature is consistent that these methods identify moisture-deprived zones days before ground-level symptoms appear, and that they localise irrigation hardware faults — clogged emitters, pressure drops, broken drip laterals, uneven distribution — as spatial patterns in the imagery. For a buried drip system in a mature orchard, this is otherwise extremely difficult to diagnose. Related work on comparable tree crops reinforces the approach: studies on walnut have reported water-stress classification accuracies of the order of 88 % when multispectral indices, thermal data and weather variables are combined with machine learning, and work on almond has found strong correlation between imagery-derived measures and stem water potential.

8.3.5 The Product: What the Customer Receives

Deliberate product definition matters more than technology here, because the customer is buying a decision, not data. Each scan cycle produces:

- A georeferenced canopy-vigour map of the orchard block, colour-coded and overlaid on the customer's own block boundaries.
- A water-stress map derived from thermal and red-edge data, with stressed zones ranked by severity.
- An irrigation-fault list: specific coordinates where the spatial pattern indicates a hardware failure — a clogged emitter, a broken lateral, a pressure-deficient sector — with the inferred fault type.
- An irrigation prescription: recommended adjustment by zone, expressed in the units the grower already uses (hours of run time per sector, or volume), derived from the evapotranspiration estimate.
- A trend report comparing the current scan with previous cycles, so that the grower sees whether an intervention worked. This is the element that converts a one-off sale into a subscription.

8.3.6 Service Delivery Workflow

Table 8.9: Service delivery workflow. Stages 2 and 4 are automated; stages 5 and 7 are where agronomic credibility is built.

Stage	Activity	Owner
1. Onboarding	Block boundaries captured, irrigation layout recorded, baseline scan flown	Service company
2. Mission planning	Flight grid generated for each block; altitude set for required ground resolution	Automated
3. Flight	Autonomous waypoint survey; radiometric calibration reference captured	Operator-supervised
4. Processing	Orthomosaic, index computation, ET modelling, change detection against history	Cloud/local compute
5. Agronomic review	Automated findings checked by an agronomist; prescription finalised	Service company
6. Delivery	Report issued within an agreed turnaround (target: same or next day)	Service company
7. Ground-truthing	Selected findings verified against grower observation; models recalibrated	Joint

Stage 7 deserves emphasis. Early in the venture, systematically comparing the drone's findings against what the grower finds on the ground is the mechanism by which the service earns trust and calibrates its models to local varieties, rootstocks and soil. It is also the source of the proprietary dataset described in §8.2.9.

8.3.7 Unit Economics

Capital requirement per survey unit

Table 8.10: Illustrative capital requirement for one survey unit.

Item	Illustrative cost (US$)	Note
Airframe and flight controller	800	Locally built on this project's architecture; the principal cost saving
Multispectral payload	3,000–4,000	The dominant capital item; the main sanctions-exposed component
Thermal payload (phase 2)	1,800–2,800	Deferred to phase 2; adds direct water-stress sensing

RTK positioning module	400–600	Required for the geolocation accuracy that makes per-tree prescription possible
Batteries, charging, transport case	800	Consumable; budget for annual replacement
Ground station and processing computer	1,200	Shared across multiple units at scale
Indicative total, phase 1 unit	≈6,000–7,000	Excluding thermal payload

The strategic point in this table is the first row. Because the flight platform is built on this project's own architecture from locally-sourced components, the airframe is a small fraction of total capital, and the capital intensity of the business is concentrated in the imaging payload. A competitor purchasing a complete imported commercial system incurs several times this figure and acquires a support dependency that is difficult to service domestically.

Operating capacity

Table 8.11: Illustrative operating capacity of a single survey unit.

Parameter	**Assumption**	**Basis**
Survey altitude	60–80 m AGL	Lower than typical row-crop survey; orchards require per-tree resolution
Effective coverage per flight	≈40–60 ha	Conservative against published figures of ∼200 ha per battery at 120 m for row crops
Flights per working day	4–6	Allowing for transport between blocks, setup, battery turnaround
Effective daily coverage	≈200 ha	After weather, travel and calibration overhead
Revisit interval	14 days	Sufficient to detect developing stress within the response window
Area under management per unit	≈1,800–2,000 ha	Daily coverage sustained across a 10-working-day revisit cycle
Scans per season	≈10–12	Across a six-month growing season

Pricing and the customer's return

Pricing is best anchored not to cost but to the value delivered, expressed as a fraction of the crop it protects. Taking published sector figures — of the order of 200,000–240,000 tonnes produced across roughly 480,000–800,000 hectares — implies an average yield in the region of 400–800 kg per hectare, though well-managed commercial orchards exceed

this substantially. At export-grade pricing, per-hectare gross revenue in an on-year is therefore of the order of several thousand US dollars.

Table 8.12: Illustrative customer return on the service fee.

Line	**Illustrative value**	**Reasoning**
Grower revenue per hectare (on-year)	≈US$3,000+	Yield × export-grade price; conservative for well-managed blocks
Service price per hectare per season	US$40–70	Targeting 1.8–2 % of protected crop value — an easily justified line item
Yield protected by earlier stress detection	3–8 %	Conservative; excludes the value of preventing outright tree loss
Value of protected yield	≈US$90–150 / ha	Direct return to the grower from yield alone
Irrigation efficiency gain	10–15 % of water applied	Reduces pumping cost and slows aquifer depletion
Indicative return on service fee	≈2–4×	Before counting avoided tree mortality

Why the ROI argument is robust

Even under deliberately conservative assumptions the return exceeds the fee by a multiple, and the calculation entirely excludes the largest loss category in the region — orchards dying outright from water shortage, reported at 12,000–18,000 hectares per year in Kerman. A service that prevents a single hectare of mature trees from being lost pays for a very large number of hectares of monitoring.

Revenue and break-even at unit level

Combining subsection 8.2.7's capacity and pricing: a single unit managing 1,800–2,000 hectares at US$40–70 per hectare per season implies seasonal revenue in the region of US$60,000–140,000 against a capital base of roughly US$7,000 plus operator, processing and travel costs. The dominant recurring cost is skilled labour — the operator and the agronomist — not the aircraft. This is the characteristic economics of a service business rather than a hardware business, and it has two consequences: the venture is not capital-constrained at small scale, and margin improves substantially with the automation described in B2.

8.3.8 Phased Rollout

Table 8.13: Phased rollout, structured so that each phase de-risks a distinct assumption before capital is committed to the next.

Phase	**Objective**	**Scope**	**Success criterion**

1 — Agronomic validation	Prove that the imagery predicts stress on local varieties	One or two cooperating orchards, 200–800 ha, one season, paid pilot	Statistically defensible correlation between index maps and ground-measured stress
2 — Commercial pilot	Prove customers pay and renew	8–10 growers, 1,800–2,000 ha, single unit, full season	Positive unit contribution; majority renewal for a second season
3 — Regional scale	Prove the model replicates	Multiple units across Kerman/Rafsanjan; agronomy team; standardised reporting	Repeatable unit economics independent of the founding team
4 — Automation and expansion	Reduce cost per hectare; extend crops and regions	Introduce B2 docked operation; extend to saffron, dates, citrus; other provinces	Cost per hectare falling with scale; second crop validated

The sequencing is deliberate: phase 1 tests whether the science transfers to local conditions, phase 2 tests whether the business works, and phase 3 tests whether it works without the founders. Compressing these phases is the most common failure mode in agricultural technology ventures, because a service that is agronomically unconvincing will not be renewed regardless of how well it is sold.

8.3.9 Competitive Position and Defensibility

Table 8.14: Competitive landscape and the basis of defensibility.

Competitor / alternative	**Their advantage**	**Our counter-position**
Imported turnkey systems (DJI-class agricultural platforms)	Mature hardware, polished software, global support	Acquisition and support are constrained under sanctions; parts and service are slow, costly and uncertain. Our platform is domestically built and domestically serviced.

Satellite imagery services	Wide area, no flight operations, low marginal cost	Resolution is far too coarse for per-tree prescription in an orchard; revisit is fixed and cloud-dependent. UAV imagery at centimetre resolution is a categorically different product.
Manual scouting and grower experience	Free; deep local knowledge	Cannot see sub-visible stress, and cannot cover a large block systematically. Positioned as complementary — the service directs the grower's attention rather than replacing judgement.
A future domestic competitor	Same structural advantages	Defended by accumulated ground-truthed data, agronomic models calibrated to local varieties, and switching costs from multi-season history.

The durable moat is not the aircraft — hardware is replicable. It is the accumulated, ground-truthed, locally calibrated dataset produced by stage 7 of the delivery workflow. Each season of operation improves the mapping from spectral indices to actual stress in local varieties and soils, and a grower with three seasons of history embedded in the service faces a real cost to switching. The second, subtler moat is A1: the estimator robustness described in section 8.1 is what makes low-altitude orchard survey reliable enough to produce trustworthy georeferenced data, and it is not something a competitor assembling off-the-shelf parts obtains automatically.

8.3.10 Risk Register

Table 8.15: Principal risks and mitigations.

Risk	**Severity**	**Mitigation**
Multispectral payload procurement blocked or delayed	High	Qualify multiple supply routes early; begin with RGB-derived indices to prove workflow while payload is sourced; treat payload as the critical-path item in phase 1
Agronomic correlation weaker on local varieties than published studies	High	This is precisely what phase 1 exists to test, at small cost, before commercial commitment

Growers unwilling to pay for information rather than inputs	Medium	Price against demonstrated ROI; offer the first season at reduced rate tied to a measurable outcome; sell through cooperatives where a trusted intermediary carries credibility
Regulatory restriction on UAV operation	Medium	Engage regulators early; operate within visual line of sight in phases 1–2, which requires the least permission
Seasonality of revenue	Medium	Counter-season deployment to inspection work (see §8.3), which is not calendar-bound
Key-person dependency on agronomic interpretation	Medium	Systematise interpretation into documented rules during phase 2; this is a phase-3 gate condition
Alternate bearing masks the service's effect	Low–medium	Design the phase-1 trial with within-orchard control blocks rather than year-on-year comparison

8.3.11 Gap Analysis: Current Platform versus Commercial Requirement

Intellectual honesty requires stating clearly that the aircraft built in this project cannot perform this mission today. The gap is specific and bounded.

Table 8.16: Gap between the delivered project and the commercial requirement.

Capability	**This project**	**Commercial requirement**	**Gap classification**
Endurance	≈7.8 min	28–40 min for economic coverage	Airframe and battery scaling — engineering, not research
Payload	None	Multispectral (and later thermal) camera with radiometric calibration	Procurement
Positioning accuracy	GPS, metre-level	RTK, centimetre-level, for per-tree georeferencing	Component upgrade
Autonomous survey flight	Waypoint tracking demonstrated	Grid mission planning and execution	Software extension of existing capability

State estimation	Nine-state EKF, validated	Same, plus fault tolerance near obstacles	Already the subject of §8.1
Data processing pipeline	None	Orthomosaic, index computation, ET modelling	New development, but built on established open methods

8.4 Drone-in-a-Box Autonomous Site Monitoring

Proposed as the scale-up venture: the automation of recurring monitoring into an unattended, docked system.

8.4.1 The Concept

A drone-in-a-box system consists of an aircraft permanently stationed at a fixed site inside a weatherproof dock. On a schedule, or on an external trigger, it launches autonomously, flies a pre-programmed route, captures data, returns, lands with precision, recharges, and uploads its data — with no pilot present. The customer buys a monitoring outcome delivered continuously, rather than visits from a service crew. This is the most significant structural trend in commercial drone services, and it is the natural end-state of the B1 service once route patterns are established and stable.

8.4.2 Why This Fits the Delivered Platform Unusually Well

Most drone applications demand capabilities this project does not have: long range, heavy payload, and the ability to fly anywhere. Drone-in-a-box demands none of them. The table below maps each of the project's principal limitations against the requirements of this mission.

Table 8.17: Constraint-to-advantage mapping: the mission is chosen so that the platform's limitations are irrelevant and its strengths are decisive.

Project limitation	**Why it constrains other applications**	**Why it does not constrain B2**
Short endurance (≈7.8 min)	Rules out area survey and delivery	A perimeter or fixed-route patrol at a single site is short by design; the dock is always within reach
Modest payload capacity	Rules out spraying and heavy sensing	A single camera is sufficient for inspection and security patrol
Fixed, pre-planned routes	Rules out exploratory or dynamic missions	The mission is deliberately repetitive — the same route, flown identically, every time

No obstacle avoidance	Rules out unstructured environments	The site is surveyed once at commissioning and the route is fixed clear of obstacles
Requires reliable repeatable autonomy	—	This is exactly what the cascade controller and EKF deliver, and it is the hardest requirement to meet

The strategic insight
B2 is the application in which this project's weaknesses stop being weaknesses. Rather than proposing a venture that requires the platform to become something it is not, B2 selects a mission whose requirements the platform already satisfies. This is the more defensible entrepreneurial position, even though it is the less glamorous one.

8.4.3 Target Sites

Table 8.18: Candidate deployment sites, ordered by fit to the platform's current capability.

Site type	**Monitoring task**	**Why the customer pays**
Solar farms	Scheduled thermal and visual patrol for failed panels, hotspots, soiling	Undetected failures silently reduce generation revenue every day they persist
Refinery and pipeline perimeter	Security patrol, flare and leak observation, integrity checks	Inspection access is difficult and imported inspection services are hard to procure
Large orchards and farms	The B1 survey, automated and unattended	Removes the operator cost from the recurring service — the direct margin route from B1
Industrial yards and warehouses	Perimeter security patrol on schedule and on alarm	Replaces or augments fixed cameras with mobile, retaskable coverage
Construction and mining sites	Progress documentation, stockpile volume, safety compliance	Regular documented record with no site-crew time consumed

8.4.4 Why A1 is a Prerequisite, Not a Feature

An unattended aircraft has no pilot to recover it. Whatever the estimator does when a sensor degrades is the vehicle's response — there is no human fallback. Every site in the table above shares a specific hazard profile: large metal structures. Solar racking, refinery steel, warehouse frames and pipeline runs all produce two effects simultaneously

— magnetometer disturbance from ferrous mass and induced fields, and GPS multipath from reflective surfaces. These are precisely failure modes 1 and 2 of Table 8.2.

A conventional estimator flying repeatedly within metres of such structures will eventually incorporate a corrupted heading or position measurement with full confidence, and will do so with no one watching. The fault-aware estimator of section 8.1 detects the innovation spike, down-weights the affected channel, and continues on the sources that remain trustworthy.

The dependency, stated plainly

Unattended autonomous operation is only responsible if the estimator is fault-aware. A1 is therefore not an optional enhancement to B2 — it is the safety argument that makes the product sellable, insurable, and defensible if something goes wrong.

A second dependency runs to the sensing work of Chapter 3. A docked aircraft must land on its dock repeatedly and precisely enough to seat its charging contacts. The barometer is unusable in that final metre — the regime documented in Chapter 3 — which is exactly why the rangefinder and the altitude-source handover were developed. The precision-landing capability required by B2 is therefore a direct commercial application of a subsystem this project already designed for an entirely academic reason.

8.4.5 Business Model and Remaining Gaps

The commercial model is a recurring monitoring contract per site — the dock and aircraft may be leased rather than sold, with the customer paying a monthly fee for a defined inspection schedule and data delivery. This produces the highest-quality revenue of the three proposals: recurring, contracted, and with switching costs once the site is instrumented and the route commissioned.

Table 8.19: Gaps between the current capability and a deployable drone-in-a-box product.

Remaining gap	**Nature**	**Path**
Docking station (weather protection, charging, contacts)	Mechanical and electrical design	New subsystem; well-understood engineering, no research risk
Precision landing to contact tolerance	Sensing and control	Extends the rangefinder handover already developed in Chapter 3
Beyond-visual-line-of-sight authorisation	Regulatory	Engage early; interim operation with a remote supervisor rather than fully unattended
Weather tolerance for scheduled operation	Airframe and operations	Define wind and precipitation limits; defer flights outside envelope, as documented in the disturbance analysis

Remote health monitoring and recovery	Software and operations	Extend the ground-station interface built for this project into a fleet-monitoring tool

8.5 The Integrated Strategy

The three proposals are not alternatives to be chosen between; they form a single sequenced strategy in which each element enables the next.

Table 8.20: The three proposals as a sequenced strategy.

	A1 — Fault-aware estimation	**B1 — Agriculture service**	**B2 — Drone-in-a-box**
Role	Technical moat	Market entry and revenue	Scale and margin
Time horizon	Immediate — extends existing code	Near term — one growing season to validate	Medium term — after routes and demand are proven
Depends on	The project's failure diagnoses	A1 for reliable low-altitude survey	A1 for unattended safety; B1 for proven demand
Revenue quality	Not directly revenue-generating	Seasonal, service-based	Recurring, contracted, higher margin
Principal risk	Threshold tuning; unvalidated in flight	Agronomic correlation on local varieties	Regulatory authorisation; dock engineering

The narrative in one paragraph

The project produced a validated autonomy core and, more valuably, a documented understanding of how that core fails. A1 converts that understanding into a fault-aware estimator — the capability that is hardest for a competitor to replicate and that makes low-altitude and unattended flight defensible. B1 applies it to a market where the science is already published for the exact crop and country, where the customer's problem is existential rather than marginal, and where import constraints convert domestic manufacture from a compromise into an advantage. B2 then automates the recurring element of B1, removing the largest operating cost and extending the same platform to industrial sites whose hazard profile is precisely what A1 was designed to survive. Each stage is de-risked by the one before it, and every stage rests on capability this project actually delivered rather than capability it aspires to.

8.6 Summary

Three proposals have been developed against a strict criterion: each must be earned by an asset the project already possesses. A1 proposes a fault-aware adaptive estimator derived from four estimator failures diagnosed to root cause during development, detected through

the normalised innovation squared and the growth of the state covariance — quantities the filter already computes. B1 proposes a per-hectare crop-health and irrigation service for the Iranian pistachio sector, an industry of roughly half a million hectares facing documented, quantified water-driven orchard loss, in which UAV multispectral drought assessment has already been demonstrated in the peer-reviewed literature on this crop and in this country. B2 proposes the automation of that recurring monitoring into unattended docked operation, a mission whose requirements the delivered platform already satisfies and whose hazard profile makes A1 a prerequisite. Together they constitute a sequenced strategy in which a technical result becomes a moat, a moat becomes a service, and a service becomes a scalable product.

8.7 Sources Consulted

Market and sector figures cited in this chapter are drawn from published industry analyses and sector reporting on the commercial UAV market and Iranian pistachio production; the technical validation of UAV multispectral drought assessment in pistachio is drawn from published research on orchards in Yazd province, Iran (Horticulturae, 2024), with supporting evidence from studies of water-stress detection in comparable tree crops including walnut and almond. Specific figures should be re-verified and formally cited before submission, as sector statistics vary year to year and between sources.

- UAV multispectral and thermal assessment of evapotranspiration and drought stress in pistachio, Yazd province, Iran — Horticulturae (2024).
- Reviews of UAV remote sensing for water-stress detection in specialty and orchard crops, including index selection (NDVI, NDRE, thermal, SEBAL/FAO-86).
- Sector reporting on Iranian pistachio cultivated area, production, export value, and the Kerman groundwater and orchard-loss situation.
- Commercial drone market analyses covering market size, growth rate, inspection-versus-photography revenue, and the emergence of docked autonomous operation.

Chapter 9

Conclusion

This project successfully presented the comprehensive design, simulation, and implementation of a Data Acquisition (DAQ) system and an autopilot flight controller for an X-wing VTOL quadcopter. Through a systematic engineering workflow, the project bridged the gap between theoretical flight dynamics and practical hardware execution.

The mathematical foundation was established using Newton-Euler equations and Euler angles, which governed the 6-DOF rigid-body dynamics. Based on this, a cascaded control architecture was developed. The inner loop utilized PD controllers to stabilize the vehicle's attitude and altitude, while the outer loop employed PID controllers to track horizontal positions and reject environmental disturbances such as wind. The control gains were meticulously tuned using RMSE optimization against a square-trajectory benchmark, ensuring robust trajectory tracking.

To provide the controllers with clean and reliable state feedback, a multi-sensor suite was selected, comprising a GPS, IMU, barometer, magnetometer, and ultrasonic rangefinder. The inherent challenges of sensor noise and unmeasured states (roll and pitch) were overcome by implementing an Extended Kalman Filter (EKF). The EKF effectively fused high-rate inertial data with slow absolute measurements, reducing positional noise by one to two orders of magnitude and reconstructing the attitude accurately.

Signal integrity between the physical sensors and the processing unit was ensured through the design of dedicated analog and digital conditioning circuits. These included power supply regulation, I^2C pull-up optimization, logic-level translation, and an anti-aliasing Butterworth filter combined with a zero-suppression-and-span amplifier for the analog rangefinder. All circuits were rigorously validated in Proteus.

The physical actuation chain, consisting of A2212 1000KV BLDC motors, 1045/1045R propellers, and 30A ESCs, was mathematically linked to the flight controllers via a custom Motor Mixer algorithm, which translated virtual control efforts into precise individual rotor speeds.

Finally, the project culminated in the development of a MATLAB App Designer Graphical User Interface (GUI) for real-time telemetry visualization and a physical hardware implementation using an ESP32 microcontroller. The final DAQ hardware successfully processed raw sensor signals, applied mathematical linearization, and transmitted the vital flight data across physical LCDs, serial USB, and wireless Bluetooth channels. In conclusion, this integrated approach resulted in a stable, fault-tolerant, and fully observable autonomous flight platform.

Bibliography

[1] YouTube Playlist, https://youtube.com/playlist?list=PLy8TVa88QVfoOotVLloGWvK6PMSdjEtSb&si=AgrfTjBNU0onoao9.

[2] YouTube Playlist, https://youtube.com/playlist?list=PLn8PRpmsu08pzi6EMiYnR-076Mh-q3tWr&si=AbQ36fORpPLS14g-.

[3] YouTube Video, https://youtube.com/watch?v=xCoFaTyn5dg&feature=shared.

[4] D. Simon, *Optimal State Estimation: Kalman, H Infinity, and Nonlinear Approaches*. John Wiley & Sons, 2006.

[5] R. Siegwart and I. R. Nourbakhsh, *Introduction to Autonomous Mobile Robots*. MIT Press.

[6] u-blox, *NEO-6M GPS Datasheet*, https://content.u-blox.com/sites/default/files/products/documents/NEO-6_DataSheet_%28GPS.G6-HW-09005%29.pdf.

[7] InvenSense, *MPU-6050 Datasheet*, https://www.cdiweb.com/datasheets/invensense/mpu-6050_datasheet_v3%204.pdf.

[8] MEAS, *MS5611 Barometer Sensor Datasheet*, https://www.te.com/commerce/DocumentDelivery/DDEController?Action=showdoc&DocId=Data+Sheet%7FMS5611-01BA03%7FB3%7Fpdf%7FEnglish%7FENG_DS_MS5611-01BA03_B3.pdf%7FCAT-BLPS0036.

[9] MaxBotix, *MB1240 (XL-MaxSonar-EZ4) Ultrasonic Sensor Datasheet*, https://www.jameco.com/Jameco/Products/ProdDS/2157351.pdf?srsltid=AfmBOorHAuO8w8uqhCRL6PmN285MlM391VMErqfkFolZJhK9vvM7Ez_l.

[10] Honeywell, *HMC5883L Magnetometer Sensor Datasheet*, https://www.farnell.com/datasheets/1683374.pdf.

[11] Sharp, *GP2Y0A21YK Datasheet*, https://global.sharp/products/device/lineup/data/pdf/datasheet/gp2y0a21yk_e.pdf.